

Aligned Fragments and Constrained Crossover

Undecidability, exact generation depth,
and rational growth of context-free recombination closures

Research draft prepared with ChatGPT
for Vladimir Reshetnikov

29 September 2026

Abstract

For equal-length words, constrained one-point crossover exchanges suffixes at the same position. We prove that deciding whether a context-free language satisfies $L \otimes_c L = L$ is co-recursively-enumerable-complete, already over a fixed three-letter alphabet. This answers the decision question explicitly left open in Section 4 of Hughes, arXiv:2608.27755v1. A delimiter amplification also proves undecidability of eventual finite stabilization, with an exact rank- $r + 1$ witness built from r omitted blocks.

The structural invariant is the least number $\rho_L(w)$ of consecutive fragments of w that occur at the same positions in seed words of the same length. For parallel self-crossover we prove $\rho_{L \otimes_c L}(w) = \lceil \rho_L(w)/2 \rceil$ and hence exact generation depth $\lceil \log_2 \rho_L(w) \rceil$. Rank spectra have no gaps, and every length- n slice stabilizes by $\lceil \log_2 n \rceil$, sharply.

For regular inputs, one-step closure has a polynomial-time DFA test and is PSPACE-complete for NFAs; finite stabilization reduces to the classical limitedness problem for distance automata. For every context-free seed, the full closure has an effectively rational commutative generating function. Nevertheless, an explicit binary linear context-free seed has a closure that is not context-free. Its finite generations have polynomial growth and minimal recurrence order $q2^k + 1$, whereas the limit has exponential growth. Complete proofs, exact finite audits, and a research agenda are included.

Status. Unrefereed research draft. The proofs are mathematical arguments, not Lean or Rocq certificates. A bounded literature search supports the connection to the stated open question, not a guarantee of first discovery for every structural lemma or consequence.

[write] A report of the ProveIt research-report collection (category automata-and-formal-languages, report constrained-crossover-closure), written from batch-44 manuscript 02; text marked [write] was added then. The sibling report on insertion-degree spectra is related in Section 1.4; provenance is in Appendix D. [Added 30 September 2026, batch 66: Part II (from Section 13) proves finite stabilization of regular seeds PSPACE-complete for NFA input; Part III (from Section 29) decides regularity of the full closure of context-free seeds and classifies sparse one-marker seeds. Both come from manuscripts of batch 66; their opening sections give their provenance.]

Contents

Part I. Aligned fragments and constrained crossover	4
1 Problem, provenance, and principal results	4
1.1 A specific question behind the repository	4
1.2 What the theorems say	4
1.3 Attribution and limits of the novelty claim	5
1.4 [write] Relation to the sibling report on insertion-degree spectra	5
1.5 [write] Notation beside the neighbouring texts	6
2 Conventions and the coordinate hull	6
3 Aligned fragments and exact generation depth	8
3.1 The fragment invariant	8
3.2 There are no missing finite ranks	9
3.3 Sharp stabilization bounds	9
3.4 The frozen-source convention is also exactly solvable	10
4 One-step equality is undecidable for context-free inputs	10
4.1 A guard construction with universal first generation	11
5 Finite stabilization and an exact block amplification	12
6 Polynomial-time rank computation for a fixed target	13
7 Regular inputs: finite-state tests and limitedness	14
7.1 One-step closure for deterministic automata	14
7.2 Encoding all fragment ranks by a distance automaton	15
8 Rational counting for the full closure of every CFL	16
8.1 Effective semilinear position supports	16
8.2 A rational commutative generating function	17
9 A linear context-free seed with a non-context-free closure	18
9.1 Definition, grammar, and position geometry	18
9.2 Every rank, every generation, and exact counts	20
9.3 Sharp recurrence orders of the finite generations	20
10 Verification, reproducibility, and proof boundaries	21
10.1 Executed exact audits	21
10.2 Files and rebuilding	22
10.3 What was not verified or claimed	23
11 Further research questions and a formalization plan	23
11.1 A realistic Lean development	25
12 Conclusion	26
Part II. Finite stabilization of regular seeds is PSPACE-complete	27
13 [write, batch 66] Provenance and notation of Part II	27
14 The question and the contribution	29

15 Crossover, aligned fragments, and the two clocks	30
16 The exact-length geometry of an NFA	32
17 Uniform periodicity and its quantitative cost	33
18 Periodic interiors and the obstruction dichotomy	34
19 Polynomial-space decision and finite obstruction bounds	37
20 Binary PSPACE-hardness	38
21 Deterministic automata and safe cores	40
22 Quantitative consequences and worked examples	41
23 Executable audits and their limits	43
24 Further research questions	45
25 A formalization and implementation roadmap	47
26 Conclusion	48
27 Proof and edge-case audit	48
28 Source ledger and novelty boundary	49
 Part III. Decidable regularity and exact stabilization of crossover closures	 50
29 [write, batch 66] Provenance and notation of Part III	50
30 Research target and relation to ProveIt	52
31 Coordinate supports and the full closure	54
32 Closure comparison and Boolean length theory	55
33 Decidable regularity through row types	56
34 One-marker seeds: exact generations	58
35 Sparse semilinear masks and arithmetic rays	59
36 A finite-ray pumping principle	60
37 The sparse-marker language-class trichotomy	61
38 An exact compatibility-graph formula	63
39 Exact-depth complexity for succinct regular seeds	64
40 Explicit examples and arbitrary finite depths	65
41 Verification, implementation, and proof status	67
42 Further research questions	69

43 Conclusion	70
44 Proof audit and boundary cases	70
45 Package and reproducibility	72
 Appendices to Part I	 74
A The classical universality input	74
B Why the positional counts are eventually affine	74
C A compact proof-dependency map	75
D [write] Provenance of this report	75

1 Problem, provenance, and principal results

1.1 A specific question behind the repository

The ProveIt report *Gaps in Bounded-Shuffle Hierarchies*, including its later binary-profile extension, studies insertion-degree spectra and cites Hughes’s recent work on language operations [2, 1]. The report distinguishes its spectrum results from the surrounding decision problems. We follow that connection, but study a different operation: *equal-length, aligned, one-point crossover*.

Hughes defines constrained crossover on page 2 and, on page 3, explicitly leaves the decidability of $L \otimes_c L = L$ open for context-free languages. The obstruction to his general theorem is precise: constrained crossover does not contain concatenation. Our reduction does not try to restore concatenation. Instead, it makes every omitted word obtainable by a single aligned repair using a fresh guard letter. The decision problem is thereby settled directly.

[write] *Where the question is posed.* Checked when this report was written (29 September 2026) against the 13-page PDF of arXiv:2608.27755v1, whose printed page numbers are its PDF page numbers. Hughes defines unconstrained crossover $\otimes_u$ and constrained crossover $\otimes_c$ (the same symbol $\otimes_c$ as here) in Section 2 (“Definitions and Notations”, p. 2). The question is posed in Section 4 (“Foundational Results”), under “Single step equality”. His Theorem 1 makes single-step equality $L \odot L = L$ at least as hard as universality for every operator $\odot$ with $L \cdot L \subseteq L \odot L$ (on a class of languages for which containment of every letter and of ε is decidable but universality is not), and its Corollary applies this to simple insertion, shuffle and unconstrained crossover. The note after the Corollary (p. 3) observes that constrained crossover fails this subsumption criterion, so “the complexity of the question, ‘does $L \otimes_c L = L$ ’, is open.” The list of open questions that follows repeats it as its second item, for a context-free L (top of p. 4). Theorem 4.2 answers that question: the problem is co-r.e.-complete (Π_1^0 -complete), hence undecidable, already over a fixed three-letter alphabet. The first item of Hughes’s list, whether the shuffle of two context-free languages is context-free, is not addressed here. Hughes’s Section 14 (p. 12) also lists “Study constrained crossover” among its open directions without posing a specific question; Section 11 records how the present results bear on his other Section 14 questions.

The inspected ProveIt revision is

9b24a3a8d545af9624f6ac455f5b548be62818b6.

The relevant source is

SetTheory/Cardinals/docs/reports/
automata-and-formal-languages/insertion-degree-spectra/article.tex.

At this revision the source has both the original spectrum paper and its binary-profile Part II. We do not reprove their arbitrary-spectrum realization as a claimed contribution, and we do not resolve their both-singleton insertion conjecture. No theorem in the present paper assumes the correctness of an unverified ProveIt mathematical claim. [write] That source is the sibling report *insertion-degree-spectra* of the same collection category; its directory did not change between the pin and the placement of this report (commit 203016015). Section 1.4 states how the two reports relate.

1.2 What the theorems say

The results have three interacting parts. The first is a precise resource calculus: each generation halves an intrinsic fragment rank. The second is an algorithmic boundary: global stabilization is undecidable for

grammars but decidable for finite automata. The third is an enumeration theorem: the commutative counting series of a context-free seed’s full recombination closure is rational even when the resulting language is not context-free.

Question	Result proved here	Section
Exact generation of a word	$L^{[k]} = \{w : \rho_L(w) \leq 2^k\}$; a no-gap rank spectrum	3
One-step equality for CFGs	Co-r.e.-complete over a fixed ternary alphabet	4
Eventual equality for CFGs	Undecidable over four letters; strictness witnesses at every stage	5
A fixed target and grammar	Exact rank and generation decidable in polynomial time	6
Regular-input decision problems	DFA polynomial one-step test; NFA PSPACE-completeness; decidable finite stabilization	7
Counting the full closure of a CFL	Effective rationality, including commuting letter weights	8
Closure need not be context-free	Explicit linear seed; exact counts and sharp recurrence orders	9

1.3 Attribution and limits of the novelty claim

The conjectural-status claim is deliberately narrow: the one-step decision question is explicitly open in the inspected Hughes version. The proofs below answer that question under exactly its equal-length, equal-cut definition. The coordinate-product closure is an elementary recombination fact; it is reproved for completeness and is not advertised as a first discovery. Nor are context-free universality, effective Parikh semilinearity, Presburger counting, or distance-automaton limitedness new here.

The proposed contribution consists of the guarded reduction, its exact amplification, the quantitative fragment calculus and algorithms, and the rational-growth consequences in this setting. Equivalent formulations in recombination, splicing, or genetic-algorithm literature have not been exhaustively excluded. The article gives proofs so that its mathematical content can be assessed independently of priority. The source search and verification boundary are recorded again in Section 10.

1.4 [write] Relation to the sibling report on insertion-degree spectra

The sibling report *Gaps in Bounded-Shuffle Hierarchies* (ProveIt path SetTheory/Cardinals/docs/reports/automata-and-formal-languages/insertion-degree-spectra/) and this report start from the same preprint of Hughes, but from different parts of it.

- The sibling’s Part I answers questions of Hughes’s Sections 10–11. It refutes the insertion-degree no-gap conjecture (every finite set containing 1 is a spectrum), answers the finite representation-erasure question with a binary example, and proves an iteration-depth no-gap theorem (its Theorem 8.2). Its Part II realizes every finite spectrum containing 1 over a binary alphabet with one inserted unary word. Its scope sentence, under its abstract, says that it addresses “the indicated no-gap questions, not the surrounding undecidability results”. It never names the crossover question: the word “crossover” occurs there only in the title of Hughes’s preprint.
- This report answers the single-step question for constrained crossover posed in Hughes’s Section 4 (Section 1 above), one of those surrounding decision problems, for an operation the sibling does not study.
- No theorem of either report is used by the other, and none is proved in both. The no-gap theorem for aligned rank (Theorem 3.4) is an analogue, for a different operation, of the sibling’s iteration-depth no-gap theorem; neither implies the other. The sibling iterates $X_{m+1} = A \sqcup X_m$ with the inserted language A

held fixed. Its closest crossover counterpart is therefore the frozen-source iteration of Theorem 3.8, not the parallel iteration of Definition 2.1.

- Nothing here bears on the questions the sibling leaves open, in particular Hughes’s singleton insertion-degree no-gap conjecture (his Conjecture 2, the sibling’s Question 21.1), as the paragraph after Theorem 3.4 already says.

The sibling report carries a dated remark, under its scope sentence, that points to this report.

1.5 [write] Notation beside the neighbouring texts

No symbol of the manuscript was renamed. The table lists the readings that Hughes’s preprint or the sibling report could suggest.

Here	Meaning here	Not to be confused with
$A \otimes_c B$	constrained one-point crossover (Definition 2.1); the same operation and symbol as Hughes’s $\otimes_c$	Hughes’s unconstrained crossover $\otimes_u$, which contains concatenation and is covered by his Theorem 1
$T(L), L^{[k]}$	parallel self-crossover: both parents from generation k	Y_k , with one parent frozen in L (Theorem 3.8). Hughes’s iterated insertion starts from B and applies A on the left each time, and his iterated crossovers are defined “as with other operations”; for $A = B = L$ that reading gives Y_k , not $L^{[k]}$
$\rho_L(w)$	aligned-fragment rank (Definition 3.1)	the sibling’s insertion degree $d_{A,B}(w)$, a number of inserted pieces; and the unspecified “rank condition” of Hughes’s Section 14
$d_L(w)$	generation depth $\lceil \log_2 \rho_L(w) \rceil$ (5)	the sibling’s $d_{A,B}(w)$ (a degree, not a depth) and its iteration depth (a number of applications of $A \sqcup \cdot$)
“no gaps”	the ranks on a slice form $\{1, \dots, R_L(n)\}$ (Theorem 3.4)	the sibling’s spectra $\mathcal{D}(A, B)$, which <i>can</i> have gaps, and its iteration depths, which cannot
stabilization	some pair of consecutive generations is equal (Corollary 3.5)	Hughes’s “adjacent equality”; by Lemma 2.2 (and likewise for Y_k) adjacent equality here is already his “fixed point”
co-r.e.	negative instances effectively enumerable; co-r.e.-complete means Π_1^0 -complete	r.e. (the positive instances)

2 Conventions and the coordinate hull

All alphabets are finite. Put $\mathbb{N} = \{0, 1, 2, \dots\}$ and write ε for the empty word. For a word w of length n and $0 \leq i \leq j \leq n$, $w[i : j]$ denotes positions $i + 1, \dots, j$. Thus the indexing of boundaries is zero-based, but the indexing of letters is one-based. Write $L_n = L \cap \Sigma^n$.

Definition 2.1 (Constrained crossover). For languages $A, B \subseteq \Sigma^*$, define

$$A \otimes_c B = \{wz, yx : wx \in A, yz \in B, |w| = |y|, |x| = |z|\}.$$

For self-crossover put $T(L) = L \otimes_c L$. Its *parallel iterates* are

$$L^{[0]} = L, \quad L^{[k+1]} = T(L^{[k]}).$$

Both parents in generation $k + 1$ may be chosen from generation k .

The two length equalities imply that the complete parents have the same length and that their cut positions agree. Cuts at either endpoint are allowed. On a fixed slice,

$$T(L)_n = \bigcup_{i=0}^n \text{Pref}_i(L_n) \text{ Suff}_{n-i}(L_n). \quad (1)$$

The prefix and suffix in this product may come from different parents, but both parents must lie in L_n .

Lemma 2.2. *The map T is monotone and inflationary. Consequently the iterates increase, and equality at one pair of consecutive generations implies equality at all subsequent generations.*

Proof. Every pair of parents available in a smaller language is available in a larger one. Also, crossing a word with itself returns it. Thus $L \subseteq T(L)$. If $L^{[k]} = L^{[k+1]}$, applying the same map T to both sides proves the next equality, and induction finishes the argument. $\square$

Remark 2.3 (Do not change the iteration rule). The iteration $X_{k+1} = L \otimes_c X_k$, with one parent frozen in the original language, is not the iteration in Definition 2.1. In particular its resource accounting need not double each generation. The self-operation above is inflationary even though an iteration involving two different input languages can lose words of lengths absent from one input. These distinctions remove any ambiguity suggested by general remarks about nonmonotone constrained crossover.

[write] The general remark meant is in Hughes's Section 2 (p. 2): the series produced by iterated constrained crossover, unlike that of unconstrained crossover, can drop earlier elements from later sets. With two different inputs, as in $X_{m+1} = A \otimes_c X_m$ with $X_0 = B \neq A$, it does happen: the words of B whose length no word of A has are lost at the first step. For self-crossover it cannot happen under either clock: the parallel iterates increase by Lemma 2.2, and the frozen-source iterates of Theorem 3.8 increase by its formula.

Definition 2.4 (Coordinate hull). For $n \geq 1$ and $1 \leq i \leq n$, let

$$P_{n,i}(L) = \{u_i : u \in L_n\}.$$

Define $\text{Rec}(L)_n = \emptyset$ if $L_n = \emptyset$, and otherwise define

$$\text{Rec}(L)_n = P_{n,1}(L)P_{n,2}(L) \cdots P_{n,n}(L).$$

At length zero put $\text{Rec}(L)_0 = L_0$. Finally $\text{Rec}(L) = \bigcup_{n \geq 0} \text{Rec}(L)_n$.

The explicit convention at length zero is necessary. The empty product of coordinate sets must not introduce ε when the seed does not contain it. Every crossover preserves membership in the coordinate hull, since it introduces no new letter at any position. The converse reachability statement will follow from a stronger exact-rank theorem.

3 Aligned fragments and exact generation depth

3.1 The fragment invariant

Definition 3.1 (Aligned fragment and rank). Fix $w \in \Sigma^n$, $n \geq 1$. A nonempty interval $[i, j]$ is L -compatible for w if there is $u \in L_n$ with $u[i : j] = w[i : j]$. The rank $\rho_L(w)$ is the least positive integer r for which there is a partition

$$0 = i_0 < i_1 < \dots < i_r = n \quad (2)$$

whose r intervals are L -compatible. If there is no such partition, put $\rho_L(w) = \infty$. Set $\rho_L(\varepsilon) = 1$ when $\varepsilon \in L$, and ∞ otherwise.

The seed witnessing one fragment need not witness another. However, every witness is a complete seed word of length n , and every fragment is at its *original position*. Ordinary factor occurrence without alignment would give a different and incorrect invariant.

We have $\rho_L(w) = 1$ exactly when $w \in L$. For nonempty w , finite rank is at most $|w|$, because one may split any compatible partition into single-letter intervals. The existence of such a singleton partition is exactly membership in the coordinate hull.

Theorem 3.2 (Exact rank contraction). *For every language L and every word w ,*

$$\rho_{T(L)}(w) = \left\lceil \frac{\rho_L(w)}{2} \right\rceil, \quad (3)$$

where the right side is ∞ when $\rho_L(w) = \infty$.

Proof. First suppose w is nonempty. A fragment matching a word of $T(L)$ crosses at most one cut of its two seed parents. Splitting the fragment at that cut, when the cut lies strictly inside it, gives at most two nonempty L -compatible fragments. A partition of w into s $T(L)$ -compatible fragments therefore yields an L -compatible partition of size at most $2s$. In particular finite $\rho_{T(L)}(w)$ implies finite $\rho_L(w)$ and

$$\rho_{T(L)}(w) \geq \lceil \rho_L(w)/2 \rceil.$$

Conversely, take a shortest partition (2) of size r . Group consecutive fragments in pairs. If two adjacent fragments are witnessed by $u, v \in L_n$, cross u with v at their common boundary. The resulting word agrees with w throughout their union. A leftover final fragment already matches a seed, and seeds belong to $T(L)$. Thus the grouped partition has $\lceil r/2 \rceil$ $T(L)$ -compatible pieces. This proves the reverse inequality. The same argument rules out a finite rank on either side when the other side is infinite. For $w = \varepsilon$, crossover neither creates nor removes the empty word, so the convention gives the stated identity. $\square$

Theorem 3.3 (Generation formula and full closure). *For every $k \geq 0$,*

$$L^{[k]} = \{w : \rho_L(w) \leq 2^k\}, \quad \bigcup_{k \geq 0} L^{[k]} = \text{Rec}(L). \quad (4)$$

A reachable word of rank r first occurs at generation

$$d_L(w) = \lceil \log_2 r \rceil. \quad (5)$$

Consequently $T(L) = L$ if and only if every nonempty slice L_n is its full Cartesian product of coordinate projections.

Proof. Iterating Theorem 3.2 gives $\rho_{L^{[k]}}(w) = \lceil \rho_L(w)/2^k \rceil$ for finite ranks. Membership in $L^{[k]}$ is equivalent to rank one relative to that language, which proves the first formula and the depth formula. Finite rank is equivalent to coordinate-hull membership, proving the union formula. The hull is fixed by crossover because each output letter stays in its designated coordinate set. If L is itself fixed, all its iterates equal L and therefore so does their union. The converse follows from the hull's fixedness. $\square$

The proof can also be viewed as a balanced assembly tree. Each leaf supplies one aligned seed fragment, and each internal vertex joins two adjacent fragment blocks. The contraction argument has the advantage of giving a matching lower bound for *every* representation, not merely constructing one efficient derivation.

3.2 There are no missing finite ranks

Theorem 3.4 (No-gap theorem for aligned rank). *If $L_n \neq \emptyset$, the ranks attained on $\text{Rec}(L)_n$ form exactly $\{1, 2, \dots, R_L(n)\}$ for some $R_L(n)$. For $n \geq 1$, $1 \leq R_L(n) \leq n$; for the active zero slice, $R_L(0) = 1$.*

Proof. Let w have rank r , and choose a shortest partition with boundaries $i_0, \dots, i_r$ and corresponding seed witnesses $u_1, \dots, u_r$. For each $1 \leq k \leq r$, form

$$v_k = w[0 : i_{k-1}] u_k[i_{k-1} : n].$$

The first $k - 1$ old fragments followed by the suffix of u_k show that $\rho_L(v_k) \leq k$. Also v_k agrees with w through boundary i_k . If $k \geq 2$ and v_k had a partition into at most $k - 1$ compatible fragments, restrict that partition to its prefix ending at i_k and discard empty intervals. Append the last $r - k$ fragments of the original partition of w . This would give w at most $(k - 1) + (r - k) = r - 1$ fragments, a contradiction. Thus v_k has rank exactly k . The case $k = 1$ follows from positive rank. There are finitely many words of length n , so the largest rank exists. The earlier singleton-fragment bound proves $R_L(n) \leq n$. $\square$

This no-gap assertion concerns aligned seed fragments, not the number of runs in an insertion mask. Its proof changes the output word and retains alignment throughout; it gives no proof of the both-singleton insertion conjecture in the repository.

3.3 Sharp stabilization bounds

Corollary 3.5. *An active slice stabilizes at the exact stage*

$$h_L(n) = \lceil \log_2 R_L(n) \rceil.$$

Empty slices stabilize at stage zero. In particular, for $n \geq 1$ all length- n behavior stabilizes by $\lceil \log_2 n \rceil$. The whole language stabilizes after finitely many generations if and only if its finite ranks are uniformly bounded. If their maximum is $M \geq 1$, the least global stage is $\lceil \log_2 M \rceil$; the empty language has stage zero.

Proof. A slice equals its hull exactly when the rank threshold 2^k covers its largest rank. If it equals its next generation, Lemma 2.2 and Theorem 3.3 imply that it already equals the hull. The global statement follows by taking the supremum of all finite ranks. $\square$

Example 3.6 (Sharpness with a regular seed). Let $L = 0^* \cup 0^*10^*$, the binary words having at most one 1. Its hull is all of $\{0, 1\}^*$, and

$$\rho_L(w) = \max(1, |w|_1), \quad L^{[k]} = \{w : |w|_1 \leq 2^k\}.$$

A compatible fragment contains at most one 1, giving the lower bound. Cutting between consecutive 1's gives the reverse bound, because each resulting fragment extends by zeros to an allowed seed of the same length. Thus $R_L(n) = n$ for $n \geq 1$, and the logarithmic slice bound is sharp. Nevertheless every global inclusion $L^{[k]} \subsetneq L^{[k+1]}$ is strict.

Corollary 3.7 (Shortest possible late word). *Every word in $L^{[k+1]} \setminus L^{[k]}$ has length at least $2^k + 1$. For every $k \geq 0$, Example 3.6 attains this lower bound with 1^{2^k+1} .*

Proof. A newly appearing word has rank greater than 2^k and finite rank at most its length. The example has rank equal to the number of 1's. $\square$

3.4 The frozen-source convention is also exactly solvable

For comparison, let $Y_0 = L$ and $Y_{k+1} = L \otimes_c Y_k$, using both children in Definition 2.1. This convention keeps one parent in the seed at every step. It has the same full closure, but a different clock.

Theorem 3.8 (Frozen-source generation). *For every $k \geq 0$,*

$$Y_k = \{w : \rho_L(w) \leq k + 1\}.$$

Thus a word of finite rank r has frozen-source depth exactly $r - 1$. Finite stabilization is again equivalent to bounded finite rank, and its least stage is $M - 1$ when the maximum finite rank is $M \geq 1$. The empty seed stabilizes at stage zero.

Proof. The assertion at $k = 0$ is the definition of rank one. If the assertion holds at k , restrict a compatible partition of the Y_k parent to the portion inherited by a child. It uses at most $k + 1$ fragments. The inherited portion of the seed parent uses at most one more. Every child therefore has rank at most $k + 2$.

Conversely, suppose w has a compatible partition into $r \leq k + 2$ pieces, with seeds $u_1, \dots, u_r$ and boundaries $i_0, \dots, i_r$. A rank-one word persists by crossing it with itself. For $r \geq 2$, form

$$v = w[0 : i_{r-2}] u_{r-1} [i_{r-2} : n].$$

It has rank at most $r - 1 \leq k + 1$ and agrees with w through boundary i_{r-1} . By induction $v \in Y_k$. Its prefix through that boundary, crossed with the suffix of $u_r \in L$, produces w . This proves the reverse inclusion. The stabilization statements follow because the thresholds $k + 1$ exhaust precisely the finite ranks. $\square$

The distinction is quantitative: the parallel clock is logarithmic in rank, whereas the frozen-source clock is linear. Both clocks detect the same boundedness property. Statements about one convention should not silently be read as finite-stage statements about the other.

4 One-step equality is undecidable for context-free inputs

We use the classical fact that universality of a context-free grammar over a fixed binary alphabet is co-r.e.-complete. Appendix A recalls the usual reduction from Post correspondence and explains the fixed-alphabet step. Here “co-r.e.” means that the negative instances can be effectively enumerated; completeness is under computable many-one reductions.

4.1 A guard construction with universal first generation

Let $\Sigma = \{0, 1\}$, let $@$ be a fresh letter, and put $\Gamma = \{0, 1, @\}$. For any $K \subseteq \Sigma^*$ define

$$G_K = (\Gamma^* \setminus 00\Sigma^*) \cup 00K. \quad (6)$$

This language contains every word involving $@$, every word of length less than two, and every word not beginning with 00 . The only potentially missing words are the words $00x$ with $x \notin K$.

Lemma 4.1 (One-cut repair). *For every $K \subseteq \{0, 1\}^*$,*

$$T(G_K) = \Gamma^*, \quad G_K = \Gamma^* \iff K = \{0, 1\}^*.$$

If K is context-free, a grammar for G_K is effectively constructible. If K is given by an NFA, an NFA for G_K has size polynomial in the input.

Proof. The second equivalence is immediate from the list of missing words. Every word already in G_K remains in $T(G_K)$. A missing word $w = 00x$ has length $n \geq 2$. Define two parents

$$u = w[0 : 1] @ w[2 : n], \quad v = @ w[1 : n].$$

Both contain the guard and hence belong to G_K , both have length n , and crossing after their first letter gives $u[0 : 1]v[1 : n] = w$. Thus every missing word is repaired in one step. The language $\Gamma^* \setminus 00\Sigma^*$ is a fixed regular language; union with the prefix extension $00K$ gives the asserted effective grammar and automaton constructions. $\square$

Theorem 4.2 (Resolution of the one-step decision question). *Given a context-free grammar over the fixed alphabet $\{0, 1, @\}$, deciding whether its language satisfies $L \otimes_c L = L$ is co-r.e.-complete. This remains true on the effective family in which the first crossover generation is the entire free monoid.*

Proof. By Lemma 4.1,

$$T(G_K) = G_K \iff G_K = \Gamma^* \iff K = \{0, 1\}^*.$$

This proves co-r.e.-hardness and the promised restriction. For membership in co-r.e., enumerate pairs of equal-length words and their cut positions. Grammar membership is decidable. Whenever both parents belong to L but their child does not, the enumeration finds a finite witness to failure of closure. Such a witness exists exactly when $T(L) \neq L$, since $L \subseteq T(L)$ always holds. $\square$

The use of 00 is not cosmetic. It ensures that every omitted word has at least two positions, allowing two distinct guard replacements and a genuinely internal repair cut. Without a length guard, a missing empty word or single-letter word cannot be created by aligned crossover from other words of the same length.

Corollary 4.3. *There is no computable function of grammar-description size bounding the length of the shortest counterexample to one-step closure for all nonclosed context-free grammars over this fixed alphabet.*

Proof. Such a bound would allow a finite exhaustive search. For each word length below the bound, enumerate all equal-length parents and cuts and use grammar membership to test the resulting violations. If no violation were found, the bound would certify closure, contradicting Theorem 4.2. $\square$

5 Finite stabilization and an exact block amplification

One-step undecidability does not by itself imply undecidability of eventual stabilization: the guarded examples above *always* stabilize by stage one. We now amplify a single omitted block into arbitrarily high rank.

Introduce a fresh separator $\#$ and put $\Delta = \{0, 1, @, \#\}$. Define

$$B_K = G_K(\#G_K)^*. \quad (7)$$

Thus a word belongs to B_K exactly when every block between successive separators belongs to G_K , including the initial and final blocks. Empty blocks are allowed because $\varepsilon \in G_K$. This construction preserves context-freeness effectively.

Theorem 5.1 (Exact rank of repeated forbidden blocks). *Suppose $x \notin K$, put $w = 00x$, and let $m = |w| \geq 2$. For $r \geq 1$ define*

$$W_r = w(\#w)^{r-1}.$$

Then

$$|W_r| = (m+1)r - 1, \quad \rho_{B_K}(W_r) = r + 1, \quad d_{B_K}(W_r) = \lceil \log_2(r+1) \rceil. \quad (8)$$

Proof. Number the r copies of w . The j th copy occupies the interval between boundaries

$$s_j = (j-1)(m+1), \quad t_j = s_j + m.$$

Consider any compatible-fragment partition of W_r . We claim it must have an internal partition boundary in the integer interval $[s_j, t_j]$ for each j .

If there is no such boundary, one partition fragment contains the entire j th copy of w . When a neighboring separator exists, the fragment also contains that separator, because a cut immediately before or after the copy would be a boundary in $[s_j, t_j]$. At the beginning or end of the whole word, the corresponding end of the seed is fixed by alignment and equal total length. Therefore every complete seed matching this fragment contains w as a whole separator-delimited block. This is impossible in B_K , because $w = 00x \notin G_K$.

The r integer intervals $[s_j, t_j]$ are pairwise disjoint: $s_{j+1} = t_j + 1$. Thus at least r distinct internal partition boundaries are needed, and there are at least $r+1$ fragments.

For the upper bound, place one cut strictly inside each copy of w , for example immediately after its first letter. These cuts give $r+1$ fragments. No resulting fragment contains a whole copy of w . For any one fragment, retain the separator positions of W_r , retain the letters of that fragment, and replace all other nonseparator positions by $@$. In every block at least one position has been replaced, so every block contains a guard and belongs to G_K . The resulting full word lies in B_K and matches the chosen fragment at the required position. Hence the partition is compatible and has $r+1$ pieces. The depth formula follows from Theorem 3.3. $\square$

The lower bound allows seed words with arbitrary separator placements outside the chosen fragment. It does not quietly restrict the minimization to seeds with the same global block layout. The separators *inside* an overlong fragment, together with the actual word endpoints, force the bad block.

Theorem 5.2 (Finite-stabilization undecidability). *It is undecidable, given a context-free grammar over the fixed four-letter alphabet Δ , whether its parallel self-crossover iterates eventually stabilize. This holds even under the promise that the language is either Δ^* or has strict inclusions at every generation. For each fixed $k \geq 0$, deciding $L^{[k]} = L^{[k+1]}$ is co-r.e.-complete.*

Proof. If $K = \{0, 1\}^*$, then $G_K = \Gamma^*$ and $B_K = \Delta^*$, so stabilization occurs at stage zero. Otherwise fix $x \notin K$ and apply Theorem 5.1. For every $k \geq 0$ the word W_{2^k} has rank $2^k + 1$, and therefore

$$W_{2^k} \in B_K^{[k+1]} \setminus B_K^{[k]}, \quad |W_{2^k}| = (m+1)2^k - 1. \quad (9)$$

Thus B_K satisfies the promise and stabilizes if and only if K is universal. This proves undecidability and co-r.e.-hardness for each fixed adjacent equality problem.

For a fixed k , membership in either finite generation is decidable: for a candidate word of length n , one may enumerate the finite set of seed words of length n using grammar membership, then compute crossover $k+1$ times. Enumerating words consequently semidecides a failure of the adjacent equality. Section 6 gives a much faster membership algorithm. $\square$

Corollary 5.3 (The same decision boundary with a frozen source). *Finite stabilization is also undecidable for $Y_{k+1} = L \otimes_c Y_k$, and every fixed adjacent equality problem $Y_k = Y_{k+1}$ is co-r.e.-complete for CFGs. On the nonuniversal side of the block construction, W_{k+1} has frozen-source depth $k+1$ and therefore witnesses strictness at stage k .*

Proof. Theorem 3.8 turns the exact block rank $r+1$ into depth r . Use $r = k+1$ for a fixed-stage witness. The effective reduction and the semidecision of failures are the same as in Theorem 5.2. $\square$

Proposition 5.4 (An arithmetical upper bound). *The set of context-free grammars whose self-crossover sequence eventually stabilizes belongs to Σ_2^0 and is Π_1^0 -hard. No completeness claim at level Σ_2^0 is made here.*

Proof. The condition has the form

$$\exists k \forall w \quad (w \in L^{[k+1]} \implies w \in L^{[k]}),$$

with a decidable matrix, uniformly in the grammar, k , and w . Theorem 5.2 reduces the co-r.e.-complete universality problem to this condition, proving Π_1^0 -hardness. $\square$

6 Polynomial-time rank computation for a fixed target

Global undecidability is compatible with efficient local membership. The aligned-fragment invariant makes this particularly transparent.

Proposition 6.1 (Compatibility by constrained parsing). *Let G be a context-free grammar in Chomsky normal form, of size g , and let w have length $n \geq 1$. All interval-compatibility answers for w can be computed in $O(gn^5)$ time. Consequently $\rho_{L(G)}(w)$ and its first generation, when finite, can be computed in that time. The bound is in the size after the usual polynomial-time grammar normalization.*

Proof. For $0 \leq i < j \leq n$, compatibility is equivalent to

$$L(G) \cap \Sigma^i \{w[i:j]\} \Sigma^{n-j} \neq \emptyset. \quad (10)$$

Run CYK on a length- n string whose positions inside $[i, j)$ are fixed to the corresponding letters of w and whose other positions are wildcards. A terminal production is allowed at a wildcard if it produces any alphabet letter; at a fixed position it must produce that letter. The binary CYK recursion is unchanged. Its acceptance means that some full word meeting these position constraints belongs to the language. Each query costs $O(gn^3)$, and there are $O(n^2)$ intervals.

After the queries, make a directed acyclic graph on boundaries $0, 1, \dots, n$, with an edge $i \rightarrow j$ exactly for the compatible intervals. The least number of edges from 0 to n is exactly the fragment rank. Equivalently,

$$D[0] = 0, \quad D[j] = 1 + \min\{D[i] : 0 \leq i < j, [i, j) \text{ compatible}\},$$

with the minimum of the empty set equal to ∞ . This last step takes $O(n^2)$ time. The depth is obtained from (5). $\square$

This algorithm answers membership in $L^{[k]}$ even when k is given in binary and is very large: compare the computed depth with k , rather than constructing the integer 2^k . A word has finite depth at most $\lceil \log_2 n \rceil$. The empty word is handled by a single grammar membership test. Back pointers in the dynamic program and parsing tables recover a shortest partition and its seed witnesses; balanced pairing then recovers an explicit crossover derivation.

The conservative $O(gn^5)$ bound does not assert optimality. Sharing work between wildcard parses should improve it. More importantly, no step requires a context-free description of an intermediate generation. Such a description has not been assumed to exist.

7 Regular inputs: finite-state tests and limitedness

7.1 One-step closure for deterministic automata

Let $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ be a complete DFA with s states. The completion convention includes a rejecting sink when necessary.

Theorem 7.1 (A cubic-state violation graph). *One can decide $T(L(\mathcal{A})) = L(\mathcal{A})$ in $O(s^3|\Sigma|^2)$ time by graph reachability. If closure fails, there is a counterexample word of length at most $2s^3 - 1$.*

Proof. Use states $(p, q, r, e) \in Q^3 \times \{0, 1\}$. The first two components read the two parent words; the third reads the child; e records whether the cut has occurred. Initially all three states are q_0 and $e = 0$. There is a single possible phase change $(p, q, r, 0) \rightarrow (p, q, r, 1)$, consuming no letter.

On a child letter a , choose an arbitrary letter b for the parent whose letter is not inherited. In phase zero update

$$(p, q, r) \mapsto (\delta(p, a), \delta(q, b), \delta(r, a));$$

in phase one update

$$(p, q, r) \mapsto (\delta(p, b), \delta(q, a), \delta(r, a)).$$

A state is accepting for the violation graph when $p, q \in F$ and $r \notin F$. Every accepting path supplies two accepted equal-length parents and a rejected child, with one aligned cut. Conversely every closure violation supplies such a path, using phase changes at the endpoints when needed.

There are $2s^3$ states and $O(s^3|\Sigma|^2)$ transitions. Reachability therefore has the stated complexity. A shortest edge path to an accepting state is simple, has at most $2s^3 - 1$ edges, and consumes no more letters than that. Thus a counterexample of the asserted length exists. $\square$

Ordinary graph search finds a bounded witness; using zero–one breadth-first search, assigning zero to the phase change and one to letter transitions, finds a shortest child word. The graph tracks the two parents independently outside the inherited portions, which is essential for exact-length matching.

Theorem 7.2 (NFA complexity). *For a language given by an NFA, one-step crossover closure is PSPACE-complete, already over a fixed three-letter alphabet.*

Proof. For hardness, apply the polynomial NFA construction in Lemma 4.1 to binary NFA universality. The latter problem is PSPACE-complete; fixed-alphabet hardness is classical, and is also available under stronger automaton restrictions [6].

For membership, first eliminate epsilon transitions. Adapt the violation graph: its parent components choose individual NFA runs, while its child component is the subset of all states reachable on the child prefix. Acceptance requires accepted parent runs and a child subset disjoint from F . A state consists of two NFA states, one subset, and one phase bit, and can be stored in polynomial space. Reachability in the exponentially large implicit graph is decidable in polynomial space. PSPACE is closed under complementation, so the closure rather than violation problem has the same upper bound. $\square$

7.2 Encoding all fragment ranks by a distance automaton

A *distance automaton* is a finite nondeterministic automaton with transition costs 0 or 1. The value of a word is the minimum total cost of an accepting path, or ∞ if there is no accepting path. The classical *limitedness problem* asks whether all finite values are bounded by one integer. It is decidable in PSPACE in the explicit automaton size [5]. We use this theorem, rather than reprove it.

For a complete DFA $\mathcal{A}$ define, for $S \subseteq Q$,

$$\text{Post}(S) = \{\delta(q, a) : q \in S, a \in \Sigma\}, \quad \text{Pre}(S) = \{q : \exists a \in \Sigma, \delta(q, a) \in S\}.$$

For a target length n , set

$$U_i = \delta(q_0, \Sigma^i), \quad V_i = \{q : \delta(q, v) \in F \text{ for some } v \in \Sigma^{n-i}\}.$$

Then $U_0 = \{q_0\}$, $U_{i+1} = \text{Post}(U_i)$, $V_n = F$, and $V_i = \text{Pre}(V_{i+1})$. An interval $w[i : j]$ is compatible exactly when reading it from some state in U_i ends in V_j .

Theorem 7.3 (A finite-state representation of rank). *From an s -state complete DFA for L one can construct a distance automaton with at most $s4^s$ states, allowing a set of initial states, whose finite-word value is*

$$\text{val}(w) = \rho_L(w) - 1$$

when $\rho_L(w) < \infty$, and ∞ otherwise. In particular $\text{Rec}(L)$ is effectively regular.

Proof. A state is a triple (U, V, q) , where $U, V \subseteq Q$ and $q \in Q$. The initial states have $U = \{q_0\}$, $q = q_0$, and arbitrary V . Final states satisfy $V = F$ and $q \in F$.

When consuming the next child letter a , choose a new subset V' subject to $\text{Pre}(V') = V$ and set $U' = \text{Post}(U)$. There are two sorts of transitions:

$$\begin{aligned} (U, V, q) &\longrightarrow (U', V', \delta(q, a)) && \text{at cost 0;} \\ (U, V, q) &\longrightarrow (U', V', \delta(p, a)) && \text{at cost 1, } q \in V, p \in U. \end{aligned}$$

The second transition closes the current fragment and starts a new one just before consuming a .

For any accepting path of length n , the boundary conditions $V_n = F$ and $V_i = \text{Pre}(V_{i+1})$ force the guessed subsets to be exactly the sets V_i defined above. The U_i are forced by their forward recurrence. A completed

fragment starts in a state reachable by an arbitrary prefix of the required length and ends in a state having an arbitrary accepting suffix of the required length. It therefore extends to a genuine seed in L_n . The last fragment ends in F and has the same property.

An accepting path of cost c thus gives a compatible partition into at most $c + 1$ nonempty fragments. A reset before the first letter is redundant: $U_0 = \{q_0\}$, so deleting that reset preserves its state transition and reduces its cost. There are no resets after the last letter, because resets are combined with a consumed letter. A minimum-cost path consequently corresponds to exactly its positive-length fragments.

Conversely, given a compatible partition into r fragments, choose the states and runs of their seed witnesses. Continue at cost zero within a fragment and reset at cost one at each of its $r - 1$ internal boundaries. The accepting path has cost $r - 1$. This proves the equality of values. For ε , a cost-zero accepting initial state exists exactly when $q_0 \in F$, agreeing with the rank convention.

There are $2^s \cdot 2^s \cdot s = s4^s$ states. Forgetting the weights yields an NFA recognizing the finite-rank domain, which is $\text{Rec}(L)$ by Theorem 3.3. $\square$

Corollary 7.4 (Decidable regular-input stabilization). *Finite stabilization of parallel self-crossover, and of frozen-source crossover, is decidable for regular languages. For complete DFA input, the preceding construction and the classical limitedness algorithm give an EXPSPACE upper bound in the DFA size. This is an upper bound, not a claimed optimal complexity.*

Proof. By Corollary 3.5 and Theorem 3.8, either form of stabilization is equivalent to bounded finite rank. Theorem 7.3 converts this into limitedness of a distance automaton. Its explicit state and transition description has size $2^{\text{poly}(s+|\Sigma|)}$; polynomial space in that size is exponential space in the original description size. $\square$

[Added 30 September 2026, batch 66: Part II improves this upper bound to PSPACE, for NFA and for DFA input (Theorem 19.2), by a periodic-interior criterion instead of generic limitedness, and proves the problem PSPACE-complete for NFA input over a binary alphabet (Corollary 20.3); every stabilizing s -state NFA language has rank at most $(2s + 5)^2$ (Corollary 19.3). The exact complexity for DFA input stays open. The EXPSPACE bound stated here remains true; Part II is unrefereed and not formalized.]

When limitedness holds, the least stabilization stage is also computable. Indeed, each finite iterate of a regular language is effectively regular by a product-and-cut construction, and equality of regular languages is decidable. Generate iterates and test adjacent equality until the first one occurs. The limitedness decision guarantees termination of this second procedure on the bounded side. This gives an effective answer, not a practical implementation of an optimal limitedness solver.

8 Rational counting for the full closure of every CFL

The regularity conclusion of Theorem 7.3 does not extend to context-free inputs. A weaker conclusion does, and it is stronger than rationality of the ordinary length series: all commuting letter weights may be retained.

8.1 Effective semilinear position supports

Lemma 8.1. *Let $L \subseteq \Sigma^*$ be context-free. For each $a \in \Sigma$, the set*

$$E_a = \{(i, j) \in \mathbb{N}^2 : \exists u \in \Sigma^i, v \in \Sigma^j, uav \in L\} \quad (11)$$

is effectively semilinear, and hence effectively definable in Presburger arithmetic.

Proof. Take two tagged copies of the alphabet and a new marker $\bullet$ that is mapped to a . The inverse image of L under the letter-forgetting homomorphism is context-free. Intersect it with the regular language “left-tagged letters, then $\bullet$, then right-tagged letters.” Its Parikh image is effectively semilinear by the effective form of Parikh’s theorem [3]. Sum the counts of left-tagged letters, sum the counts of right-tagged letters, and discard the marker coordinate. The resulting linear image is exactly E_a . $\square$

For $0 \leq i < n$, the letters allowed at position $i + 1$ of a hull word are

$$P_{n,i+1}(L) = \{a \in \Sigma : (i, n - 1 - i) \in E_a\}.$$

Therefore, for each $S \subseteq \Sigma$, the predicate $P_{n,i+1}(L) = S$ is a Boolean combination of effective Presburger predicates. Define the number of positions of each type by

$$A_S(n) = \#\{i : 0 \leq i < n, P_{n,i+1}(L) = S\}. \quad (12)$$

Lemma 8.2 (Eventually affine counts on residue classes). *There are an effectively obtainable period $p \geq 1$ and, for each residue class, a starting length N_r in that class such that simultaneously for all $S \subseteq \Sigma$,*

$$A_S(N_r + pt) = b_{S,r} + d_{S,r}t \quad (t \geq 0), \quad (13)$$

with $b_{S,r}, d_{S,r} \in \mathbb{N}$. Each such progression can also be chosen either entirely active ($L_n \neq \emptyset$) or entirely inactive.

Proof. Each A_S is a Presburger counting function. Woods’s counting theorem implies that it is a piecewise quasi-polynomial [4, Theorem 1.10]. In one parameter, beyond a finite threshold the polyhedral partition has only unbounded rays, so the function is an eventual quasi-polynomial. Since $0 \leq A_S(n) \leq n$, each constituent polynomial has degree at most one. Take a common period and a sufficiently large starting length in each residue class for all the finitely many subsets S .

The value at the starting length is an integer $b_{S,r} \geq 0$, and the constant first difference along the progression is an integer $d_{S,r}$. A negative first difference would eventually make the count negative, so $d_{S,r} \geq 0$. These procedures are effective: Presburger quantifier elimination and the counting construction obtain a representation, and all subsequent finite refinements are effective. Appendix B also explains the one-parameter affine conclusion directly.

The length set of a context-free language is effectively semilinear by Parikh’s theorem. A semilinear subset of $\mathbb{N}$ is eventually periodic. Refine the same period and thresholds so that activity is constant on each progression. This deals explicitly with empty slices. $\square$

8.2 A rational commutative generating function

For commuting variables $\mathbf{x} = (x_a)_{a \in \Sigma}$ put

$$\text{wt}(w) = \prod_{a \in \Sigma} x_a^{|w|_a}, \quad \Phi_L(z, \mathbf{x}) = \sum_{w \in \text{Rec}(L)} z^{|w|} \text{wt}(w).$$

The series counts distinct words, not derivations or seed representations.

Theorem 8.3 (Rational commutative growth). *For every context-free language L , the series $\Phi_L(z, \mathbf{x})$ is an effectively computable rational function. More precisely, it has a representation*

$$\Phi_L(z, \mathbf{x}) = P(z, \mathbf{x}) + \sum_{r \in \mathcal{R}} \frac{z^{N_r} C_r(\mathbf{x})}{1 - z^p D_r(\mathbf{x})}, \quad (14)$$

where P is a polynomial, $\mathcal{R}$ indexes the eventually active residue classes, and C_r, D_r are products of powers of the nonzero linear forms $\sum_{a \in S} x_a$ for $\emptyset \neq S \subseteq \Sigma$. In particular, along each sufficiently late active arithmetic progression,

$$|\text{Rec}(L)_{N_r+pt}| = C_r(\mathbf{1})D_r(\mathbf{1})^t, \quad (15)$$

where both constants are positive integers. Inactive progressions have count zero.

Proof. On an active slice the choices of letters at different positions are independent. Hence

$$\begin{aligned} \sum_{w \in \text{Rec}(L)_n} \text{wt}(w) &= \prod_{i=1}^n \left(\sum_{a \in P_{n,i}(L)} x_a \right) \\ &= \prod_{\emptyset \neq S \subseteq \Sigma} \left(\sum_{a \in S} x_a \right)^{A_S(n)}. \end{aligned} \quad (16)$$

Insert (13). For $n = N_r + pt$ this becomes $C_r(\mathbf{x})D_r(\mathbf{x})^t$, with

$$C_r(\mathbf{x}) = \prod_{S \neq \emptyset} \left(\sum_{a \in S} x_a \right)^{b_{S,r}}, \quad D_r(\mathbf{x}) = \prod_{S \neq \emptyset} \left(\sum_{a \in S} x_a \right)^{d_{S,r}}.$$

Summing this geometric series in t gives its summand in (14). The finitely many lengths before the chosen starts, including length zero, supply P . Their coordinate sets and activity are decidable, so this polynomial is effective. Specializing each x_a to one yields (15). $\square$

Corollary 8.4. *The ordinary length generating function of $\text{Rec}(L)$ is effectively rational. Its coefficients satisfy an effectively computable constant-coefficient linear recurrence. If $\text{Rec}(L)$ is infinite, its upper exponential growth rate is*

$$\limsup_{n \rightarrow \infty} |\text{Rec}(L)_n|^{1/n} = \max_{r \in \mathcal{R}} D_r(\mathbf{1})^{1/p}.$$

Proof. Rationality and the recurrence follow by setting $\mathbf{x} = \mathbf{1}$. On an active progression the n th root of $C_r(\mathbf{1})D_r(\mathbf{1})^t$, with $n = N_r + pt$, tends to $D_r(\mathbf{1})^{1/p}$. There are finitely many progressions. Infinitude ensures that at least one is active. $\square$

This is not a rationality theorem for the noncommutative word series and not a regularity theorem for the hull. The generating function has forgotten *where* each coordinate type occurs. Its rational form reflects the eventually affine *number* of positions of each type. The next family shows why that distinction is substantive.

9 A linear context-free seed with a non-context-free closure

9.1 Definition, grammar, and position geometry

Fix an integer $q \geq 3$ and define a binary seed

$$\begin{aligned} S_q &= 0^* \\ &\cup \{0^i 1 0^j : j \geq (q-1)i + q - 1\} \\ &\cup \{0^i 1 0^j : i \geq (q-1)j + q - 1\}. \end{aligned} \quad (17)$$

Every seed has at most one 1. The asymmetric inequalities allow that 1 only in an initial or final portion of the word.

Lemma 9.1. *The language S_q is linear context-free. If $n = qm + r$ with $0 \leq r < q$, then the coordinate hull permits 1 exactly in the first m and last m positions.*

Proof. For the second language in (17), use

$$A \rightarrow 0A0^{q-1} \mid 10^{q-1}Z, \quad Z \rightarrow 0Z \mid \varepsilon.$$

After i recursive uses, the trailing zero count is $(q-1)i + q - 1 + t$, $t \geq 0$, as required. Reversing the productions gives a linear grammar for the third language. Their union with 0^* is linear: each production still has at most one nonterminal.

For a word of length $n = i + 1 + j$, the first inequality is equivalent to $n \geq q(i+1)$. Thus the allowed position $i+1$ is at most $\lfloor n/q \rfloor = m$. The second inequality gives the symmetric last m positions. The all-zero seed allows 0 everywhere. $\square$

Write $K_q = \text{Rec}(S_q)$. Lemma 9.1 gives the exact description

$$K_q = \bigcup_{\substack{m \geq 0 \\ 0 \leq r < q}} \{u0^{(q-2)m+r}v : u, v \in \{0, 1\}^m\}. \quad (18)$$

For example, at length 10 and $q = 3$ the first and last three positions are free, while the middle four positions are zero. There are $2^6 = 64$ hull words, but only seven seed words.

Theorem 9.2 (The closure is not context-free). *For every $q \geq 3$, the language K_q is not context-free, although its seed S_q is linear context-free.*

Proof. Intersect K_q with the regular language $1^+0^+1^+$. A word $1^a0^b1^c$ with $a, b, c \geq 1$ lies in the intersection exactly when

$$(q-1)a \leq b+c, \quad (q-1)c \leq a+b. \quad (19)$$

Indeed, the left block of ones fits in the first $\lfloor n/q \rfloor$ positions exactly when $qa \leq n = a+b+c$, and similarly for the right block.

Suppose this intersection were context-free, with pumping length p . The word

$$z = 1^p 0^{(q-2)p} 1^p$$

satisfies both inequalities at equality. For any admissible pumping factorization, the two pumped portions are contained in one window of length at most p , and their combined length is positive. This window cannot meet both outer blocks of ones, because the intervening zero block has length at least p .

Assume, as the pumping property would require, that pumping both down to zero copies and up to two copies stays in the intersection. If either pumped word fails to have three nonempty blocks in the required order, this already is a contradiction. Otherwise let $\delta_a, \delta_b, \delta_c$ be the numbers of letters removed from the three original blocks by pumping down. The up-pumped word adds these same counts. Apply (19) to both pumped words and use equality for z . Opposite signs force

$$(q-1)\delta_a = \delta_b + \delta_c, \quad (q-1)\delta_c = \delta_a + \delta_b.$$

Subtracting gives $\delta_a = \delta_c$, and then $\delta_b = (q-2)\delta_a$. The window misses at least one outer block, so one of δ_a, δ_c is zero. All three counts are therefore zero, contradicting the positive combined length of the pumped portions. The intersection is not context-free, hence neither is K_q . $\square$

9.2 Every rank, every generation, and exact counts

Theorem 9.3. For $w \in K_q$,

$$\rho_{S_q}(w) = \max(1, |w|_1). \quad (20)$$

Consequently

$$S_q^{[k]} = K_q \cap \{w : |w|_1 \leq 2^k\}. \quad (21)$$

For $n = qm + r$, $0 \leq r < q$,

$$|(K_q)_n| = 4^m, \quad (22)$$

$$|(S_q^{[k]})_n| = \sum_{j=0}^{2^k} \binom{2m}{j}, \quad (23)$$

where $\binom{2m}{j} = 0$ for $j > 2m$. The exact slice stabilization stage is $\lceil \log_2 \max(1, 2m) \rceil$.

Proof. Any fragment matching a seed contains at most one 1, so rank is at least $\max(1, |w|_1)$. Partition w between its consecutive ones. Each fragment then has at most one 1; if it has one, that position is seed-supported by Lemma 9.1. Fill all other positions of the full word with zero to extend the fragment to a seed. A fragment with no ones extends to the all-zero seed. This proves (20), including the all-zero and empty cases under the stated convention.

There are $2m$ free binary coordinates. Choosing which j of them are one gives $\binom{2m}{j}$ words. Sum over all j for the hull and over $j \leq 2^k$ for generation k . The maximum rank is $\max(1, 2m)$, giving the last assertion. $\square$

Marking each 1 by a variable x yields

$$\sum_{w \in K_q} z^{|w|} x^{|w|_1} = \frac{1 + z + \cdots + z^{q-1}}{1 - (1+x)^2 z^q}. \quad (24)$$

In particular,

$$\sum_{n \geq 0} |(K_q)_n| z^n = \frac{1 + z + \cdots + z^{q-1}}{1 - 4z^q}. \quad (25)$$

The rational series is exceptionally simple, even though the underlying language is not context-free.

9.3 Sharp recurrence orders of the finite generations

Theorem 9.4 (Exponential growth of minimal recurrence order). Fix $q \geq 3$ and $k \geq 0$, and write $D = 2^k$. The ordinary length generating function of $S_q^{[k]}$, in reduced form over $\mathbb{Q}$, has denominator

$$(1 - z)(1 - z^q)^D. \quad (26)$$

Its minimal eventual constant-coefficient recurrence has order $qD + 1 = q2^k + 1$. For each fixed k , its coefficients grow polynomially of degree D , while the full hull has exponential growth rate $4^{1/q}$.

Proof. The polynomial

$$b_D(m) = \sum_{j=0}^D \binom{2m}{j}$$

has degree exactly D and leading coefficient $2^D/D!$. Interpreting each binomial as a polynomial in m agrees with the combinatorial convention at all $m \in \mathbb{N}$, since the falling factorial vanishes when $j > 2m$. Consequently

$$\sum_{m \geq 0} b_D(m) t^m = \frac{Q_D(t)}{(1-t)^{D+1}}, \quad \deg Q_D \leq D, \quad Q_D(1) = 2^D \neq 0.$$

One proof is to expand $b_D(m)$ in the binomial-polynomial basis; the coefficient of $\binom{m}{D}$ is $D! \cdot 2^D/D! = 2^D$, and only that term contributes to the numerator at $t = 1$.

Lengths $qm, \dots, qm + q - 1$ have the same coefficient. Thus

$$\begin{aligned} \sum_{n \geq 0} |(S_q^{[k]})_n| z^n &= (1 + z + \dots + z^{q-1}) \frac{Q_D(z^q)}{(1 - z^q)^{D+1}} \\ &= \frac{Q_D(z^q)}{(1 - z)(1 - z^q)^D}. \end{aligned}$$

All roots of the displayed denominator satisfy $z^q = 1$, where its numerator is $Q_D(1) \neq 0$. Hence it is reduced and its degree $qD + 1$ is the minimal recurrence order. Finally $b_D(\lfloor n/q \rfloor) \sim (2^D/(D!q^D))n^D$, whereas $|(K_q)_n| = 4^{\lfloor n/q \rfloor}$ has exponential rate $4^{1/q}$. $\square$

For the frozen-source sequence of Theorem 3.8, the same calculation holds with $D = k + 1$ rather than $D = 2^k$. Its generation- k minimal recurrence order is therefore $q(k + 1) + 1$.

The finite-generation and full-closure limits therefore do not commute at the level of exponential growth. Every fixed generation has growth rate one, but their increasing union has larger growth rate. This supplies a concrete warning against extrapolating an infinite closure from a few rational finite-stage enumerators.

q	generation k	rank threshold 2^k	polynomial growth degree	recurrence order
3	0	1	1	4
3	1	2	2	7
3	2	4	4	13
3	3	8	8	25
4	2	4	4	17
5	3	8	8	41

10 Verification, reproducibility, and proof boundaries

10.1 Executed exact audits

The accompanying standard-library Python program performs independent finite checks using explicit source sets, direct crossover, interval dynamic programming, finite-state paths, and integer counting. The delivered run completed **1,766,639 checks**, all passing. The JSON receipt records the detailed counts; these are finite audits, not universal proof certificates.

Audit	Executed scope
All finite binary slices	All 65,814 seed subsets at lengths 0, 1, 2, 3, 4; 1,050,698 target-rank queries. Checks exact contraction, coordinate hull, every finite rank, generation thresholds, frozen-source generations, and stabilization.
All two-state binary DFAs	All 64 complete transition/final-state choices with fixed initial state; 8,128 target queries at lengths 0 through 6. Compares source enumeration, DFA fragment rank, and the distance-automaton cost.
Guard repair	All 255 omitted binary patterns $00x$ of lengths 2 through 9; checks the actual same-length parent repair.
Block amplification	Ranks and distances of repeated forbidden block 00 for 1 through 64 blocks. Independent enumeration over the full four-letter alphabet at lengths 2, 5, 8.
The linear-seed family	$q = 3, \dots, 7$, all binary target words through length 12; exact ranks, weighted histograms, and finite-stage counts.
Recurrence checks	25 parameter pairs, $q = 3, \dots, 7$ and $k = 0, \dots, 4$; integer convolution with the proved denominator and nonzero top finite differences.

For the distance-automaton audit, the accepting boundary condition forces a unique sequence of right subsets V_i . The program computes that sequence backward rather than enumerate unsuccessful guesses. It then evaluates the cost transitions separately from interval-partition dynamic programming. This checks the accepted-path cost semantics; it does not implement or test the general limitedness algorithm imported from the literature.

The block test uses the regular specialization $G = \{0, 1, @\}^* \setminus \{00\}$. This does not prove the grammar undecidability reduction. It checks the combinatorial amplification while allowing arbitrary source separator placements, including the placements that could invalidate a careless lower-bound proof.

10.2 Files and rebuilding

The main source is standalone and has its bibliography internally. The package layout is

```
ProveIt_Constrained_Crossover/
  article.tex
  article.pdf
  README.md
  Makefile
  code/verify.py
  data/verification.json
  data/verification.txt
  notes/provenance.md
  notes/proof-audit.md
```

From the package directory, run

```
python3 code/verify.py
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

The verifier uses no external packages and no floating-point mathematics; the only floating-point output is elapsed wall-clock time. Assertions are implemented as explicit exception-raising checks and remain active under `python -O`. The LaTeX build uses commonly distributed packages, including `newtxtext` and `newtxmath`.

[write] *Shipped layout*. In the collection the files live at the report root,

```
SetTheory/Cardinals/docs/reports/
automata-and-formal-languages/constrained-crossover-closure/.
```

There, `notes/provenance.md` and `notes/proof-audit.md` are shipped as `provenance.md` and `proof-audit.md`, and `Makefile` as `code/Makefile`; the other paths are unchanged, and all these files are byte-identical to the delivery. The delivered README and 23-page PDF are not shipped: the report's README replaces the former, and `article.pdf` is a build of this text. The verifier writes `../data/verification.json` relative to its own directory, so run in place it overwrites the shipped receipt; the README gives commands for a copy. The build now also uses the `shuffle` package, for the sibling report's operation in Section 1.4. [Added 30 September 2026, batch 66: this section describes Part I's files. Part II's files carry the prefix `02-pspace-stabilization-` and Part III's the prefix `03-closure-classification-`; Sections 23 and 45 list them. Both of their verifiers also write `data/verification.json` by default, that is, over Part I's receipt, and Part III's verifier and Part II's example program import sibling modules under their delivered names, so they are rerun only on copies, as the README describes.]

10.3 What was not verified or claimed

No Lean or Rocq kernel has checked the proofs. The external Parikh, Presburger, and distance-limitedness theorems are explicitly cited mathematical inputs, not imported formal declarations. No general CFG-to-Presburger compiler, limitedness solver, or full symbolic rational-enumerator implementation is included. The polynomial-time grammar rank algorithm is proved but the executable reference verifier specializes to finite seeds and DFAs.

The novelty search inspected the cited Hughes version, the relevant ProveIt source and collection catalogue, and targeted literature on crossover, universality, effective Parikh images, Presburger counting, and distance automata. It did not identify a prior resolution of the specific constrained one-step question or an equivalent package of the quantitative results. Search coverage is not an exhaustive bibliography and is not a proof of priority. In particular, the product-hull observation and possible equivalent rank formulations deserve review by specialists in recombination systems.

[write] Placement in ProveIt, a repository with Lean and Rocq developments, confers no formal status on this report, and none of its statements is formalized. ProveIt has no formal development of crossover, aligned rank, context-free grammars, Post correspondence, Parikh images, Presburger counting or distance-automaton limitedness. The nearest formal neighbour is `Logic/PresburgerArithmetic`, whose Lean development proves Cooper quantifier elimination for Presburger arithmetic over $\mathbb{Z}$ (`PresburgerArithmetic.Formula.holds_iff_quantifierEliminate` and the decision procedure `PresburgerArithmetic.Formula.presburgerArithmetic_decidable`, in `Lean/PresburgerArithmetic/Decision.lean`), with an independent Rocq proof of the one-variable Cooper step. The effectivity argument of Lemma 8.2 and Appendix B invokes quantifier elimination of this kind but is not connected to that development, and the counting theorem it needs is not formalized anywhere in ProveIt. The Lean plan of Section 11.1 is a plan, not an executed development.

11 Further research questions and a formalization plan

The following are proposed research tasks relative to the present results. Only the Hughes question identified in Section 1 is assigned a specific published open-problem provenance. The rest are not asserted to be newly discovered or globally open without further literature review.

Question 11.1 (Exact arithmetical complexity). Is finite stabilization of a context-free seed Σ_2^0 -complete? The present proof gives a Σ_2^0 upper bound and Π_1^0 -hardness, not a matching lower bound. A stronger construction must encode a genuinely existential bound rather than just universality of one grammar.

[Added 30 September 2026, batch 66: Still open. Part II restates it (Question 24.6) and explains why its regular-input analysis does not reach it.]

Question 11.2 (Sharp regular-input complexity). What is the optimal complexity of deciding finite stabilization for DFA and NFA inputs? Can the $s4^s$ distance-automaton construction be represented more economically, or can its special two-sided subset structure yield a better algorithm than generic limitedness? Matching hardness results would determine whether the EXPSPACE upper bound is loose.

[Added 30 September 2026, batch 66: Answered for NFAs in Part II: finite stabilization of NFA input is PSPACE-complete, already over a binary alphabet and for factorial languages with universal hull (Corollary 20.3); the special two-sided structure does give a better algorithm than generic limitedness (Theorem 18.3). For DFAs, Part II gives a PSPACE upper bound (Theorem 19.2), so the EXPSPACE bound is loose, and a safe-core criterion (Theorem 21.2); the exact DFA complexity stays open (Question 24.1). Part III's Theorem 39.2 gives coNP-completeness of a depth threshold for succinct ray inputs, a different input format.]

Question 11.3 (Two-letter undecidability). Can Theorems 4.2 and 5.2 be proved over a binary alphabet? Fixed-length coding does not automatically preserve one-step crossover: a cut inside a codeword can create spurious children. A successful binary encoding must control such internal cuts, not merely preserve language universality.

Question 11.4 (Grammar classes preserving context-freeness). Which context-free seeds have a context-free coordinate hull? The linear counterexample rules out linearity as a sufficient condition. Restrictions on the position-support sets E_a , rather than just grammar ambiguity or linearity, may provide a useful criterion. Deterministic and visibly pushdown subclasses are natural places to investigate.

[Added 30 September 2026, batch 66: Partly answered in Part III. For every context-free seed, regularity of the hull is decidable, by finite row type of the supports E_a , with a DFA or an infinite obstruction (Theorems 33.1 and 33.2). For binary one-marker seeds $L_R = 0^* \cup \{0^i 10^j : (i, j) \in R\}$ with semilinear R and finite stabilization, the hull is regular, linear context-free and non-regular, or not context-free, according to a test on the velocities of the arithmetic rays of R (Theorem 37.1, Corollary 37.2), a criterion of exactly the suggested kind. The general question, including nonsparse masks (Question 42.1) and the deterministic and visibly pushdown subclasses, stays open.]

Question 11.5 (Faster target-word algorithms). Can the $O(gn^5)$ wildcard-parsing method be reduced to one shared chart computation of cubic or near-cubic complexity? A useful implementation should return a shortest fragment partition and full seed witnesses, not just a membership bit, so that crossover derivations can be independently checked.

Question 11.6 (Which rational counting series occur?). Characterize the commutative series realized by context-free recombination closures. Theorem 8.3 forces eventual products of linear forms along arithmetic progressions. This is more restrictive than arbitrary rational nonnegative series. Which compatibility constraints on the semilinear position supports are necessary and sufficient for realization?

[Added 30 September 2026, batch 66: Part III adds examples, not a characterization: the series $(1 + z + 4z^2)/(1 - z^3)$ of a non-context-free closure with at most four words per length (Theorem 40.1) and the family (76).]

Question 11.7 (Restricted cuts and multipoint crossover). For cut positions restricted by a regular or Presburger condition, what replaces aligned fragment rank? For crossover with several cuts between the same two parents, different fragments are coupled by parent reuse; simply replacing 2^k by a larger exponential is not justified without a new proof.

[Added 30 September 2026, batch 66: Part II’s Question 24.7 asks the same for its periodic dichotomy; neither added Part answers it.]

Question 11.8 (Quantitative transition windows). For S_q , the proportion reached by generation k at length $qm + r$ is $4^{-m} \sum_{j \leq 2^k} \binom{2^m}{j}$. Study uniform asymptotics when k grows with m , rather than keeping k fixed. The explicit binomial law gives a tractable test case for crossover depth distributions in less rigid seeds.

Question 11.9 (Sharp automaton witness bounds). Determine the best possible shortest-violation bound for an s -state DFA. The $2s^3 - 1$ bound comes from a direct triple-state graph. It may not be sharp because the three components share one transition structure. Explicit lower bound families and state-complexity analyses would be informative.

[write] *Bearing on Hughes’s Sections 13–14*. Hughes asks, for his insertion and shuffle iterations, for the exact arithmetical complexity of finite convergence, for restricted language families on which it becomes decidable, for whether the least stabilization index can be computed, and for a general “rank condition” behind his separator proofs; his Section 13 asks for which monotone operators eventual reaching of a fixed point is decidable. For constrained self-crossover, which is monotone and inflationary (Lemma 2.2), the results above give the following counterparts. Single-step and each fixed adjacent equality are co-r.e.-complete for grammars (Theorems 4.2 and 5.2); eventual stabilization is in Σ_2^0 and Π_1^0 -hard (Proposition 5.4), and Question 11.1 asks for the exact level. Regular inputs make stabilization decidable, and the least stage is then computable (Corollary 7.4 and the paragraph after it); every slice stabilizes by stage $\lceil \log_2 n \rceil$ (Corollary 3.5). Aligned-fragment rank with the separator amplification of Theorem 5.1 is a rank-based separator argument for this one operation, not the general condition Hughes asks for. None of this answers his questions for insertion or shuffle. [Added 30 September 2026, batch 66: For regular seeds Part II locates finite stabilization exactly (PSPACE-complete for NFA input) and bounds the least stage (Corollary 19.3); Part III decides it, and computes the least stage, for semilinear one-marker seeds (Theorems 35.1 and 38.2). These too concern crossover only.]

11.1 A realistic Lean development

A formal development can begin with fixed-length words as functions $\text{Fin}(n) \rightarrow \Sigma$, avoiding casts between lists of different lengths. A cut is a boundary $i \leq n$, and its child chooses the first parent before i and the second afterwards. A compatible partition is a strictly increasing finite boundary list, with one seed witness per interval.

The first milestones should be, in order, the two fragment-transfer lemmas used in Theorem 3.2, the exact iterate formula, the no-gap prefix-replacement argument, and the block-boundary charging lemma. These are finite combinatorial statements independent of grammars and automata, and would establish the core trusted interface.

A second layer can define an executable compatibility matrix and prove that the boundary shortest-path program computes the least rank. The finite reference tests can then be replaced by proved checkers over this interface. For the decision-theoretic reductions, a separate grammar layer must certify regular union, prefixing, concatenation, and star constructions and connect them to a formally stated universality theorem. The present paper does not claim that such a bridge already exists in ProveIt.

The counting theorem requires substantially different infrastructure: effective semilinearity of the marked grammar, Boolean Presburger operations, and the affine residue-class counting lemma. Its dependencies should be tracked separately rather than presenting the whole article as formalized because its finite combinatorics has been checked.

12 Conclusion

Constrained crossover is length-preserving, but length preservation does not make its global grammar problems decidable. A one-letter guard repairs any suitably padded omission in one generation, settling the specific one-step question. Repeated separator-delimited omissions then give rank exactly one more than the number of bad blocks, separating local logarithmic convergence from global finite stabilization.

The aligned-fragment invariant unifies the structural and algorithmic results. It computes exact depth, forces gap-free rank spectra, provides a polynomial local membership procedure, and turns regular-input stabilization into distance-automaton limitedness. At the counting level, semilinear coordinate supports imply rational commutative growth for every context-free seed, but not context-freeness of the generated language. The explicit linear family demonstrates this separation and gives exact recurrence orders at every finite generation.

The principal proofs stand independently of the finite experiments. The experiments supply a reproducible adversarial audit of their finite mechanisms. Independent proof review, priority checking, and formalization are the next steps before treating the draft as a settled research publication.

[Added 30 September 2026, batch 66: Parts II and III continue this account: the regular-input stabilization problem is PSPACE-complete for NFAs, with a quadratic rank bound replacing the generic limitedness bound, and the full closure of a context-free seed has a decidable regularity problem, while sparse one-marker seeds can lose context-freeness after a single generation. Both are equally unrefereed and unformalized.]

Part II. Finite stabilization of regular seeds is PSPACE-complete

[write, batch 66] Sections 1–12 and Appendices A–D form the original report (Part I) of 29 September 2026, unchanged except for dated notes that point to the additions; its appendices are printed after Part III. Part II was added on 30 September 2026 in batch 66 of ProveIt’s incoming-report intake, from one manuscript written against Part I as printed above. For regular seeds it decides finite stabilization of constrained self-crossover in polynomial space, proves the problem PSPACE-complete for NFA input over a binary alphabet, and bounds the rank quadratically in the stabilizing case. It thereby answers the NFA half of Part I’s Question 11.2 and improves the EXPSPACE bound of Corollary 7.4 to PSPACE; the exact complexity for DFA input stays open. Section 13 records the provenance and fixes the notation of the addition.

13 [write, batch 66] Provenance and notation of Part II

13.1 One manuscript, one addition

Part II is batch-66 manuscript 01, *Finite Crossover Stabilization Is PSPACE-Complete: periodic interior universality, quadratic rank bounds, and linear-growth obstructions*, dated 30 September 2026 (23 US-letter pages as delivered). It is an AI-assisted research draft: its title page and PDF metadata name ChatGPT as the drafting assistant and Vladimir Reshetnikov as the person it was prepared for. It arrived as the archive ProveIt_Crossover_Stabilization.zip (inner directory of the same name) in ProveIt commit 9ad899cbe and was placed in commit 4fee1cd07. It was written against ProveIt commit b8b0fa218 (in full b8b0fa2184a044d46ce9ed0f25f88d7bb60fa042), at which Part I’s `article.tex` had the Git blob eb4f427f; that blob is unchanged at the placement commit, so every statement the manuscript makes about “the inspected report” refers to Part I as printed above. Its code, data and audit notes are shipped beside Part I’s files with the prefix 02-pspace-stabilization- (Part I’s unprefix files count as 01); its manuscript, delivery README and PDF are not shipped and survive in the arrival commit. Section 23 and the report’s README list the shipped names.

Every result, proof, example, remark, figure, table, limitation and question of the manuscript is printed here, in its order. Its title page, abstract and status box became this section; its Sections 1–13 are Sections 14–26; its two appendices became Sections 27 and 28, because the report’s appendices belong to Part I. Its own reproof of Part I’s rank calculus (Proposition 15.2) is printed as a marked second proof of results stated once, in Part I; Part III reproves the coordinate hull once more, by a third route (Lemma 31.3). Text marked [write, batch 66] was added when the manuscript was written into this report; no theorem was strengthened or weakened.

13.2 The manuscript’s summary and status

Abstract, as delivered (with “the inspected ProveIt crossover report” now Part I). For a regular language, does repeated equal-length, aligned one-point self-crossover reach its full closure after finitely many generations? We prove that this problem is PSPACE-complete for NFA input, already over a binary alphabet and even on factorial languages whose coordinate hull is universal. The upper bound also applies to DFA input, improving the EXPSPACE upper bound of Part I (Corollary 7.4).

The proof replaces generic distance-automaton limitedness by a finite periodic-interior universality test. Either every compatible interior word is one aligned seed fragment, or one forbidden word can be repeated to

force linearly many fragments. In the first case, for an s -state NFA, every word in the closure has rank at most $(2s + 5)^2$, so parallel stabilization occurs by $\lceil \log_2(2s + 5)^2 \rceil$ generations. In the second case the maximum rank is linear along every sufficiently large length in at least one arithmetic progression. We also give a deterministic safe-core criterion, a polynomial-in-period obstruction bound for DFAs, and a logarithmically sharp generation lower family. Complete proofs, reproducible finite audits, and a research agenda accompany the article.

Status and scope, as delivered. Unrefereed mathematical research draft, not a Lean or Rocq development. The NFA complexity question is settled here; the exact DFA complexity and the optimal rank bound are not. Priority is not asserted: the literature audit is targeted, and the claimed extension is measured against a pinned repository revision.

13.3 Notation beside Part I and Part III

Part II uses Part I's operation $\otimes_c$, crossover map $T(L) = L \otimes_c L$, parallel iterates $L^{[k]}$, frozen-source iterates Y_k , aligned-fragment rank ρ_L and coordinate hull $\text{Rec}(L)$ (Definitions 2.1, 2.4 and 3.1). Three symbols were renamed when the manuscript was written in: its coordinate hull $\mathcal{H}(L)$ is printed as Part I's $\text{Rec}(L)$ (the same set, with the same length-zero convention), and its delimiter languages $G(K)$, $B(K)$ of Section 20 are printed as $\widehat{G}(K)$, $\widehat{B}(K)$, because they are *not* Part I's G_K , B_K . No normalization changed. The table lists the other symbols that could be misread.

In Part II	Meaning in Part II	Not to be confused with
$Y_{k+1} = Y_k \otimes_c L$	frozen-source iterates	Part I writes $L \otimes_c Y_k$; since $\otimes_c$ returns both children, the two are the same set
$\widehat{G}(K)$, $\widehat{B}(K)$	regular delimiter languages over $\{0, 1, @ \}$ and $\{0, 1, @, \# \}$ (Proposition 20.4)	Part I's guard and block languages G_K , B_K (6), (7), which are different sets
$M_L(n)$	largest rank on the length- n hull slice; 0 on an empty slice	the same as Part I's $R_L(n)$ (Theorem 3.4) on active slices
$R(L)$	supremum of all finite ranks, with 1 adjoined	Part I's $R_L(n)$ (one slice); Part III's $R \subseteq \mathbb{N}^2$ (a marker mask)
P_i , R_j	forward and backward exact-length state sets of an NFA (32)	Part I's coordinate sets $P_{n,i}(L)$; Part I's U_i , V_i (Section 7) are the DFA case P_i , R_{n-i}
$A_{n,i}$	letters allowed at zero-based position i (35)	Part I's $P_{n,i+1}(L)$ (one-based); Part III's $A_L(n, i)$ (zero-based, the same set)
$S_{n,i}$, $S_{r,a}$, $C_{r,a}$	viable state sets and stable alphabets	Part I's seed family S_q and the factors $C_r(\mathbf{x})$ of Theorem 8.3
m , T_s , p_s , B_s , K_s	the constants (27), $K_s = \lceil \log_2 B_s \rceil$	the operator $T(L)$; Part I's B_K , K , K_q ; Part III's gap-ray languages $\text{Gap}_s(a, d)$
$\mathcal{R}$	set of live residues	Part I's index set $\mathcal{R}$ of eventually active residue classes (Theorem 8.3), a similar notion for context-free seeds
L_m	the one-1 family of Theorem 22.1	Part III's family L_k^{fam} and seeds L_R ; $L_m = L_R \setminus 0^*$ for the finite mask
$D_L(n)$, d_j	largest parallel depth on a slice; the rank dynamic program	$R = \{(i, m - 1 - i) : i < m\}$ Part I's $d_L(w)$ (depth of one word), its D in Theorem 9.4 and $D[j]$ in Proposition 6.1
Γ	the alphabet $\{0, 1, @ \}$	the same as Part I's Γ ; Part III's graph Γ_R

Positions inside a word are zero based here ($v = v_0 \cdots v_{\ell-1}$), as in Part III; Part I indexes letters from one and boundaries from zero, and the interval notation $w[i : j]$ is the same in all three Parts.

14 The question and the contribution

14.1 A precise gap in the repository

[write, batch 66] “The ProveIt report” of this section is Part I.

The ProveIt report *Aligned Fragments and Constrained Crossover* (Part I) develops a fragment-rank calculus for constrained crossover. Its regular-input section (Section 7) constructs a distance automaton with at most $s4^s$ states (Theorem 7.3) and obtains an EXPSPACE upper bound for finite stabilization from a complete s -state DFA (Corollary 7.4). Its Section 11 question headed *Sharp regular-input complexity* (Question 11.2) asks whether the special two-sided subset structure permits a better algorithm and asks for matching hardness results for DFA and NFA inputs.

This is the question pursued here. The inspected repository revision is

b8b0fa2184a044d46ce9ed0f25f88d7bb60fa042.

The relevant report is at

SetTheory/Cardinals/docs/reports/
automata-and-formal-languages/constrained-crossover-closure/.

The source and README were inspected, with the regular-input conclusion and its explicit open question retrieved separately. We do not present the older fragment formulas, the coordinate-hull observation, or the report’s context-free undecidability theorem as new results.

Hughes’s preprint [1] supplies the surrounding language-operation setting. The present problem concerns *self*-crossover with equal parent lengths and identical cut positions. It is not unrestricted splicing, insertion degree, or arbitrary-length crossover. In particular, we do not claim to be solving anew the context-free one-step question already answered in the repository report (Theorem 4.2).

14.2 Main theorem

An NFA may have several initial states and no ε -transitions. A single-initial-state convention gives the same complexity results, as explained in Section 20. Let $s \geq 1$ be its number of states. Set

$$m = s + 2, \quad T_s = 2m^2 + 2m, \quad p_s = \text{lcm}(1, 2, \dots, m), \quad B_s = 2T_s + 1 = (2s + 5)^2. \quad (27)$$

The constants are deliberately conservative. The relevant qualitative facts are a quadratic transient, a period with polynomially many bits, and a rank bound independent of that period.

Theorem 14.1 (Main result). *Let L be recognized by an s -state NFA.*

1. *Finite stabilization of parallel constrained self-crossover is decidable in PSPACE. It is PSPACE-complete already for a fixed binary alphabet, even for factorial languages with universal coordinate hull. The same statements hold for frozen-source self-crossover.*
2. *If stabilization occurs, every word of the full closure has aligned fragment rank at most B_s . Hence the parallel sequence stabilizes by generation $\lceil \log_2 B_s \rceil$ and the frozen-source sequence by generation $B_s - 1$.*
3. *If stabilization does not occur, there are constants $c > 0$, C , and a residue r modulo p_s such that, for every sufficiently large $n \equiv r \pmod{p_s}$, the maximum finite rank on length n lies between $cn - C$ and n .*

4. For DFA input, a periodic safe-core criterion decides the same property. It gives an interior obstruction of length at most $p_s s^2$ whenever stabilization fails. The exact complexity for DFA input is not determined here.

The proof is divided into independent components. Section 15 recalls and proves the rank interpretation. Sections 16–18 establish the structural dichotomy. Section 19 derives the space upper bound, and Section 20 proves binary hardness. Section 21 handles the deterministic refinement.

14.3 What changes, and what does not

Question	Inspected report	This article
DFA finite stabilization	Decidable; EXPSPACE upper bound	PSPACE upper bound; periodic safe-core criterion
NFA finite stabilization	Sharp complexity asked explicitly	PSPACE-complete over two letters
Bound in the stabilizing case	Reduction to generic limitedness	Rank at most $(2s + 5)^2$; logarithmic generation bound
Nonstabilizing regular inputs	Unbounded finite rank	Linear maximum rank on a live residue class
Exact DFA complexity	Open in the report	Still not settled here

The article’s mathematical claims are accompanied by proofs rather than an assertion that no equivalent theorem has ever appeared. The bounded source audit covers the pinned report, Hughes, the unary-periodicity input, and the NFA universality input. It does not exclude all equivalent statements in recombination, symbolic dynamics, or semigroup literature. The software checks finite instances; it neither proves the complexity lower bound nor substitutes for the universal arguments.

15 Crossover, aligned fragments, and the two clocks

15.1 Definitions, including the empty word

Fix a finite nonempty alphabet Σ . For words x, y of a common length n and a boundary $0 \leq i \leq n$, an aligned crossover output is $x[0 : i]y[i : n]$. Both choices of parent order are allowed, and endpoint cuts are permitted. For languages A, B define

$$A \otimes_c B = \{x[0 : i]y[i : n], y[0 : i]x[i : n] : x \in A_n, y \in B_n, 0 \leq i \leq n\},$$

where the displayed set ranges over all $n \geq 0$ and $A_n = A \cap \Sigma^n$. Define

$$T(L) = L \otimes_c L, \quad L^{[0]} = L, \quad L^{[k+1]} = T(L^{[k]}),$$

and the frozen-source sequence

$$Y_0 = L, \quad Y_{k+1} = Y_k \otimes_c L.$$

Both sequences preserve the set of lengths present in L ; endpoint cuts then make both sequences increasing. These are different clocks. In a parallel generation both parents may already be recombinants; in a frozen generation one parent must be a seed.

Definition 15.1 (Aligned fragment rank). For a nonempty word w of length n , a nonempty interval $[i, j]$ is *admissible* if some $v \in L_n$ satisfies $v[i : j] = w[i : j]$. The rank $\rho_L(w)$ is the smallest number of admissible intervals in a partition

$$0 = i_0 < i_1 < \dots < i_d = n.$$

Set $\rho_L(w) = \infty$ if no such partition exists. For the empty word, set $\rho_L(\varepsilon) = 1$ when $\varepsilon \in L$, and ∞ otherwise.

The total length and the coordinates in this definition are indispensable. An ordinary factor occurring somewhere in a seed of an unrelated length need not be an admissible aligned fragment.

For $n \geq 1$, the coordinate hull is

$$\text{Rec}(L)_n = \prod_{i=0}^{n-1} \{a \in \Sigma : \exists v \in L_n (v_i = a)\}. \quad (28)$$

If L_n is empty the product is empty. At length zero we stipulate $\text{Rec}(L)_0 = L_0$, rather than using the unconditional empty-product convention. Thus ε is never introduced from an empty length-zero seed.

[write, batch 66] The operation, the two clocks, Definition 15.1 and (28) restate Part I's Definition 2.1, Theorem 3.8's iteration, Definition 3.1 ("admissible" here is Part I's " L -compatible") and Definition 2.4, with letters indexed from zero. They are printed because the proofs below are written in this form; they define nothing new.

15.2 Rank formulas, credited background

The following formulas are part of the earlier ProveIt report, Part I (Theorems 3.2, 3.3 and 3.8, and Corollary 3.5). Their proofs are included to make the dependency on rank completely transparent. [write, batch 66] The statements are printed once, in Part I; the proposition and corollary below restate them in Part II's notation, and their proofs are a second route, by the manuscript. Part III proves the hull formula (31) by a third route (Lemma 31.3).

Proposition 15.2 (The fragment interpretation; Part I, restated). *For every $k \geq 0$,*

$$L^{[k]} = \{w : \rho_L(w) \leq 2^k\}, \quad Y_k = \{w : \rho_L(w) \leq k + 1\}. \quad (29)$$

Moreover,

$$\rho_{T(L)}(w) = \left\lceil \frac{\rho_L(w)}{2} \right\rceil \quad (30)$$

with the natural convention for infinity, and

$$\bigcup_{k \geq 0} L^{[k]} = \bigcup_{k \geq 0} Y_k = \text{Rec}(L). \quad (31)$$

Second proof (manuscript 01). A fragment of a word in $T(L)$ meets at most two pieces inherited from seed parents. Intersect it with that parent's crossover cut, discarding any empty intersection. Therefore a partition into d $T(L)$ -fragments gives one into at most $2d$ seed fragments.

Conversely, pair consecutive pieces of a shortest seed-fragment partition. For a pair, choose a seed parent supplying each piece and cross the parents at their shared boundary. The pair is then one admissible $T(L)$ -fragment. A final unpaired piece remains admissible because $L \subseteq T(L)$. This proves (30). Iteration proves the first formula of (29), since rank at most one is exactly seed membership.

A frozen crossover appends, or prepends, at most one seed fragment to an inherited rank partition, so every word in Y_k has rank at most $k + 1$. For the reverse inclusion, take a partition into $d \leq k + 1$ seed fragments and corresponding parents $v_1, \dots, v_d$. Starting with v_1 , cross successively with $v_2, \dots, v_d$ at the partition boundaries. After the j th parent has been used, the prefix through that parent's fragment is correct; the last parent supplies the remaining suffix. This constructs w in $Y_{d-1} \subseteq Y_k$.

Finally, a nonempty word has finite rank exactly when each of its coordinates is supplied by a length-matched seed: singleton pieces give the reverse implication. That is (28). The empty word follows from its stipulated convention. $\square$

Corollary 15.3 (Finite stabilization is bounded rank). *Put*

$$R(L) = \sup(\{1\} \cup \{\rho_L(w) : w \in \text{Rec}(L)\}).$$

The parallel sequence stabilizes at a finite generation if and only if $R(L) < \infty$, and the same is true for the frozen-source sequence. When finite, the least stabilization indices are respectively $\lceil \log_2 R(L) \rceil$ and $R(L) - 1$.

Second proof (manuscript 01); Part I's Corollary 3.5 and Theorem 3.8. Both operators are monotone and extensive along their respective sequences. An adjacent equality therefore makes all later stages equal, so a stabilized value must be the union (31). The rank thresholds (29) give all the assertions, including $L = \emptyset$ through the extra element 1 in the supremum. $\square$

The trivial slice bound is $\rho_L(w) \leq |w|$ for nonempty hull words. What is not trivial is a bound uniform in word length. The rest of the article explains why regularity forces such a bound to be small whenever it exists.

16 The exact-length geometry of an NFA

Let $\mathcal{A} = (Q, \Sigma, (\Delta_a)_{a \in \Sigma}, I, F)$ be an ε -free NFA, with $|Q| = s$. A transition relation acts on subsets by

$$\Delta_a(X) = \{q' : \exists q \in X (q, q') \in \Delta_a\}.$$

For a word x , let Δ_x be the composed relation, with Δ_ε the identity. Put $E = \bigcup_a \Delta_a$. Forgetting labels but counting edges defines the exact-length reachability sets

$$P_i = E^i(I), \quad R_j = \{q : \exists f \in F (q, f) \in E^j\}. \quad (32)$$

Thus $P_0 = I$, $R_0 = F$, $P_{i+1} = E(P_i)$, and $R_{j+1} = E^{-1}(R_j)$.

For a fixed total length n , the states which can occur after i letters in some accepting length- n computation are

$$S_{n,i} = P_i \cap R_{n-i} \quad (0 \leq i \leq n). \quad (33)$$

An NFA run may be chosen separately for each admissible fragment. There is no requirement that different fragments use compatible seed runs.

Lemma 16.1 (Aligned fragment criterion). *For $0 \leq i < j \leq n$, a word x of length $j - i$ can occur as an aligned seed fragment on $[i, j]$ if and only if*

$$\Delta_x(P_i) \cap R_{n-j} \neq \emptyset. \quad (34)$$

Equivalently, there is a path labelled x through the successive viable layers $S_{n,i}, \dots, S_{n,j}$.

Proof. An accepting seed run supplies the path and hence (34). Conversely, a path counted by (34) extends backwards by some length- i path from an initial state and forwards by some length- $(n - j)$ path to a final state. The labels of those extensions may be arbitrary. The resulting accepting word has the prescribed fragment at precisely the prescribed coordinates. Every intermediate state of this completed run lies in its viable layer, which proves the equivalent formulation. $\square$

Corollary 16.2 (Coordinate alphabets). *The letters available at position i of a length- n seed are exactly*

$$A_{n,i} = \{a \in \Sigma : \Delta_a(S_{n,i}) \cap S_{n,i+1} \neq \emptyset\}. \quad (35)$$

For $n \geq 1$, $\text{Rec}(L)_n = \prod_{i < n} A_{n,i}$.

Only two unary processes, P_i and R_j , encode the dependence on the absolute length. The labelled transitions still encode arbitrary nondeterminism, but away from the two boundaries their allowed state sets will be periodic.

16.1 A direct fixed-word rank algorithm

For later audits, it is useful to record the elementary dynamic program. Given $w \in \Sigma^n$, precompute P_i and R_j for $i, j \leq n$ and set $d_0 = 0$. For $j = 1, \dots, n$, let

$$d_j = 1 + \min\{d_i : 0 \leq i < j, \Delta_{w[i:j]}(P_i) \cap R_{n-j} \neq \emptyset\}, \quad (36)$$

where a minimum over the empty set is infinity. Then $d_n = \rho_L(w)$ for $n > 0$. Maintaining the subset image as j increases for each fixed i implements the interval tests in polynomial time. The finite verifier independently compares this path-based algorithm with literal seed-substring enumeration; they do not share an interval oracle.

17 Uniform periodicity and its quantitative cost

17.1 Elementary eventual periodicity

The recurrence taking (P_i, R_i) to (P_{i+1}, R_{i+1}) is deterministic on at most 4^s pairs. Thus there exist $t \geq 0$ and $p \geq 1$ such that

$$P_{i+p} = P_i, \quad R_{i+p} = R_i \quad (i \geq t), \quad (37)$$

and one can choose $t + p \leq 4^s$. This elementary observation suffices for the qualitative structural theorem and, with polynomial-bit counters, for the space upper bound. A stronger classical unary result supplies the quadratic rank bound.

17.2 The imported unary theorem, with the normalization exposed

We use the corrected unary arithmetic-progression theorem of To [8]. In its normalized m -state form, the accepted lengths are a finite union of singleton exceptions no larger than $2m^2 + m$ and arithmetic progressions whose positive periods are at most m and whose initial terms are smaller than $2m^2 + 2m$. These constants are taken from the *corrected* proof in Section 3 of that paper, not the earlier argument whose gap the paper identifies.

Lemma 17.1 (A common quadratic transient). *For an s -state NFA, (37) holds with $t = T_s$ and $p = p_s$ as defined in (27). Moreover $\log_2 p_s = O(s \log(s + 2))$.*

Proof. Fix a target state q . Regard E as a unary NFA transition relation. Add a new source with one unary edge to each state of I , and a new sink with one unary edge from q . The new source has no incoming edges and the new sink no outgoing edges. A path of length i from I to q in the old graph exists exactly when the normalized graph accepts length $i + 2$. There are $m = s + 2$ states. Empty path languages cause no difficulty.

Beyond T_s , every singleton exception has disappeared and every positive progression is active. Membership is unchanged on adding a common multiple of all its periods. Since every period is at most m , p_s is such a common multiple. The harmless length shift by two is already past the same threshold whenever $i \geq T_s$. Thus membership of q in P_i is p_s -periodic there. The same normalization with source q and target set F proves the assertion for R_i . Taking all states does not change the common threshold or the common period.

Finally $p_s \leq m! \leq m^m$, giving the bit-length estimate. No optimal estimate for the least common multiple is needed. $\square$

The imported theorem is used only for this quantitative improvement. The structural mechanism below uses arbitrary t, p satisfying (37); it does not require a minimal period, nor does a large period force a large stabilization rank.

18 Periodic interiors and the obstruction dichotomy

Fix t, p satisfying (37). All phase arithmetic in this section is modulo p . Define

$$P_a^\infty = P_i \quad (i \geq t, i \equiv a \pmod{p}), \quad R_a^\infty = R_i \quad (i \geq t, i \equiv a \pmod{p}).$$

These definitions are independent of the representative. A residue r is *live* when

$$P_r^\infty \cap F \neq \emptyset. \tag{38}$$

Let $\mathcal{R}$ denote the set of live residues. For $n \geq t$, $L_n \neq \emptyset$ precisely when $n \bmod p \in \mathcal{R}$.

18.1 The finite periodic system

Fix a live residue r . At phase a , put

$$S_{r,a} = P_a^\infty \cap R_{r-a}^\infty, \tag{39}$$

$$C_{r,a} = \{c \in \Sigma : \Delta_c(S_{r,a}) \cap S_{r,a+1} \neq \emptyset\}. \tag{40}$$

The set $S_{r,a}$ is the viable state set at a sufficiently interior boundary of phase a in total length residue r . The set $C_{r,a}$ is the corresponding coordinate alphabet.

Lemma 18.1 (Nonvacuity and exact agreement). *For live r , all sets $S_{r,a}$ and $C_{r,a}$ are nonempty. If $n \equiv r \pmod{p}$ and $t \leq i \leq n - t$, then $S_{n,i} = S_{r,i \bmod p}$. When both boundaries of a letter lie in that interior, $A_{n,i} = C_{r,i \bmod p}$.*

Proof. The equalities follow directly from periodicity and (33)–(35). For nonemptiness, choose an arbitrarily large accepted length in residue r and an interior boundary of phase a . An accepting run supplies a viable state and, with one further interior position available, a permitted letter. $\square$

Definition 18.2 (Phase-admissible and liftable words). A word $v = v_0 \cdots v_{\ell-1}$ is *phase-admissible* at (r, a) if $v_j \in C_{r,a+j}$ for every j . It is *liftable* if there is a path with label v through $S_{r,a}, S_{r,a+1}, \dots, S_{r,a+\ell}$, starting at any state of $S_{r,a}$. A phase-admissible word that is not liftable is an *interior obstruction*.

Equivalently, start with $Z_0 = S_{r,a}$ and compute

$$Z_{j+1} = \Delta_{v_j}(Z_j) \cap S_{r,a+j+1}. \quad (41)$$

Liftability is $Z_\ell \neq \emptyset$. Every phase-admissible single letter is liftable by definition of $C_{r,a}$, so an obstruction has length at least two. This small fact is important when cuts are counted later.

18.2 Universal interiors imply a bounded number of fragments

Theorem 18.3 (Interior universality criterion). *For a regular seed language L , the following are equivalent:*

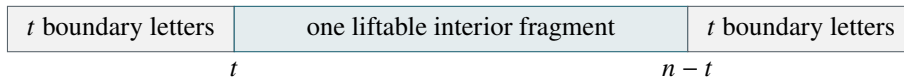
1. *Parallel self-crossover stabilizes after finitely many generations.*
2. *Frozen-source self-crossover stabilizes after finitely many generations.*
3. *At every live (r, a) , every phase-admissible word is liftable.*

When these conditions hold,

$$R(L) \leq 2t + 1. \quad (42)$$

Proof of sufficiency and the rank bound. Let $w \in \text{Rec}(L)$, with $n = |w|$. The empty word has rank one. If $1 \leq n \leq 2t$, singleton fragments give rank at most $2t$. Suppose $n > 2t$. Then $r = n \bmod p$ is live. Every letter of $w[t : n-t]$ belongs to the corresponding $C_{r,a}$ by Lemma 18.1, so this central word is phase-admissible. By condition (3), it is liftable. Lemma 16.1 makes the entire central word one admissible seed fragment.

Partition the first t and last t positions into singletons, retaining the central fragment as one piece. This gives at most $2t + 1$ pieces. If $t = 0$, the central word is the entire nonempty word and the same argument gives rank one. Corollary 15.3 proves both forms of stabilization. The necessity is established in the next theorem. $\square$



18.3 One obstruction forces linear rank growth

Theorem 18.4 (Periodic pumping of an obstruction). *Suppose v is an interior obstruction at a live pair (r, a) . There are fixed words u, b, z with $b \neq \varepsilon$ and $|b| \equiv 0 \pmod{p}$ such that, for every $k \geq 1$,*

$$w_k = ub^kz \in \text{Rec}(L), \quad \rho_L(w_k) \geq k + 1. \quad (43)$$

More strongly, for every sufficiently large $n \equiv r \pmod{p}$ there is $w \in \text{Rec}(L)_n$ with

$$\rho_L(w) \geq \left\lfloor \frac{n - |u| - |z|}{|b|} \right\rfloor + 1. \quad (44)$$

Proof. Extend v on the right by arbitrary phase-admissible letters until its length is a positive multiple h of p . This is possible because every coordinate alphabet is nonempty. Call the result b . It starts at phase a and returns to phase a , so b^k is phase-admissible for every k . Its first $|v|$ positions in each copy form the same obstruction.

Choose lengths $U, V \geq \max(1, t)$ such that

$$U \equiv a \pmod{p}, \quad U + V \equiv r \pmod{p}.$$

The length $U + V$ is at least t and has live residue, so choose an accepted word $uz \in L_{U+V}$ with $|u| = U$, $|z| = V$.

We first check all coordinates of ub^kz . For a prefix position $i < U$, the forward length i has not changed. Its suffix length has increased by kh , which is a multiple of p , and even the shorter original suffix length is at least $V \geq t$. Thus (34) for that single letter has the same truth value as in the accepted word uz . For suffix letters the argument is symmetric: the suffix length is unchanged and the prefix length increases by kh , starting at least $U \geq t$. All middle letters are in the stable alphabets $C_{r,a+j}$. Hence every coordinate is allowed, and $ub^kz \in \text{Rec}(L)$.

Now consider any partition of ub^kz into admissible seed fragments. If one occurrence of v had no cut strictly between its first and last letters, that whole occurrence would be contained in one fragment. Restricting the fragment's witnessing seed run to this occurrence would lift v through its stable viable layers. That contradicts the choice of v . Therefore every occurrence contains an internal partition cut. The k occurrences are disjoint and each has length at least two, so their internal boundary sets are disjoint. There are at least k cuts, hence at least $k + 1$ fragments. This proves (43) and, with Corollary 15.3, completes the necessity in Theorem 18.3.

For an arbitrary sufficiently large $n \equiv r \pmod{p}$, write

$$n - U - V = kh + d, \quad 0 \leq d < h.$$

Both $n - U - V$ and h are multiples of p , so d is a multiple of p . After b^k , insert any phase-admissible word e_d of length d . It also returns to phase a . The same coordinate argument proves $ub^ke_dz \in \text{Rec}(L)_n$, and the same k disjoint occurrences give (44). $\square$

Corollary 18.5 (Residue-wise bounded/linear dichotomy). *For $n \geq 1$, write*

$$M_L(n) = \max\{\rho_L(w) : w \in \text{Rec}(L)_n\},$$

using $M_L(n) = 0$ for an empty slice. On each live residue r , exactly one of the following alternatives holds:

1. $M_L(n) \leq 2t + 1$ for all $n > 2t$ in that residue;
2. $M_L(n) = \Theta(n)$ as $n \rightarrow \infty$ in that residue.

The second alternative occurs precisely when that residue has an interior obstruction. It can occur in some residues and not others.

Proof. Apply the central-fragment proof separately to each residue with no obstruction. Otherwise combine (44) with $M_L(n) \leq n$. These alternatives cannot overlap. $\square$

Corollary 18.6 (Asymmetric boundary bound). *If P_i is p -periodic for $i \geq t_P$ and R_i is p -periodic for $i \geq t_R$, then finite stabilization implies*

$$R(L) \leq t_P + t_R + 1.$$

Proof. Use the same stable state sets with separate valid representatives. The pumping proof works with $U \geq \max(1, t_P)$ and $V \geq \max(1, t_R)$. If no obstruction exists, keep the central interval $[t_P, n - t_R)$ as one fragment and use singleton boundary pieces. Shorter words require at most $t_P + t_R$ singletons. $\square$

19 Polynomial-space decision and finite obstruction bounds

19.1 A finite graph, represented without expansion

Fix live r and a starting phase a . The subset evolution (41) is a directed graph on configurations

$$(j, Z), \quad j \in \mathbb{Z}/p\mathbb{Z}, \quad Z \subseteq Q.$$

From (j, Z) a letter $c \in C_{r,j}$ gives

$$(j, Z) \longrightarrow (j + 1, \Delta_c(Z) \cap S_{r,j+1}). \quad (45)$$

The initial configuration is $(a, S_{r,a})$, and an obstruction is exactly a path to an empty-subset configuration.

Lemma 19.1 (A short finite obstruction). *If an interior obstruction exists at (r, a) , one exists of length at most $p2^s$. It may be padded to a multiple-of- p block of length at most $p2^s$.*

Proof. A shortest path to an empty subset repeats no nonterminal configuration: a repeated phase and subset could be deleted without changing the remainder of the path, including its phase-dependent alphabet constraints. There are at most $p2^s$ configurations. Rounding the resulting positive length up to a multiple of p does not exceed $p2^s$, since the latter is itself a multiple of p . $\square$

This bound controls an *interior obstruction*, not a rejected word of the original language and not the length of a full crossover witness. The exterior words in Theorem 18.4 are still needed.

19.2 The PSPACE upper bound

Theorem 19.2 (Polynomial-space upper bound). *For an explicitly given NFA over an explicitly given finite alphabet, finite constrained self-crossover stabilization, under either clock, belongs to PSPACE.*

Proof. Choose the canonical $t = T_s$ and $p = p_s$. Store p in binary. It has $O(s \log(s + 2))$ bits. For a residue a , a suitable representative is

$$i_a = t + ((a - t) \bmod p).$$

The sets P_a^∞ and R_a^∞ are computed from E^{i_a} by Boolean matrix powering. The exponent has polynomially many bits and the matrices have s^2 entries, so this requires polynomial space (and polynomial time in the bit length of the exponent).

To recognize *nonstabilization*, nondeterministically select residues $r, a < p$. Check that r is live using (38). Initialize $j = a$ and $Z = S_{r,a}$. Repeatedly choose a letter c , verify $c \in C_{r,j}$, and update (j, Z) by (45). Accept if Z becomes empty. Terminate and reject that branch after $p2^s$ steps. The counter, phase, residue, subset, transition data, and matrix-powering workspace all have polynomial size in the input.

Lemma 19.1 and Theorem 18.3 show that this nondeterministic machine accepts exactly the nonstabilizing inputs. Thus nonstabilization is in NPSPACE. Savitch's theorem [10] gives $\text{NPSPACE} = \text{PSPACE}$, and deterministic polynomial space is closed under complement. Stabilization is consequently in PSPACE too. Corollary 15.3 identifies the two clocks. $\square$

For completeness, the resource conversion may also be seen directly. Reachability in an exponentially large graph with polynomial-bit vertices can be decided by recursively guessing a midpoint and splitting a path length in half. A deterministic search enumerates the midpoint encodings. The recursion depth is logarithmic in the number of configurations, hence polynomial, and each level stores polynomially many bits. This is the configuration-graph form of the cited theorem, not an assumption that nondeterministic time and deterministic time are interchangeable.

19.3 Quadratic rank and a finite generation cutoff

Corollary 19.3 (Uniform finite horizon). *Let $K_s = \lceil \log_2 B_s \rceil$. For an s -state NFA language, the following are equivalent:*

$$\text{finite parallel stabilization,} \quad L^{[K_s]} = \text{Rec}(L), \quad L^{[K_s]} = L^{[K_s+1]}.$$

For the frozen-source sequence the corresponding cutoff is $B_s - 1$.

Proof. Lemma 17.1 and Theorem 18.3 give $R(L) \leq B_s$ in the stabilizing case. Use (29). Conversely, any adjacent equality stabilizes the sequence by monotonicity, independently of the chosen cutoff. $\square$

This is a *uniform cutoff*, not a proposal to construct the enormous intermediate automata for all generations. The polynomial-space algorithm tests the periodic interior directly. It is the structural criterion, not an unrestricted inclusion test on exponentially expanded automata, that keeps the space bound polynomial.

Corollary 19.4 (An explicit full-witness length bound). *If an s -state NFA does not stabilize, then, for every integer $q \geq 1$, there is a hull word of rank at least $q + 1$ and length at most*

$$2T_s + 2p_s - 2 + qp_s 2^s.$$

Proof. Use Lemma 19.1 for a block of length at most $p_s 2^s$. In the pumping construction, choose U and V in $[T_s, T_s + p_s - 1]$ with their required residues. The word $ub^q z$ has the asserted length and rank. $\square$

In particular, rank just beyond the polynomial bound has a witness of singly exponential length, with a succinct periodic description. The word need not itself have a polynomial-length explicit representation.

20 Binary PSPACE-hardness

20.1 A factorial-language amplification lemma

A language is *factorial* if every contiguous factor of every word in the language also belongs to the language. The language of an NFA in which every state is both initial and final is factorial: restrict any accepting path to the interval carrying the desired factor.

Lemma 20.1 (Missing factors cannot be repaired in bounded rank). *Let $L \subseteq \{0, 1\}^*$ be factorial and contain $0^* \cup 1^*$. Then $\text{Rec}(L) = \{0, 1\}^*$. Moreover, finite crossover stabilization is equivalent to $L = \{0, 1\}^*$.*

Proof. For every n and every coordinate, 0^n and 1^n supply both possible letters, and $\varepsilon \in L$. Hence the hull is universal.

If L is not universal, choose $v \notin L$. Because L contains both unary languages, v has length at least two. No word of L can contain v as a factor. For the target v^k , each of its k disjoint displayed copies of v must

therefore contain an internal partition cut: a seed fragment containing an entire copy would force $v \in L$. Consequently $\rho_L(v^k) \geq k + 1$. The target is in the universal hull, so rank is unbounded. Corollary 15.3 rules out stabilization. The universal language is already fixed, proving the converse. $\square$

20.2 The reduction over two letters

We use the published result of Kao, Rampersad, and Shallit [9, Lemma 6]: nonuniversality of binary NFAs with every state both initial and final is PSPACE-complete. By complement closure, universality for that class is PSPACE-complete as well. That hardness theorem is an imported complexity result, not reproved by finite testing here.

Theorem 20.2 (Binary hardness with a universal hull). *Finite crossover stabilization is PSPACE-hard on binary NFAs. Hardness persists when all states are initial and final, the language is factorial, and its coordinate hull is universal. Both iteration clocks have this property.*

Proof. Let $\mathcal{A}$ be a binary all-initial/all-final NFA recognizing K . Form the disjoint union with one state having only a 0-loop and one state having only a 1-loop. Declare both added states initial and final. The new automaton recognizes

$$L = K \cup 0^* \cup 1^*. \quad (46)$$

It adds exactly two states. Its language is factorial, and its hull is universal by Lemma 20.1.

If K is universal, so is L . Conversely, if $x \notin K$, then $x01 \notin K$, since otherwise factoriality would put its factor x in K . The word $x01$ contains both letters, so it is in neither 0^* nor 1^* . Thus L is nonuniversal. We have proved

$$K = \{0, 1\}^* \iff L = \{0, 1\}^* \iff L \text{ stabilizes.}$$

The last equivalence is Lemma 20.1; the reduction is linear in the automaton size. This proves the result for both clocks. $\square$

Corollary 20.3 (Exact complexity and least alphabet). *Finite constrained self-crossover stabilization is PSPACE-complete for NFA input over a fixed binary alphabet. Over a unary alphabet every language, regular or not, is already fixed by constrained self-crossover. Thus two letters is the least alphabet size permitting the hardness result.*

Proof. Combine Theorems 19.2 and 20.2. Over a unary alphabet there is at most one word of each length, so two same-length parents, if present, are identical. No crossover adds a word. $\square$

The reduction also handles the standard single-initial-state NFA encoding. Add one fresh initial state, make it accepting because $\varepsilon \in L$, and on each letter give it the union of the transitions out of the old initial states. Leave every old transition in place and make only the new state initial. This ε -free construction adds one state and preserves the language. The all-initial/all-final promise itself uses the multiple-start convention, but the ordinary PSPACE-completeness statement does not depend on that convention.

20.3 A second reduction connecting to the repository amplifier

The preceding proof is the sharper alphabet result. The following four-letter version explains directly how the old report's delimiter amplification also yields a hardness proof from ordinary binary NFA universality. Its block mechanism is credited to that earlier line of argument; it is not a second claimed discovery of the amplifier. [write, batch 66] The amplifier is Part I's Theorem 5.1, with the context-free languages G_K and B_K

of (6)–(7); the regular languages $\widehat{G}(K)$, $\widehat{B}(K)$ below (the manuscript's $G(K)$, $B(K)$) are different: $\widehat{G}(K)$ adds to K every word containing @, with no 00 guard, and $\widehat{B}(K)$ leaves its first and last blocks unrestricted.

Let $K \subseteq \{0, 1\}^*$ be given by an NFA, let $\Gamma = \{0, 1, @, \#\}$, and put

$$\widehat{G}(K) = K \cup \Gamma^* @ \Gamma^*, \quad \widehat{B}(K) = \Gamma^* \cup \Gamma^* \# (\widehat{G}(K) \#)^* \Gamma^*.$$

An internal complete #-delimited block must lie in $\widehat{G}(K)$; the first and last incomplete blocks are unrestricted. Standard union, concatenation, and star constructions give an NFA of size linear in the input, and ε -elimination preserves the state set.

Proposition 20.4 (Exact delimiter ranks). *The hull of $\widehat{B}(K)$ is $\{0, 1, @, \#\}^*$. If K is universal, so is $\widehat{B}(K)$. If $x \notin K$, then for every $q \geq 1$,*

$$\rho_{\widehat{B}(K)}(\#(x\#)^q) = q + 1.$$

Proof. A word with at most one delimiter belongs to $\widehat{B}(K)$. For any prescribed letter at any prescribed position, fill every other position with @ to obtain such a seed. This proves the hull assertion, including the empty word.

If K is universal, $\widehat{G}(K) = \Gamma^*$ and the displayed expression for $\widehat{B}(K)$ permits every word. If $x \notin K$, each displayed $\#x\#$ in $\#(x\#)^q$ is forbidden inside a seed. Consecutive occurrences may share a delimiter letter, but their *internal boundary slots* are disjoint. Hence every fragment partition needs at least q cuts. For the opposite bound, partition as q copies of $\#x$ and one final $\#$. Each piece contains at most one delimiter and extends to a length-matched seed by filling outside positions with @. These are $q + 1$ admissible fragments. If $x = \varepsilon$, the same argument partitions $\#^{q+1}$ into single letters and remains valid. $\square$

This establishes the four-letter reduction without the all-initial/all-final hardness theorem. It is included for comparison and proof portability; the binary theorem above remains the stronger final result.

21 Deterministic automata and safe cores

Assume now that each $\Delta_c(q)$ has at most one state. The original DFA may be complete, but after restriction to viable phase sets some transitions become undefined. Fix a live residue r and regard

$$(a, q), \quad q \in S_{r,a},$$

as vertices of a partial deterministic phase graph. Its allowed input letters at phase a are $C_{r,a}$.

Definition 21.1 (Safe core). A family $(U_a)_{a \bmod p}$ with $U_a \subseteq S_{r,a}$ is safe if

$$q \in U_a, c \in C_{r,a} \implies \delta(q, c) \text{ is defined and belongs to } U_{a+1}.$$

The safe core $(\mathcal{U}_{r,a})$ is the greatest such family.

It is obtained by starting with all viable vertices and repeatedly deleting any vertex with an allowed letter leading outside the surviving set. The process terminates after at most ps vertex deletions. An efficient implementation stores reverse edges and a queue of newly unsafe vertices.

Theorem 21.2 (Safe-core characterization). *For a deterministic automaton and a fixed live residue r , all phase-admissible words are liftable at every starting phase if and only if the safe core is nonempty. A nonempty safe core meets every phase.*

Proof. If a safe vertex exists, any allowed letter takes it to a safe vertex in the next phase. Following one allowed letter at a time, possible because all $C_{r,a}$ are nonempty, shows that safe vertices exist in every phase. Starting with any safe vertex at a desired phase then reads every phase-admissible word without leaving the viable graph. This proves sufficiency.

Suppose the safe core is empty. A vertex is safe exactly when no phase-admissible continuation can cause its deterministic run to leave the viable graph. Equivalently, every vertex now has some killing continuation. One can see this directly from deletion times: a vertex removed first has an immediately invalid allowed transition; a later removed vertex has a transition to an earlier removed vertex.

Start with all viable states at some phase. Select one survivor and append a killing word for that survivor. The statewise determinism implies that every current state has at most one resulting image. Since the selected state has no image, the number of surviving images drops by at least one. Repeat with a surviving state at the new phase. After at most s rounds there are no survivors. The concatenated word is phase-admissible but not liftable. Thus universality fails. $\square$

Corollary 21.3 (A shorter deterministic obstruction). *If a live residue of a deterministic s -state automaton has an interior obstruction, it has one of length at most ps^2 .*

Proof. When the safe core is empty, a shortest route from any given vertex to an invalid allowed transition traverses no vertex twice before failure. It has length at most the number of phase vertices, at most ps . Use such a route in each of at most s killing rounds in the proof of Theorem 21.2. $\square$

For a fixed residue with its phase sets explicitly listed, the safe core can be computed in $O(ps|\Sigma|)$ time by backward deletion. Treating all p residues separately gives $O(p^2s|\Sigma|)$ core work, in addition to constructing the phase sets. These are bounds in the *expanded periodic presentation*. Since p can be exponentially large relative to the original state count, this is not a polynomial-time algorithm for arbitrary DFA input. The PSPACE upper bound still follows from Theorem 19.2; no matching DFA lower bound is claimed.

Example 21.4 (Why the deterministic argument cannot be used on NFAs). Take two states x, y , both initial and final, with transitions

$$\Delta_0(x) = \{x, y\}, \quad \Delta_1(x) = \emptyset, \quad \Delta_0(y) = \emptyset, \quad \Delta_1(y) = \{x, y\}.$$

The language is all binary words: after any letter the full two-state subset is available again. Yet neither individual state supports both possible first letters. A vertexwise safety deletion removes both states, even though the subset evolution never empties. The deterministic cardinality-drop argument fails because a single NFA state can branch into several surviving states. This is a concrete reason to retain subset universality in the NFA algorithm.

22 Quantitative consequences and worked examples

22.1 The generation bound has the right logarithmic order

Theorem 22.1 (Logarithmic generation lower bound). *For each $m \geq 1$, let*

$$L_m = \{0^{i-1}10^{m-i} : 1 \leq i \leq m\}.$$

This language has a complete DFA with at most $2m + 3$ states, stabilizes finitely, and satisfies $R(L_m) = m$. Its least parallel stabilization index is $\lceil \log_2 m \rceil$, and its least frozen-source index is $m - 1$.

Proof. A DFA stores the current length $0, \dots, m$, whether a 1 has already occurred, and one rejecting sink. Moving past length m or seeing a second 1 enters the sink. This uses $2(m+1) + 1$ states; unreachable states may be removed but are irrelevant to the bound.

All nonempty hull words have length m , so rank is at most m . Every coordinate of 1^m is supplied by one of the seed words. No admissible fragment of 1^m can have length at least two, because a seed has only one 1. Hence $\rho_{L_m}(1^m) = m$. The case $m = 1$ is included, with rank one. Apply Corollary 15.3. $\square$

Thus the largest possible finite parallel stabilization index for s -state automata has order $\Theta(\log s)$, even over two letters. This does *not* prove that the quadratic rank bound is optimal. The presently proved lower and upper orders for maximal finite rank are linear and quadratic, respectively.

22.2 A regular language with a repairable boundary

Example 22.2 (One generation without universality). Over $\Sigma = \{0, 1\}$, let

$$L = \{\varepsilon\} \cup 0\Sigma^* \cup \Sigma^*0.$$

Then $\text{Rec}(L)_1 = \{0\}$ and $\text{Rec}(L)_n = \Sigma^n$ for $n \geq 2$. For any $w \in \Sigma^n$ with $n \geq 2$, choose an interior cut. The prefix is supplied by a seed ending in 0 and the suffix by a seed starting in 0. Hence rank is at most two and $T(L) = \text{Rec}(L)$. The word 11 is not a seed and has rank exactly two, so the least parallel and frozen-source indices both equal one.

This example separates finite stabilization from universality for general regular languages. The hardness equivalence with universality applies to the special factorial class, not to all NFAs.

22.3 Bounded and linear residues can coexist

Example 22.3 (Even lengths are fixed; odd lengths have linear rank). Let

$$L = \{w \in \{0, 1\}^* : |w| \text{ is even}\} \cup 0^* \cup 1^*.$$

Its hull is universal. At even lengths every word is a seed, so the maximum rank is one. At odd lengths the seeds are exactly $0^n, 1^n$; an admissible nonempty fragment must be constant. Therefore the rank of an odd-length word is its number of maximal constant runs, and

$$M_L(n) = \begin{cases} 1, & n \text{ even,} \\ n, & n \text{ odd.} \end{cases}$$

Here $M_L(0)$ may also be taken as one. A four-state NFA recognizes the language by taking a disjoint union of a parity automaton and two unary loops. The periodic criterion sees a universal interior at the even total residue and an obstruction such as 01 at the odd residue.

The example shows why a statement of linear growth at *all* large lengths would be false. It also explains why the total-length residue r and the boundary phase a are separate parameters in the construction.

22.4 The strongest possible generation growth in a bad residue

On a live residue with an obstruction, let

$$D_L(n) = \max_{w \in \text{Rec}(L)_n} \min\{k : w \in L^{[k]}\}.$$

By (29) and Corollary 18.5,

$$D_L(n) = \log_2 n + O(1) \quad (n \rightarrow \infty \text{ in that residue}). \quad (47)$$

The maximum frozen-source depth is $M_L(n) - 1 = \Theta(n)$. The constants can be read from the pumped block length and boundary lengths. In a good residue the two depths remain bounded. Thus the rank dichotomy yields a complete bounded-versus-maximal-order dichotomy for the two natural iteration clocks, residue by residue.

23 Executable audits and their limits

The package contains a standard-library Python implementation in `code/verify.py`. It is an exact finite reference checker, not an optimized prover and not a formalization. Its NFA transitions and subsets are represented by integer bit masks. The recorded run used Python 3.13.5 and returned PASS; details and limitations are stored in `data/verification.json` and the console transcript `data/verification.txt`.

[write, batch 66] *Shipped names*. In this report the delivered files are shipped, byte-identical, as follows; the manuscript and its PDF are printed or replaced by this report, and the delivery README by the report's README. The unprefixed `data/verification.json` of this report is Part I's receipt (Section 10).

Shipped path	Delivered as
<code>code/02-ospace-stabilization-verify.py</code>	<code>code/verify.py</code>
<code>code/02-ospace-stabilization-example.py</code>	<code>code/example.py</code>
<code>code/02-ospace-stabilization-build.sh</code>	<code>build.sh</code>
<code>data/02-ospace-stabilization-verification.json</code>	<code>data/verification.json</code>
<code>data/02-ospace-stabilization-verification.txt</code>	<code>data/verification.txt</code>
<code>data/02-ospace-stabilization-build-receipt.json</code>	<code>data/build-receipt.json</code>
<code>02-ospace-stabilization-proof-audit.md</code>	<code>notes/proof-audit.md</code>
<code>02-ospace-stabilization-source-ledger.md</code>	<code>notes/source-ledger.md</code>

23.1 Two independent rank calculations

The path-based algorithm implements (36). The independent algorithm enumerates all accepted seed words at a given length, collects their actual aligned substrings, and runs a partition dynamic program on those substring sets. In particular, it does not use P_i , R_j , or viable layers to decide interval admissibility. All tested finite ranks, including infinity and the empty-word case, agreed.

The literal crossover check enumerates all parent pairs and cuts. It then compares its actual generated sets with the rank thresholds 2^k . These checks are logically separate from the periodic criterion.

23.2 Recorded coverage

Exact audit	Recorded count
All labelled two-state binary NFAs, all initial/final subsets	4,096
Independent rank comparisons, all binary words of lengths 0–5	258,048
Stable-rank bound checks within those comparisons	69,488
Complete three-state binary DFAs, fixed initial state	5,832
Safe-core versus subset-criterion comparisons on those DFAs	5,832
Stable three-state DFA rank checks at length 6	369,152
Additional partial-deterministic core comparisons	1,296
Pumped-witness checks across the stated families	1,641
Binary all-initial/all-final hardness-transform cases	256
All binary finite seed sets on slices of lengths 0–3	278
Literal parallel-generation comparisons on those seeds	1,112
One-1 lower-family cases, $1 \leq m \leq 24$	24
Unary graph/subset cases on 1–3 vertices	4,164
Unary periodicity equalities within those cases	4,382
Exact delimiter ranks, including the omitted empty word	504
Delimiter coordinate-supply witnesses	9,072
Parity-example rank checks, lengths 1–10	2,046
Boundary-repair example rank checks, lengths 0–7	255

The two-state NFA family has 3,720 stabilizing and 376 nonstabilizing members according to the implemented exact periodic criterion. The three-state complete DFA family has 5,768 stabilizing and 64 nonstabilizing members. These are labelled-automaton counts, not counts of distinct languages or isomorphism classes.

An additional 200 seeded random NFAs with three through six states were tested, with 4,000 sampled rank queries. The seed is 20260930. Those samples are explicitly not exhaustive. The worked examples are also checked directly: the delimiter family uses every omitted binary word of length at most five and one through eight repeated blocks; the parity example uses every binary word of lengths one through ten; and the boundary-repair example uses every binary word of length at most seven. The unary audit enumerates every directed graph on one through three labelled vertices and every chosen initial subset, taking the same subset as final; it tests both forward and backward sequences through their exact eventual cycles.

The categories overlap: for example, some of the rank-bound checks are also independent rank comparisons. Their counts must not be added and reported as a number of statistically independent experiments.

23.3 What the program actually certifies

The periodic solver explicitly enumerates the least eventual cycle of (P_i, R_i) and then searches phase/subset graphs by breadth first search. On the tested small automata this is convenient, but it stores the expanded graph. It is *not* an implementation with the polynomial-space resource guarantee of Section 19. The latter is an on-the-fly theoretical algorithm with binary counters and matrix powers.

For each detected small obstruction, the program constructs the exterior accepted word and the padded repeated block from the proof, then verifies finite ranks of the resulting full words. On stabilizing cases it checks the stronger observed-transient bound $2t + 1$, where t is obtained from the least pair cycle, rather than only the much looser universal B_s .

No finite test proves that every rank above every state count is bounded, or that every obstruction pumps. Those are theorems proved in Sections 18–19. No solver is used to infer PSPACE-hardness, no verification

output is represented as a Lean kernel certificate, and no literature theorem is declared proved because small instances pass.

23.4 Reproduction

From the package root, run

```
python3 code/verify.py --output data/local-verification.json
latexmk -pdf -interaction=nonstopmode -halt-on-error article.tex
```

The explicit output option preserves the delivered receipt. The default output path is `data/verification.json`; running without the option replaces that file in the local copy. There are no network calls, third-party Python dependencies, or external theorem provers in the checker. The included build script performs only the local LaTeX build.

[write, batch 66] *In this report, do not run the shipped files in place.* The verifier’s default output path is `../data/verification.json` relative to its own directory, which here is Part I’s receipt; the example program imports the verifier under its delivered module name `verify`, which the prefixed file does not have; and the build script runs `latexmk` on an `article.tex` in its own directory, `code/`, where there is none. Copy the two programs into a scratch directory under their delivered names `code/verify.py` and `code/example.py`, and run them there, with an explicit `--output`; the report’s README gives the commands and the result of such a rerun. The article is rebuilt from the report directory as in Section 10. The delivered example program’s comment calls the parity language “article Example 9.3”; in this report it is Example 22.3.

24 Further research questions

The questions below are specific continuation targets. They are not all asserted to be new or globally open. The first is the unsolved portion of the inspected report’s regular-input complexity question; the others are natural sharpenings exposed by the present proofs. [write, batch 66] The inspected report’s question is Part I’s Question 11.2. Question 24.6 below is Part I’s Question 11.1 and is printed as a pointer; Question 24.7 overlaps Part I’s Question 11.7, asking it for the dichotomy proved here.

Question 24.1 (Exact complexity for DFA input). What is the exact complexity of deciding finite stabilization for complete DFAs over a fixed binary alphabet?

The present PSPACE upper bound is stronger than the earlier EXPSPACE bound, but the NFA hardness proof does not transfer to a DFA without an exponential determinization. The safe-core characterization removes subset branching once the phases are explicit. The remaining issue is whether the exponentially long unary phase structure can be manipulated more economically, or can encode a matching hard problem. An algorithm polynomial in p does not answer this question.

Question 24.2 (The optimal finite rank bound). Let $F(s)$ be the largest finite $R(L)$ among binary languages recognized by NFAs with at most s states. Is $F(s)$ linear, quadratic, or of an intermediate order?

There are only finitely many labelled NFAs of a fixed state bound, so this maximum is well defined. For $s \geq 5$, the results here give

$$\left\lfloor \frac{s-3}{2} \right\rfloor \leq F(s) \leq (2s+5)^2.$$

A quadratic lower bound would have to exploit long unary transients while making many boundary positions require genuinely separate aligned fragments. A long transient alone is not such a proof. Conversely, a linear upper bound would need a way to compress boundary fragments that is absent from the singleton argument.

Question 24.3 (Exact residue-wise rank slope). On a bad residue, does $M_L(n)/n$ always have a limit, perhaps after refining the period? Must a resulting slope be rational, and can it be computed effectively?

The pumping theorem gives positive lower and upper linear bounds, not a limit or a formula. The quantity is a maximum over target words of a minimum partition cost. Replacing it by a shortest-path mean without justifying this order of quantifiers would be invalid. The periodic systems provide finite objects on which this problem can be studied.

Question 24.4 (Optimal obstruction lengths). How sharp are the interior bounds $p2^s$ for NFAs and ps^2 for DFAs? Can the multiplicative p be replaced by a smaller invariant of the unary reachability graph?

The NFA bound is a configuration-count bound; the DFA bound concatenates at most s single-state killing words. Neither construction is claimed optimal. The definition of an interior obstruction must be retained: shortest nonmembers of L can instead reflect only a repairable boundary.

Question 24.5 (Compressed periods and effective certificates). Can the live residues and their safe or obstructed status be represented by polynomial-size arithmetic data, rather than by all residues modulo an explicit least common multiple?

The common period used here is convenient but often highly redundant. Strongly connected component periods and arithmetic-progressions representations are candidate data structures. A positive certificate would need to establish universality in every live phase system, not merely exhibit a few successful runs. A negative certificate can already be represented by one forbidden interior word and periodic exterior padding, but that word may itself be exponentially long.

Question 24.6 (Context-free finite stabilization). [write, batch 66] This is Part I's Question 11.1, printed there: what is the exact arithmetical complexity of finite stabilization for context-free grammar input under the same aligned operation? The manuscript's comment follows.

The inspected report proves undecidability and gives an upper placement in Σ_2^0 (Theorem 5.2 and Proposition 5.4), but does not claim completeness at that level. Nothing in the regular-input analysis resolves this question: unary periodicity of the two exact-length state processes uses finiteness of the automaton state set. A reduction showing unbounded rank from the failure of a Σ_2^0 property would require a different argument.

Question 24.7 (Restricted cuts and other recombination rules). Which parts of the dichotomy survive when only a regular set of cut positions is permitted, or when a single operation exchanges several alternating intervals between two parents?

For unrestricted aligned one-point self-crossover, every singleton is available in the eventual hull. Restricted cuts may destroy that product structure. In a multi-cut operation, several pieces still come from only two parents; they are not automatically independent fragments. Thus one cannot simply replace the factor two in the rank law by the number of cuts plus one without proving an appropriate realization theorem.

Question 24.8 (Proof-assistant implementation). Can the periodic-interior equivalence, quantitative cutoff, and binary hardness reduction be formalized with an auditable separation between combinatorial proofs and imported complexity theorems?

The development proposed in the next section gives small independent milestones. Formalizing the full PSPACE lower bound includes the published all-initial/all-final universality reduction, not merely the two-state union construction used here.

25 A formalization and implementation roadmap

No Lean or Rocq files are claimed as verified in this delivery. A practical formal development can nevertheless be organized without importing the entire earlier report as an unexamined axiom.

Finite words and partitions. Define an aligned interval by two bounded natural numbers and a proof of their strict order. Define fragment admissibility by a length-matched witness word. The two rank thresholds can first be stated as existence of a partition with at most d pieces, avoiding extended-natural minima until the basic equivalences have been proved. The empty-word convention should be isolated at the start.

Runs and exact-length reachability. Represent an NFA by finite relations on a finite state type. Prove Lemma 16.1 by concatenating and restricting finite runs. This is the bridge at which absolute coordinates and total lengths must be retained. The periodic theorem can initially take t, p and the periodicity equations as explicit hypotheses.

The structural core. Prove the central-fragment sufficiency result first. Then prove the pumping lemma in three separate pieces: phase-aligned block extension, preservation of each exterior coordinate under a multiple-of-period length change, and the injection from obstruction copies to partition cuts. The last piece uses the strict internal boundary sets; it does not need a vague claim that copies are “independent”.

Quantitative periodicity. The elementary 4^s eventual-cycle argument is a short finite-state lemma and can yield a fully formal qualitative result early. The quadratic bound should be added only after a verified unary arithmetic-progression theorem is available, or else recorded as an explicit conditional corollary. The two added normalization states and the length shift by two must not disappear from the statement.

Decision complexity. A reference explicit algorithm is much easier to validate than its space bound. Separate functional correctness of the phase/subset search from the representation-size proof for binary counters, Boolean matrix powering, and nondeterministic configuration graphs. Likewise, prove the binary language equivalence in (46) independently of importing PSPACE-completeness of the source problem.

Deterministic safety. The greatest safe family is a finite greatest fixed point. Its elementary delete-to-failure proof should be particularly tractable. The proof that killing one deterministic start decreases the surviving-subset cardinality should be a named lemma, making the exact point where nondeterminism breaks the argument explicit.

These milestones produce useful formal theorems before a full complexity library is available. They also keep the distinction between an existing repository formalization, an ordinary mathematical proof, and a future formalization plan visible.

26 Conclusion

The special two-sided structure identified in the repository question really does improve on generic limitedness. Exact-length reachability becomes periodic from both ends. In a good interior, all allowed target letters can be joined by one seed run, leaving only a bounded exterior cost. In a bad interior, one finite failure repeats and forces a fresh partition cut each time. This yields both the bounded/linear dichotomy and a finite search problem on phase/subset configurations.

As a consequence, finite stabilization for NFA input is PSPACE-complete over the smallest nontrivial alphabet, while every stabilizing s -state instance has rank at most $(2s + 5)^2$. The logarithmic order of the parallel generation bound is sharp. The exact DFA complexity, optimal rank order, and precise slopes of bad residues remain substantive next problems, rather than being hidden inside an unspecified generic limitedness bound.

27 Proof and edge-case audit

[write, batch 66] This section and the next were the manuscript's Appendices A and B.

Potential failure	How the proof handles it
Empty language or empty slice	There is no live contribution; hull slices remain empty. $R(L)$ includes 1 only to normalize the empty-language stabilization index.
The empty word	Its rank is 1 exactly when it is a seed. The length-zero hull is explicitly L_0 , not an unconditional empty product.
Transient $t = 0$	The central-fragment argument then gives rank 1 in the stabilizing case. Pumping uses positive exterior lengths to avoid vacuous baselines.
An obstruction of length one	Impossible by the definition of $C_{r,a}$; every permitted letter has a viable transition. Thus every obstruction has an internal cut slot.
A forbidden word only at a particular phase	The padded block has length divisible by p , so every copy starts at the same phase.
The total-length residue changes	It does not: all inserted block and remainder lengths are multiples of p .
Exterior coordinates cease to be available	The unchanged side of the single-letter path test is exact; the changed side lies beyond its transient and changes by a multiple of p .
A fragment's parent run is not globally fixed	Each admissible fragment has its own seed witness. Restricting that witness is sufficient to contradict an interior obstruction.
Linear growth only on a sparse subsequence	The remainder-padding step supplies every sufficiently large length of the chosen residue, not merely lengths of the form $U + kh + V$.
Confusing a huge period with a huge rank bound	The period is used to build and recognize the interior. Rank cost comes from the two transients, which are quadratic independently of the period.
Replacing NFA universality by statewise safety	The two-state example in Section 21 disproves that replacement. Safety is used only for deterministic transitions.
Binary hardness filled by unary loops	A missing word x in a factorial language forces the missing mixed word $x01$, so adding $0^* \cup 1^*$ cannot create false universal cases.
Sharing a delimiter in the second reduction	The forbidden occurrences share a letter, not an internal boundary slot. Their required cuts remain distinct, including $x = \varepsilon$.

Potential failure	How the proof handles it
Claiming a polynomial-time DFA algorithm	The safe-core time is polynomial only in the expanded period. The article does not identify it with polynomial time in the original automaton.
Finite checks presented as a proof	They are bounded audits. Universal claims rest on the written arguments and the explicitly cited imported theorems.

28 Source ledger and novelty boundary

Repository source. The pinned source is the crossover report’s `article.tex` and `README.md` at the revision displayed in Section 14. The EXPSPACE upper bound is its corollary headed *Decidable regular-input stabilization*; the continuation target is its Section 11 question headed *Sharp regular-input complexity*. The rank formulas and frozen-source calculus are credited background. The report itself states that it is unrefereed and not formalized. [write, batch 66] These are Part I’s Corollary 7.4 and Question 11.2; the source ledger is shipped as `02-pspace-stabilization-source-ledger.md`.

Primary external inputs. The aligned operation was checked against Hughes’s version 1 preprint. The corrected unary constants were checked in To’s Section 3, including the page images of the correction. The binary all-initial/all-final universality hardness was checked against Lemma 6 of the version 2 Kao–Rampersad–Shallit preprint. Savitch’s space simulation is the standard complexity conversion used for the upper bound; the argument’s configuration-graph application is explained in the text.

What is imported, and what is proved here. The unary arithmetic-progression theorem and the PSPACE-hardness of the source universality problem are imported published results. The periodic-interior criterion, obstruction pumping theorem, rank and depth consequences, crossover complexity reduction, and deterministic safe-core application are proved in this article. The earlier rank calculus is reproved for completeness, not claimed as a new contribution.

Search boundary. Searches on 30 September 2026 included combinations of constrained crossover, finite stabilization, regular languages, recombination, PSPACE, unary arithmetic progressions, and all-initial/all-final NFA universality. Broad recombination searches were noisy and did not amount to an exhaustive literature classification. The defensible novelty claim is a proved extension of the inspected repository, not an unconditional assertion of worldwide priority or a claim of peer-reviewed acceptance.

No hidden formal status. No assertion in this article was checked by Lean or Rocq. The Python program and its receipts are included so that the finite audits can be reproduced, but the ordinary proofs remain independently reviewable. No repository file was modified as part of this delivery.

[write, batch 66] The last sentence describes the delivery; the write phase added this Part and dated notes in Part I. Placement beside ProveIt’s Lean and Rocq developments confers no formal status on Part II, and none of its statements is formalized. No ProveIt Lean or Rocq file defines finite automata, crossover, aligned rank, unary arithmetic-progression normal forms or space complexity classes, so To’s theorem, Kao–Rampersad–Shallit’s Lemma 6 and Savitch’s theorem enter only as cited mathematics.

Part III. Decidable regularity and exact stabilization of crossover closures

[write, batch 66] *Part III was added on 30 September 2026 in batch 66, together with Part II, from a second manuscript written against Part I as printed above (it did not know Part II). For context-free seeds it decides regularity, inclusion and equality of the full crossover closures; for semilinear one-marker seeds it computes every generation, decides finite stabilization, and classifies the stabilizing ones into regular, linear non-regular and non-context-free closures; and it shows exact stabilization depth to be a clique number, coNP-complete to threshold for succinct ray input. It partly answers Part I’s Question 11.4. It shares with Part II only a reproof of Part I’s coordinate hull, printed here as a third route (Lemma 31.3). Section 29 records the provenance and fixes the notation of the addition.*

29 [write, batch 66] Provenance and notation of Part III

29.1 One manuscript, one addition

Part III is batch-66 manuscript 03, *Decidable Regularity and Exact Stabilization of Crossover Closures: arithmetic invariants, sparse-marker trichotomies, and immediate loss of context-freeness*, dated 30 September 2026 (23 US-letter pages as delivered). It is an AI-assisted research manuscript: its title page and PDF metadata name ChatGPT as the drafting assistant and Vladimir Reshetnikov as the person it was prepared for. It arrived as the archive `ProveIt_Crossover_Classification.zip` (inner directory of the same name) in ProveIt commit 9ad899cbe and was placed in commit 4fee1cd07. Like Part II it was written against ProveIt commit b8b0fa218 (b8b0fa2184a044d46ce9ed0f25f88d7bb60fa042), at which Part I’s `article.tex` had the blob eb4f427f printed above, so “the preceding report” of this Part is Part I. Its code, recorded results, ray examples and audit notes are shipped beside Part I’s files with the prefix `03-closure-classification-`; its manuscript, delivery README and PDF are not shipped and survive in the arrival commit, and its checksum list SHA256SUMS (21 files, all verified at placement) was retired by the repository’s policy. Section 45 and the report’s README list the shipped names.

Every result, proof, example, remark, table, limitation and question of the manuscript is printed here, in its order. Its title page, abstract and status box became this section; its Sections 1–14 are Sections 30–43; its two appendices became Sections 44 and 45. Its nine unlabelled research questions received labels. Text marked [write, batch 66] was added when the manuscript was written into this report: the notes relating it to Part I and Part II, among them the credit to Part I’s Lemma 8.1 after Lemma 31.4, which the manuscript does not give. No theorem was strengthened or weakened.

29.2 The manuscript’s summary and status

Abstract, as delivered (“the ProveIt report on this operation” is Part I). Constrained crossover exchanges suffixes of equal-length words at an aligned cut. The ProveIt report on this operation proves that the infinite self-crossover closure is a product of position supports. We use that invariant to decide regularity, inclusion, equality, and eventual equality of the full closures of arbitrary context-free seeds. Regularity is exactly finite row type of the associated Presburger support relations; a negative answer has an effective infinite family of distinguishable marked prefixes.

For the linear context-free seeds $L_R = 0^* \cup \{0^i 10^j : (i, j) \in R\}$, where R is semilinear, we determine

every generation and characterize finite stabilization. In the finitely stabilizing case, we obtain a complete regular/linear context-free/non-context-free trichotomy. Two co-occurring, distinct interior marker velocities cause non-context-freeness already after one crossover. For finite unions of arithmetic rays the exact maximum marker count is a clique number, giving coNP-completeness of stabilization by an input depth for binary-encoded ray descriptions, even when the seed and closure are regular. An explicit binary linear seed stabilizes after one step at a non-context-free language with at most four words of each length. Complete proofs, executable exact ray analysis, 12,867 finite checks, and further research questions are included.

Status and attribution, as delivered. Unrefereed research manuscript, not a Lean or Rocq development. General monadic decomposition and the slender-language background are classical ingredients, not claimed discoveries. The crossover applications and sharp sparse-marker results are proposed contributions relative to the inspected repository; global priority is not established.

29.3 Notation beside Part I and Part II

Part III uses Part I's operation $\otimes_c$, its parallel iterates $L^{[g]}$ and its frozen-source iterates $Y_g = L \otimes_c Y_{g-1}$. Positions are zero based. Seven symbols were renamed when the manuscript was written in; no normalization changed:

- the full closure $\mathcal{H}(L) = \bigcup_g L^{[g]}$ is printed as Part I's $\text{Rec}(L)$, which is the same set by Theorem 3.3 (and $\mathcal{H}_R$ as Rec_R);
- the crossover map $C(L) = L \otimes_c L$ is printed as Part I's $T(L)$;
- the gap-ray language $T_s(a, d)$ of (66) is printed as $\text{Gap}_s(a, d)$, and the finite union T of Lemma 36.1 as $\mathcal{T}$, because T is the crossover map and Part II's T_s is a constant;
- the row equivalence $E_a(i, r)$ of (55) is printed as $\text{Eq}_a(i, r)$, because Part I's E_a is the position support, here called P_a^L ;
- the family $L_k = L_{R_k}$ of (75) is printed as L_k^{fam} , because Part II's L_m is another family;
- the threshold $T = 2^d + 1$ in the proof of Theorem 39.2 is printed as θ .

In Part III	Meaning in Part III	Not to be confused with
$P_a^L(i, j)$	some seed has a with i letters before and j after (48)	the same relation as $(i, j) \in E_a$ in Part I (11); not Part I's coordinate set $P_{n,i}(L)$ or Part II's state set P_i
$A_L(n, i)$	letters allowed at zero-based position i	Part I's $P_{n,i+1}(L)$ (one-based); Part II's $A_{n,i}$, the same set
$\text{Eq}_a, U_{a,r}, V_{a,r}, C_a$	row equivalence, its classes and rows, least representatives (Theorem 33.1)	Part I's E_a, U_i, V_i ; Part II's $C_{r,a}$
$R \subseteq \mathbb{N}^2$	a marker mask	Part I's $R_L(n)$; Part II's $R(L)$ and R_j
$K_R(n), k_R(n), K(R)$	marker sites at length n , their number, its supremum	Part I's languages K, K_q ; Part II's cutoff K_s
$d_{\parallel}(R), d_{\text{frozen}}(R)$	stabilization depths of L_R	Part I's depth $d_L(w)$ of one word
$\text{Gap}_s(a, d), \nu(d)$	gap-ray language with s ones, number of growing gaps	Part II's constant T_s
Γ_R, ω	ray compatibility graph, clique number	the alphabet $\Gamma = \{0, 1, @\}$ of Parts I and II
E, F	exceptional lengths, finite points of R	Part II's edge relation E and final states F
M_a	the marked language of Theorem 33.2	Part II's $M_L(n)$
$B_{k,g}, L_k^{\text{fam}}$	slice counts and the family of Theorem 40.2	Part I's B_K ; Part II's B_s and L_m (which is L_R for a finite R)
S_i	DFA states reachable by length- i words (proof of Theorem 33.1)	Part I's S_q ; Part II's $S_{n,i}$

30 Research target and relation to ProveIt

30.1 The question being pursued

A context-free language can have a non-context-free constrained-crossover closure, while the length generating function of that closure remains rational. Thus enumeration alone does not identify its language-theoretic complexity. The natural next question is whether the special product structure of the closure makes its regularity decidable, despite the undecidability of regularity for arbitrary context-free languages. A second question is quantitative: when the closure is reached in finitely many generations, what is the exact depth, and how does it interact with loss of context-freeness?

We answer the first question for every context-free seed. We answer the second, including a sharp language-class classification, for all semilinear one-marker seeds with uniformly bounded position support. These are substantial families of linear context-free languages, and they include regular seeds with succinctly specified arithmetic periods.

The inspected ProveIt revision is

b8b0fa2184a044d46ce9ed0f25f88d7bb60fa042.

The directly preceding report is *Aligned Fragments and Constrained Crossover*, dated 29 September 2026, in

SetTheory/Cardinals/docs/reports/
automata-and-formal-languages/constrained-crossover-closure/.

Its source and README were inspected at that revision. Our global-regularity question is a continuation motivated by that report; we do not attribute it to a numbered open question unless explicitly stated in the source.

[write, batch 66] The preceding report is Part I. Its Question 11.4 (*Grammar classes preserving context-freeness*) asks which context-free seeds have a context-free coordinate hull and suggests restrictions on the position supports E_a ; Theorem 37.1 and Corollary 37.2 answer it completely for finitely stabilizing semilinear one-marker seeds, by exactly such a restriction, and Theorem 33.1 decides the regular case for every context-free seed. The general question stays open (Question 42.1). The manuscript’s attribution policy above is kept: it does not claim to answer a numbered question.

30.2 What is inherited, and what is developed here

The preceding report proves a coordinate-hull description, aligned-fragment rank formulas, undecidability of one-step equality for context-free seeds, undecidability of eventual stabilization, regular-input algorithms, and rational commutative counting for closures. It also gives a linear context-free seed with a non-context-free closure and unbounded stabilization depth. We do not claim those results again. We reprove the short coordinate-hull fact because every later argument depends on its exact normalization, especially empty slices and the empty word. [write, batch 66] These are Part I’s Theorems 3.2–3.8, 4.2 and 5.2, Section 7, Theorem 8.3 and Section 9. Part II, written in parallel, adds a PSPACE-complete regular-input stabilization problem; Part III does not use it.

Hughes explicitly left the single-step constrained-crossover question open in Section 4 of his August 2026 preprint [1]. That is the question the preceding ProveIt report claims to answer. The present manuscript is not a second priority claim about the same question. Instead it develops full-closure decision procedures and a finitely stabilizing classification that are not among the preceding report’s stated results.

Input / problem	Result proved here	Location
Arbitrary CFG seeds	Inclusion, equality, eventual equality, and all Boolean-combination length sets are effective	§32
Regularity of the full closure	A decidable finite-row criterion, DFA synthesis, or an infinite obstruction certificate	§33
Semilinear one-marker seed	Exact generations and finite-stabilization criterion	§34–35
Sparse semilinear support	Complete regular / linear nonregular / non-context-free trichotomy	§37
Binary-encoded arithmetic rays	Exact clique formula; NP-complete support threshold and coNP-complete depth threshold	§38–39
One-step loss of context-freeness	Binary linear seed, depth one, at most four words per length	§40

30.3 Classical ingredients and the novelty boundary

Effective semilinearity of Parikh images and its equivalence with Presburger definability are standard [11, 3, 12]. The finite-row criterion used below is a concrete instance of monadic decomposition, not a new general solution of that problem. Relevant prior work includes monadic decomposability of regular relations [13], integer-linear-arithmetic decomposition [14], and the July 2026 results on the size of decompositions [15]. In particular, decidability does not imply that a small decomposition or small DFA exists.

The structural argument for sparse languages is closely related to the classical paired-loop description of slender context-free languages; Raz gives a constructive treatment [16]. We supply a self-contained finite-ray pumping argument, then derive the exact velocity criterion and its generation consequences for crossover. CLIQUE hardness itself is classical [17]; the contribution here is the realization of an arbitrary graph by distinct fixed marker positions with arithmetic length constraints, together with the exact depth identity.

The literature search was bounded. Equivalent sparse-recombination formulations elsewhere have not been exhaustively excluded. Mathematical correctness is supported by the proofs, not by a claim of first discovery.

31 Coordinate supports and the full closure

Throughout, $\mathbb{N} = \{0, 1, 2, \dots\}$ and Σ is a finite nonempty alphabet. Positions are zero based. A letter at position i in a word of length n has i letters before it and $j = n - 1 - i$ after it.

Definition 31.1. For $A, B \subseteq \Sigma^*$, constrained crossover is

$$A \otimes_c B = \{wz, yx : wx \in A, yz \in B, |w| = |y|, |x| = |z|\}.$$

Endpoint cuts are allowed. Put $T(L) = L \otimes_c L$, $L^{[0]} = L$, $L^{[g+1]} = T(L^{[g]})$, and $\text{Rec}(L) = \bigcup_{g \geq 0} L^{[g]}$. The *parallel stabilization depth* is the least g with $L^{[g]} = L^{[g+1]}$, or infinity if no such g exists.

Since endpoint cuts are allowed, $L \subseteq T(L)$. Monotonicity shows that one adjacent equality implies all later equalities and equality with the full closure. This is the parallel clock, not the frozen-source clock $Y_0 = L$, $Y_{g+1} = L \otimes_c Y_g$.

[write, batch 66] This definition restates Part I's Definition 2.1 and Lemma 2.2; the manuscript defines the full closure as the union of the generations, which is Part I's coordinate hull $\text{Rec}(L)$ by Theorem 3.3, hence the shared symbol. The relation P_a^L defined next is Part I's position support: $P_a^L(i, j)$ holds exactly when $(i, j) \in E_a$ (11).

Definition 31.2. The genuine position support of a language is

$$P_a^L(i, j) \iff \exists u, v \in \Sigma^* : |u| = i, |v| = j, uav \in L. \quad (48)$$

For $n > 0$ let $A_L(n, i) = \{a \in \Sigma : P_a^L(i, n - 1 - i)\}$. The empty-word bit is $e_L = 1$ precisely when $\varepsilon \in L$.

Lemma 31.3 (Coordinate hull; inherited fact, reproved). *For every language L and every $n > 0$,*

$$\text{Rec}(L) \cap \Sigma^n = \{a_0 \cdots a_{n-1} : a_i \in A_L(n, i) \text{ for all } i\}. \quad (49)$$

Moreover $P_a^{\text{Rec}(L)} = P_a^L$ for every a , and $\varepsilon \in \text{Rec}(L)$ if and only if $\varepsilon \in L$.

[write, batch 66] This is Part I's Theorem 3.3 (with Definition 2.4), restated in support form; Part II restates and reproves it as (31). The manuscript's proof, printed next, is a third route: Part I contracts ranks, Part II pairs fragments, and this proof builds the target one letter at a time from seed donors, as in Part I's frozen-source Theorem 3.8.

Third proof (manuscript 03). Crossover preserves length and copies each coordinate from a parent of that same length. Thus no generation creates a new letter support, proving one inclusion in (49). For the reverse inclusion, choose for each position i a seed word whose letter there is a_i . Starting with a donor for the first position, take a prefix already matching positions $0, \dots, i - 1$ and a suffix from a donor for position i , crossing at i . This retains the correct prefix and adds one correct letter. The other parent is a seed, hence belongs to every later generation; after at most $n - 1$ such steps the desired word is obtained. If a length slice of L is empty, all its genuine supports are empty and both sides of (49) are empty. The support equality follows from $L \subseteq \text{Rec}(L)$ and preservation. Only a length-zero seed can supply a length-zero offspring. $\square$

Lemma 31.4 (Effective arithmetic support). *Given a context-free grammar for L , one can effectively compute Presburger formulas defining every P_a^L and compute e_L .*

[write, batch 66] This is Part I’s Lemma 8.1, which the manuscript does not cite: the same effective semilinearity of the same supports (P_a^L is Part I’s E_a), there proved through an inverse homomorphism and a regular intersection, here reproved by a transducer argument. The added clause on e_L is the classical nullable-nonterminal test.

Second proof (manuscript 03). A finite-state transducer nondeterministically marks exactly one occurrence of a , maps each preceding letter to a fresh symbol x , erases the marked letter, and maps each following letter to y . Its image is the context-free language $\{x^i y^j : P_a^L(i, j)\}$, with an effectively constructible grammar. Its Parikh image is exactly the set in (48). Effective Parikh semilinearity and the effective equivalence between semilinear sets and Presburger formulas give the claim [11, 3, 12]. The nullable-nonterminal computation decides the empty-word bit. $\square$

Remark 31.5 (Why genuine supports matter). An arbitrary assignment of masks to coordinates can specify letters at some coordinates of a dead length while leaving another coordinate empty. Those assigned letters are not realized in any word. The regularity necessity argument below uses genuine supports, not such unnormalized masks. The one-marker construction of Section 34 avoids the issue by including 0^* , so every length is live.

32 Closure comparison and Boolean length theory

Theorem 32.1 (Inclusion, equality, and eventual comparison). *Given context-free grammars for L and K , it is decidable whether $\text{Rec}(L) \subseteq \text{Rec}(K)$ and whether $\text{Rec}(L) = \text{Rec}(K)$. The sets of lengths at which inclusion or equality fails are effectively ultimately periodic. Consequently eventual inclusion, eventual equality, and finiteness of the symmetric difference are decidable.*

Proof. By Lemma 31.3, inclusion is equivalent to

$$e_L \leq e_K \quad \text{and} \quad \bigwedge_{a \in \Sigma} \forall i, j \in \mathbb{N} (P_a^L(i, j) \Rightarrow P_a^K(i, j)). \quad (50)$$

Sufficiency is coordinatewise. For necessity, if the support implication fails, a witnessing seed word of L already lies outside $\text{Rec}(K)$. This also proves that for positive lengths the failure set is exactly

$$D_{L,K} = \{n > 0 : \exists a, i, j [n = i + j + 1 \wedge P_a^L(i, j) \wedge \neg P_a^K(i, j)]\}. \quad (51)$$

The alphabet quantifier is a finite disjunction. This is an effective unary Presburger set, hence effectively ultimately periodic. Add length zero when $e_L > e_K$. Equality failure is the union of the failure sets in both directions. Presburger arithmetic decides boundedness of these sets. A language over a finite alphabet is finite exactly when its length set is bounded, so the same test decides finiteness of the symmetric difference. $\square$

Corollary 32.2 (Effective witnesses). *When inclusion fails, its least failure length is computable, and a word in $L \setminus \text{Rec}(K)$ is computable. When eventual inclusion fails, one can compute a nonempty arithmetic progression of failure lengths.*

Proof. Find the least element of the effective unary failure set; solve the support formula for a letter and two lengths, then search the finite length slice using CFG membership. A witnessing seed word must exist. An unbounded unary Presburger set contains an effectively computable infinite arithmetic progression in its eventual periodic part. $\square$

Theorem 32.3 (Boolean combinations have effective periodic length sets). *For context-free $L_1, \dots, L_m$ and any Boolean expression B in the languages $\text{Rec}(L_1), \dots, \text{Rec}(L_m)$, the set $\{|w| : w \in B\}$ is effectively ultimately periodic. In particular, emptiness, universality, and finiteness of B are decidable.*

Proof. Convert the Boolean expression into a finite disjunction of conjunctions of positive and negative membership requirements. Treat $n = 0$ by the empty-word bits. Fix one conjunction and $n > 0$. If its positive indices are I , write

$$W(a, t, n) = \bigwedge_{h \in I} P_a^{L_h}(t, n - 1 - t).$$

An empty conjunction is true. Every position must admit at least one positive-compatible letter:

$$\forall t < n \bigvee_{a \in \Sigma} W(a, t, n). \quad (52)$$

For each negative index j , choose a witness position $t_j < n$ and a letter a_j such that $W(a_j, t_j, n)$ holds and $P_{a_j}^{L_j}(t_j, n - 1 - t_j)$ fails. Require that equal witness positions carry equal chosen letters. There are only finitely many choices of the a_j ; after their finite enumeration these conditions are Presburger.

Necessity follows by reading a word meeting the conjunction. Conversely, assign the chosen letters to the witness positions. The compatibility conditions make these assignments consistent. At every remaining position choose a letter supplied by (52). The resulting word satisfies all positive and negative requirements. The existence of such a word is therefore an effective Presburger condition on n . Finite union over the conjunctions proves the theorem. Universality is emptiness of the complement; finiteness is boundedness of the length set. $\square$

These procedures compare the *full closures*; they do not decide inclusion or equality of the original context-free seeds. Nor do they solve the eventual-stabilization problem for arbitrary grammars.

33 Decidable regularity through row types

For a relation $P \subseteq \mathbb{N}^2$ let $\text{row}_P(i) = \{j : P(i, j)\}$. A relation has finite row type if only finitely many such sets occur.

Theorem 33.1 (Regularity criterion and synthesis). *For a context-free seed L , the following are equivalent:*

- (i) $\text{Rec}(L)$ is regular.
- (ii) Every P_a^L has finite row type.
- (iii) Every P_a^L is a finite union of products of unary ultimately periodic sets.
- (iv) There are $B \in \mathbb{N}$ and $p \geq 1$ such that for all a, i, j ,

$$i \geq B \implies (P_a^L(i + p, j) \leftrightarrow P_a^L(i, j)), \quad (53)$$

$$j \geq B \implies (P_a^L(i, j + p) \leftrightarrow P_a^L(i, j)). \quad (54)$$

These conditions are decidable from a grammar. In the positive case a DFA for $\text{Rec}(L)$ is effectively constructible.

Proof. Suppose a DFA with state set Q recognizes $\text{Rec}(L)$. Let S_i be the set of states reachable from the start state by arbitrary words of length i . The truth of $P_a^{\text{Rec}(L)}(i, j)$ is determined by S_i , the transition on a , and existence of an accepting continuation of length j . There are at most $2^{|Q|}$ possible sets S_i , so only finitely many rows. Since $P_a^{\text{Rec}(L)} = P_a^L$, (i) implies (ii).

Assume (ii), and define the Presburger row equivalence

$$\text{Eq}_a(i, r) \iff \forall j \in \mathbb{N} (P_a^L(i, j) \leftrightarrow P_a^L(r, j)). \quad (55)$$

Choose one representative r per class. The sets $U_{a,r} = \{i : \text{Eq}_a(i, r)\}$ and $V_{a,r} = \{j : P_a^L(r, j)\}$ are unary Presburger sets. Thus $P_a^L = \bigcup_r U_{a,r} \times V_{a,r}$, proving (iii). A common threshold and a common multiple of the finitely many unary periods prove (iv). Conversely, separate periodicity partitions each axis into the finitely many values below B and the residue classes above B ; P_a^L is constant on each product cell, proving (iii) directly from (iv).

Given (iii), refine the finitely many unary sets to finite partitions on both axes. The complement of P_a^L is also a finite union of unary ultimately periodic rectangles, say $\bigcup_h U_h \times V_h$. For a unary length set U , write $\Sigma^U = \{u : |u| \in U\}$, a regular language. The words having at least one forbidden coordinate form

$$\bigcup_{a \in \Sigma} \bigcup_h \Sigma^{U_h} a \Sigma^{V_h},$$

which is effectively regular. Its complement has exactly the positive length slices of $\text{Rec}(L)$; adjust membership of ε using e_L . This proves (i) and gives DFA synthesis.

For effectivity, form the set of least row representatives

$$C_a = \{i : \forall r < i \neg \text{Eq}_a(i, r)\}. \quad (56)$$

It is Presburger, and finite row type is exactly boundedness of C_a , expressed by $\exists B \forall i > B (i \notin C_a)$. Presburger decision therefore settles (ii). In the positive case, find a bound, enumerate the representatives, and perform the construction above. Alternatively, (iv) itself is a Presburger sentence: $i + p$ and $j + p$ use addition, not multiplication of variables. $\square$

Theorem 33.2 (Effective infinite obstruction). *If $\text{Rec}(L)$ is nonregular, one can compute a letter a and integers $b \geq 0$, $p \geq 1$ such that the rows $\text{row}_{P_a^L}(b + pm)$, $m \geq 0$, are pairwise distinct. Equivalently,*

$$\forall m \neq m' \exists j \quad P_a^L(b + pm, j) \triangle P_a^L(b + pm', j),$$

where $\triangle$ denotes exclusive disjunction. This gives infinitely many pairwise distinguishable prefixes of a marked rational image of $\text{Rec}(L)$.

Proof. Some canonical representative set C_a is infinite. Its effective unary semilinear form contains an infinite progression $b + p\mathbb{N}$, all of whose members are different representatives. Consider the marked language

$$M_a = \{0^i 10^j : P_a^L(i, j)\}.$$

The prefixes $0^{b+pm}1$ have pairwise different residual languages, as the distinguishing suffixes 0^j show. The language M_a is a rational transduction of $\text{Rec}(L)$: choose one a , relabel it 1, and relabel all other letters 0. A regular $\text{Rec}(L)$ would give regular M_a , impossible. The Presburger formulas and the progression are an effective certificate; a distinguishing j for any pair can be found by arithmetic search. $\square$

Remark 33.3 (Complexity is representation dependent). The proof uses classical monadic decomposition in a specific language application [13, 14, 15]. It gives no polynomial bound in the size of the original grammar. Effective Parikh conversion, quantifier elimination, period extraction, and explicit DFA construction may all expand the representation greatly. The polynomial claim later in this article concerns a different input format: a list of arithmetic rays.

34 One-marker seeds: exact generations

Definition 34.1. For $R \subseteq \mathbb{N}^2$, define

$$L_R = 0^* \cup \{0^i 10^j : (i, j) \in R\}, \quad (57)$$

$$K_R(n) = \{i \in \mathbb{N} : i < n, (i, n-1-i) \in R\}, \quad (58)$$

$$k_R(n) = |K_R(n)|, \quad K(R) = \sup_{n \geq 0} k_R(n). \quad (59)$$

Here $K_R(0) = \emptyset$, and $K(R)$ may be infinite. Write $\text{Rec}_R = \text{Rec}(L_R)$.

Proposition 34.2 (Every semilinear mask has a linear seed). *If R is semilinear, then L_R is effectively linear context-free.*

Proof. For a linear component $(b, c) + \mathbb{N}(u_1, v_1) + \cdots + \mathbb{N}(u_t, v_t)$ use one nonterminal A with

$$A \longrightarrow 0^{u_\ell} A 0^{v_\ell} \quad (1 \leq \ell \leq t), \quad A \longrightarrow 0^b 10^c.$$

The number of each recursive production used is arbitrary, so the exponents are exactly those of the component. Finite union of these linear grammars and a grammar for 0^* gives (57). A period $(0, 0)$ can be discarded. The construction is effective, but writing 0^u as u explicit terminals is not polynomial in a binary encoding of u . $\square$

Theorem 34.3 (Exact parallel and frozen-source generations). *For every $R \subseteq \mathbb{N}^2$, every $g \geq 0$, and every binary word $w \in \{0, 1\}^*$,*

$$w \in \text{Rec}_R \iff \text{supp}_1(w) \subseteq K_R(|w|), \quad (60)$$

$$w \in L_R^{[g]} \iff w \in \text{Rec}_R \text{ and } |w|_1 \leq 2^g, \quad (61)$$

$$w \in Y_g \iff w \in \text{Rec}_R \text{ and } |w|_1 \leq g + 1. \quad (62)$$

The empty word belongs to every language here. Hence finite stabilization occurs precisely when $K(R) < \infty$. Its exact depths are

$$d_{\parallel}(R) = \lceil \log_2 \max(1, K(R)) \rceil, \quad d_{\text{frozen}}(R) = \max(0, K(R) - 1). \quad (63)$$

Proof. Zeros are always available because $0^* \subseteq L_R$, and the allowed one positions are precisely $K_R(n)$. Lemma 31.3 gives (60). At generation zero the one-count bound is one. A crossover of two words with at most b ones has at most $2b$ ones. Conversely, for a target with at most $2b$ ones, cut between the b -th and $(b+1)$ -st one when both exist, or at an endpoint otherwise. Use as first parent the target's ones before the cut and as second parent its ones at or after the cut, filling other positions with zeros. Each parent is in the hull with at most b ones, and their crossover is the target. Induction proves (61). The analogous argument removes the last one and uses a single-marker donor for it, proving (62).

If $K(R) < \infty$, the first bounds covering all possible one counts give (63). If a bound has not reached $K(R)$, there exists a length with more available sites than that bound, and a word with exactly one more one witnesses strictness of the next generation. This also proves nonstabilization when $K(R) = \infty$. $\square$

For a nonempty target with r ones, the inherited aligned-fragment rank has the particularly simple value $\max(1, r)$: every donor fragment has at most one one, and cuts between successive ones attain that bound. Theorem 34.3 is also a direct proof that does not require assuming any unformalized rank theorem from the preceding report.

[write, batch 66] The rank value is Part I's (20) in general form. Theorem 34.3 and Corollary 34.4 contain three earlier examples: Part I's Example 3.6 ($R = \mathbb{N}^2$); Part I's seed S_q of Section 9, which is L_R for $R = \{(i, j) : j \geq (q-1)i + q - 1\} \cup \{(i, j) : i \geq (q-1)j + q - 1\}$, with $k_R(qm + r) = 2m$, so that the corollary gives (22) and (23); and Part II's family L_m (Theorem 22.1), which is L_R without the all-zero words for the finite mask $R = \{(i, m-1-i) : 0 \leq i < m\}$, where $K(R) = m$ gives the same depths $\lceil \log_2 m \rceil$ and $m-1$.

Corollary 34.4 (Exact slice enumeration). *For every $n, g \geq 0$,*

$$|\text{Rec}_R \cap \{0, 1\}^n| = 2^{k_R(n)}, \quad |L_R^{[g]} \cap \{0, 1\}^n| = \sum_{r=0}^{\min(k_R(n), 2^g)} \binom{k_R(n)}{r}.$$

In particular, Rec_R is slender, meaning uniformly bounded in the number of words per length, exactly when finite stabilization occurs.

Regularity and finite stabilization are different properties. For example, $R = \mathbb{N}^2$ gives the regular closure $\{0, 1\}^*$ but has $K(R) = \infty$. The next sections classify the finitely stabilizing case completely.

35 Sparse semilinear masks and arithmetic rays

Theorem 35.1 (Decidable sparsity and effective ray normalization). *Let $R \subseteq \mathbb{N}^2$ be semilinear. Then $K(R) < \infty$ if and only if, in any finite linear-set presentation of R , the nonzero period vectors within each component are collinear. In that case R is effectively a finite union of arithmetic rays and finitely many points:*

$$R = F \cup \bigcup_{v=1}^r ((b_v, c_v) + \mathbb{N}(u_v, v_v)), \quad u_v + v_v > 0. \quad (64)$$

Thus finite stabilization of semilinear one-marker seeds is decidable.

Proof. Suppose one component has nonparallel periods $p, q \in \mathbb{N}^2$. Let $s = p_1 + p_2 > 0$ and $t = q_1 + q_2 > 0$. For every $N \geq 0$ the $N+1$ points

$$b + (N-h)tp + hsq, \quad h = 0, \dots, N,$$

belong to that component and have the same coordinate sum $b_1 + b_2 + Nst$. They are distinct because $tp \neq sq$. Consequently $K(R)$ is unbounded.

If all nonzero periods of a component are collinear, its points lie on one line with a nonnegative nonzero direction. Such a line intersects any diagonal $i + j = n - 1$ in at most one point. A finite union of components therefore gives a uniform bound; singleton components have the same property. Since a component is contained in R , the first argument also shows that no presentation of a sparse R can contain a nonparallel period pair.

It remains to normalize a collinear component effectively. Write its periods as $a_1 d, \dots, a_t d$, where $d \in \mathbb{N}^2$ is primitive and each a_ℓ is positive. Divide the a_ℓ by their gcd g . The resulting nonnegative integer semigroup has gcd one and is cofinite. Here is an explicit effectivity argument. Choose its least positive generator a . The residues of nonnegative combinations modulo a form a subgroup of the finite cyclic group and, by gcd one, are all residues. A finite shortest-path search on residues supplies a represented integer h_r in each residue class r . Every integer at least $H = \max_r h_r$ is $h_r + ma$ with $m \geq 0$, hence represented. Enumerate represented integers below H exactly; all larger integers form a tail. Returning to the original coordinates gives one arithmetic ray of step gd plus finitely many points. A component with no nonzero periods is already a point. Finite union gives (64). $\square$

The normalization may enlarge the description. No polynomial bound is claimed when a general semilinear or Presburger description is the input. The finite points must be retained for exact depth: discarding them is harmless for language-class membership but can change the largest number of simultaneous marker sites.

Definition 35.2 (Length period and velocity). For an infinite ray indexed by v , set

$$n_v = b_v + c_v + 1, \quad q_v = u_v + v_v, \quad \lambda_v = \frac{u_v}{q_v}, \quad f_v(n) = \lambda_v n + b_v - \lambda_v n_v.$$

The ray contributes exactly one site, at position $f_v(n)$, precisely when $n \geq n_v$ and $n \equiv n_v \pmod{q_v}$. Its *velocity* is $\lambda_v \in [0, 1] \cap \mathbb{Q}$. It is *interior* if $0 < \lambda_v < 1$. Two rays are *length compatible* if

$$n_v \equiv n_w \pmod{\gcd(q_v, q_w)}. \quad (65)$$

Length compatibility says that the two rays occur together at arbitrarily large lengths. Velocity zero means a fixed distance from the left endpoint; velocity one means a fixed distance from the right endpoint. An interior site recedes linearly from both endpoints.

36 A finite-ray pumping principle

The following elementary statement isolates the point at which taking a union might otherwise invalidate a pumping argument. It is a specialized form of the paired-loop phenomenon in slender language theory [16].

For $s \geq 0$ and vectors $a, d \in \mathbb{N}^{s+1}$ with $d \neq 0$, define the *gap-ray language*

$$\text{Gap}_s(a, d) = \{0^{a_0+d_0t} 10^{a_1+d_1t} 1 \dots 10^{a_s+d_st} : t \in \mathbb{N}\}. \quad (66)$$

There are exactly s ones, and the gap vector is uniquely determined by the word. Write $v(d) = |\{h : d_h > 0\}|$.

Lemma 36.1 (Finite-ray trichotomy). *Let $\mathcal{T}$ be a finite union of languages (66), with possibly different s , together with finitely many exceptional words. Then:*

- (i) $\mathcal{T}$ is regular if and only if every ray in the union has $v(d) \leq 1$.
- (ii) $\mathcal{T}$ is context-free if and only if every ray in the union has $v(d) \leq 2$. In that case $\mathcal{T}$ is linear context-free.

These tests apply to any such presentation, including overlapping rays.

Proof. For sufficiency in (i), a ray with just one growing gap is a fixed word with a factor $(0^h)^*$ inserted at a fixed location, hence regular. For sufficiency in (ii), if two gaps grow, the words have the form

$$U 0^{pt} V 0^{qt} W, \quad t \geq 0,$$

for fixed words U, V, W and positive p, q . The grammar $A \rightarrow 0^p A 0^q \mid V$, with fixed outer words U, W , is linear. The cases of one growing gap, finite words, and finite unions preserve linearity.

For necessity, treat regularity and context-freeness simultaneously, with $h = 1$ and $h = 2$, respectively. Suppose a represented ray with $v(d) > h$ occurs in $\mathcal{T}$. Work in the fixed space of gap vectors with its number s of ones. Its affine line cannot coincide with the affine line of a ray with at most h growing coordinates. Each such low-support line therefore meets it in at most one point. Choose a word far enough along the high-support ray to avoid all these finitely many intersections, to exceed every exceptional word length, and to exceed a pumping length.

A regular pumping decomposition repeats one nonempty factor. A context-free pumping decomposition repeats two factors, at least one nonempty. If any pumped factor contains a one, pumping it sufficiently many times exceeds the globally bounded number of ones in $\mathcal{T}$, a contradiction. Thus all pumped nonempty factors consist only of zeros. Their repetition adds a nonzero gap vector $e \geq 0$ supported in at most h coordinates. The pumped words have gap vectors

$$x + me, \quad m = 0, 1, 2, \dots,$$

where x is the chosen original vector. Infinitely many of these vectors must belong to one ray of the finite presentation. Taking two distinct such vectors shows that its direction is parallel to e . That ray therefore has at most h growing coordinates, and its affine line contains x . This contradicts the choice of x .

The argument explicitly handles a finite union. It does not infer non-context-freeness merely from containment of a non-context-free sublanguage, which would be invalid. $\square$

Lemma 36.2 (Gap-ray normal form for sparse hulls). *For a finite union of arithmetic rays and finitely many points as in (64), both Rec_R and every finite generation $L_R^{[g]}$ are effectively finite unions of gap-ray languages and finitely many words.*

Proof. Let M be the least common multiple of the infinite rays' length periods, taking $M = 1$ if there are none. For each residue class modulo M , discard finitely many initial lengths so that all applicable rays have started and the order of their affine site functions has stabilized. Distinct affine functions cross at most once, so there are only finitely many order exceptions. Identical functions are merged on that residue class. Also discard the finitely many lengths carrying exceptional points.

Write the remaining lengths of this class as $n = N + Mt$, $t \geq 0$. The available distinct positions then have the form $i_j(t) = \alpha_j + \beta_j t$, with integers $\beta_j = M\lambda_j$ and a fixed order. Every subset of these positions can carry ones. If its selected positions are $i_1(t) < \dots < i_s(t)$, the zero gaps have slopes

$$(\beta_1, \beta_2 - \beta_1, \dots, \beta_s - \beta_{s-1}, M - \beta_s). \quad (67)$$

They are nonnegative: eventual ordering forces the site slopes to be nondecreasing. Their sum is $M > 0$. The constants are the nonnegative gaps at $t = 0$. This gives a gap ray. For the empty subset it gives the all-zero ray. There are only finitely many residues and subsets. A finite generation retains just the subsets of size at most 2^g , by Theorem 34.3. The discarded length slices are finite, so their words give the finite exceptions. $\square$

37 The sparse-marker language-class trichotomy

Theorem 37.1 (Velocity classification). *Let R be a finite union of arithmetic rays and finitely many points. Exactly one of the following alternatives holds.*

1. **Endpoint case.** *No infinite ray is interior. Then Rec_R and every $L_R^{[g]}$ are regular.*
2. **Single co-occurring interior velocity.** *There is an interior ray, but there is no length-compatible pair of interior rays with distinct velocities. Then Rec_R is linear context-free and nonregular, and every $L_R^{[g]}$ is linear context-free and nonregular.*
3. **Two distinct co-occurring interior velocities.** *There are length-compatible rays v, w with $0 < \lambda_v < \lambda_w < 1$. Then Rec_R is not context-free. Already $L_R^{[1]}$ is not context-free, and neither is any $L_R^{[g]}$ with $g \geq 1$.*

The seed L_R is always linear context-free. The alternative is decidable using a quadratic number of gcd and rational comparisons in the number of rays, with polynomial bit complexity for binary-encoded coordinates.

Proof. Use the gap-ray normal form of Lemma 36.2. In the endpoint case every selected site has velocity zero or one. The vector in (67) has at most one nonzero coordinate: the only possible change is from zero to M , including a change at an endpoint. Lemma 36.1 makes the hull and every generation regular.

In the second case, within any common length residue class, all interior sites have the same velocity λ . Thus the distinct site velocities are drawn from $\{0, \lambda, 1\}$. There are at most two positive successive differences in (67), so all gap rays are linear context-free. An interior ray supplies single-marker words whose two zero gaps both grow. Restrict the hull or any generation to words with exactly one one. This restricted language is a finite union of gap rays containing a two-growing-gap ray; by Lemma 36.1 it is nonregular. Regular languages are closed under intersection with the regular language 0^*10^* , so the hull and all generations are nonregular.

In the third case, the generalized Chinese remainder theorem gives an infinite common length progression for v and w . Beyond a finite threshold their positions are ordered and distinct. Select just these two sites. The resulting three gap slopes are positive multiples of

$$\lambda_v, \quad \lambda_w - \lambda_v, \quad 1 - \lambda_w.$$

Consequently the hull's exact-two-ones sublanguage has a gap-ray presentation containing a ray with three growing gaps. It is not context-free by Lemma 36.1. The same exact-two-ones sublanguage occurs in every generation $g \geq 1$, by Theorem 34.3. Intersecting a hypothetical context-free generation with the regular language $0^*10^*10^*$ would make this sublanguage context-free, a contradiction.

The seed claim is Proposition 34.2. For the algorithm, first test for interior rays. If any exist, test every pair for unequal rational velocities and (65). These arithmetic operations have polynomial bit complexity. Finite points do not affect the language class. $\square$

Corollary 37.2 (Classification for every finitely stabilizing semilinear one-marker seed). *Given a semilinear R , finite stabilization of L_R is decidable. When it occurs, the language-class alternatives of Theorem 37.1 are decidable after effective ray normalization. In particular, every context-free sparse hull in this family is linear context-free. The same conclusions hold from a CFG promised to describe a binary seed containing 0^* and having at most one one per word: its semilinear mask is exactly P_1^L .*

[write, batch 66] For this family of seeds, Theorem 37.1 and Corollary 37.2 answer Part I's Question 11.4 completely, through a restriction on the position support $P_1^L = E_1$ of the kind that question suggests: a finitely stabilizing semilinear one-marker seed has a context-free hull exactly when no two length-compatible interior rays have distinct velocities. Part I's seed S_q is not covered: its mask has components with non-collinear periods, and $K(R) = \infty$ (as $k_R(qm + r) = 2m$), so its non-context-freeness (Theorem 9.2) lies outside the sparse case.

Example 37.3 (Different velocities need not interact). Take rays with bases $(0, 0)$ and $(0, 1)$ and steps $(1, 3)$ and $(3, 1)$, respectively. Their velocities are $1/4$ and $3/4$, but their lengths are $1 \pmod{4}$ and $2 \pmod{4}$. They never coexist. The hull is linear context-free and nonregular, and $K(R) = 1$, so it is already equal to its seed. Dropping the compatibility condition would give a false non-context-freeness test.

Remark 37.4 (Extension to finitely many marker colors). A fixed neutral letter 0 may be supplemented by several marker colors, with each seed word containing at most one nonzero letter. For a finite union of colored arithmetic rays, the available color set at a site is constant on each sufficiently refined length progression. After choosing sites and colors, the same gap-ray proof works with a fixed finite marker word instead of a string of ones. The language-class test depends on the union of nonzero sites, and exact depth depends on

their maximum number, not on the number of available colors. Pumping a nonzero letter again violates the bounded total number of nonzero letters. This observation does not cover seeds with two or more nonzero letters per donor.

38 An exact compatibility-graph formula

The velocity test uses only pairs of rays. Optimal stabilization depth, by contrast, depends on arbitrarily large compatible collections.

Definition 38.1. For the infinite rays of (64), let Γ_R be the simple graph with an edge between distinct vertices v, w precisely when

$$n_v \equiv n_w \pmod{\gcd(q_v, q_w)} \quad \text{and} \quad f_v \neq f_w \text{ as affine functions.} \quad (68)$$

The clique number of the empty graph is zero. Let $E = \{i + j + 1 : (i, j) \in F\}$ be the finite set of exceptional point lengths.

Theorem 38.2 (Clique formula for maximum support). *If $F = \emptyset$, then*

$$K(R) = \omega(\Gamma_R). \quad (69)$$

In general,

$$K(R) = \max \left(\omega(\Gamma_R), \max_{n \in E} k_R(n) \right), \quad (70)$$

where an empty inner maximum is zero. In particular, with no exceptional points,

$$d_{\parallel}(R) = \lceil \log_2 \max(1, \omega(\Gamma_R)) \rceil.$$

Every clique is realized by distinct available sites on an effectively computable infinite arithmetic progression after a finite threshold.

Proof. Fix a length containing only infinite-ray contributions. Choose one contributing ray for each distinct available position. Any two chosen rays have compatible length congruences. They cannot have identical affine position functions because their positions at this length are distinct. Hence the chosen vertices form a clique, giving $k_R(n) \leq \omega(\Gamma_R)$.

Conversely, let S be a clique. Pairwise consistency of the congruences $n \equiv n_v \pmod{q_v}$ implies their joint consistency by the generalized Chinese remainder theorem. Their common solutions form an infinite arithmetic progression. Take solutions above every starting length n_v . For two vertices in S , their affine functions are distinct; they either never meet or meet at one length. Avoid the finitely many crossings. Every remaining common length has $|S|$ distinct available positions. This proves (69), and the construction is effective using CRT and rational comparisons.

Finite points can affect support only at lengths in E . At all other lengths the preceding bound holds, while arbitrarily large lengths still realize a maximum clique and avoid E . Checking the finitely many exceptional lengths gives (70). $\square$

Remark 38.3 (Why a graph of congruences alone is insufficient). Two rays may have different starting points or length periods but the same affine position function. When they coexist they offer one site, not two. This is why (68) excludes identical functions. By contrast, distinct functions that cross at one length still give an edge: the crossing is a finite exception and cannot change the cofinal maximum. Both cases are included in the executable tests.

Corollary 38.4 (Effective maximizing witnesses). *For a sparse ray input, one can compute $K(R)$, both stabilization depths, and a word with $K(R)$ ones in the hull. For every $g < d_{\parallel}(R)$, one can compute a word witnessing $L_R^{[g]} \neq L_R^{[g+1]}$.*

Proof. Compute a maximum clique by finite search and compare exceptional lengths. CRT and crossing avoidance supply a maximizing length for the clique, or an exceptional length realizes the larger value. Select all sites for the first claim. For strictness at generation g , choose any $2^g + 1$ of the maximizing sites. This count is at most 2^{g+1} , so the resulting word belongs to the next generation but not the current one. $\square$

Computing the clique number may require exponential time. The next result shows that this is not merely an artifact of this particular formula.

39 Exact-depth complexity for succinct regular seeds

All coordinates and thresholds in this section are encoded in binary. The input is a list of rays, not an explicit DFA or an expanded grammar. The phrase “regular seed” refers to the represented language, not to polynomial-size availability of a recognizing DFA.

Lemma 39.1 (Every finite graph is an endpoint-ray graph). *Given a finite simple graph G on N vertices, one can construct, in polynomial time and with polynomially many bits, a union R_G of N vertical arithmetic rays such that*

$$\Gamma_{R_G} \cong G, \quad K(R_G) = \omega(G).$$

Both L_{R_G} and Rec_{R_G} are regular.

Proof. Label the vertices $0, \dots, N-1$. Assign a different prime p_{vw} to each nonedge $\{v, w\}$ with $v < w$. For a vertex v , let m_v be the product of the primes assigned to its incident nonedges, with empty product one. Impose residue zero at the smaller endpoint and residue one at the larger endpoint of each nonedge. The ordinary CRT combines these conditions into one residue $c_v \pmod{m_v}$.

For a nonedge $\{v, w\}$, the two vertex moduli share exactly its private prime, and the assigned residues disagree there. Thus the two congruences are inconsistent. For an edge they share no assigned prime, so their moduli are coprime and the congruences are consistent. Choose an integer $B_v \geq v+1$ with $B_v \equiv c_v \pmod{m_v}$, and use the ray

$$(v, B_v - v - 1) + \mathbb{N}(0, m_v). \tag{71}$$

Its marker position is the constant v , distinct at different vertices, and its length congruence is exactly the one constructed. Therefore the compatibility graph is G , and Theorem 38.2 gives the clique identity.

The seed is a finite union of 0^* and languages $0^v 10^{B_v - v - 1} (0^{m_v})^*$, so it is regular. All rays have velocity zero, so the hull is regular by Theorem 37.1. There are at most $N(N-1)/2$ primes. Standard bounds on the r -th prime, for example those of Rosser and Schoenfeld [18], permit their generation by a polynomial-size sieve. Each product m_v has polynomial bit length, and CRT uses polynomial-time integer arithmetic. A valid B_v is obtained by adding the least necessary multiple of m_v , also with polynomially many bits. $\square$

Theorem 39.2 (Support and depth thresholds). *For binary-encoded finite ray unions:*

- (i) *Deciding $K(R) \geq k$, with k part of the input, is NP-complete.*
- (ii) *Deciding whether the parallel process stabilizes by an input integer $g \geq 0$ is coNP-complete.*

Both hardness statements hold with no finite exceptional points and with all rays vertical, so both seed and full closure are regular.

Proof. For (i), a clique of size k is an NP certificate for the cofinal maximum. With finite exceptional points allowed, an exceptional point length together with direct support evaluation is an alternative certificate. The clique formula proves completeness of these certificates. If k exceeds the number of supplied rays and points, the answer is immediately negative. NP-hardness follows from CLIQUE [17] and Lemma 39.1.

For (ii), stabilization by g is equivalent to $K(R) \leq 2^g$. Its failure has the NP certificates just described, of size $2^g + 1$; when this exceeds the number of input components, failure is impossible. One need not construct an exponentially long binary integer for 2^g : first compare g with the logarithmic bound from the input component count. Thus membership in coNP holds also when g itself is binary.

For hardness, start with a nontrivial CLIQUE instance (G, k) with $2 \leq k \leq |V(G)|$. Put

$$d = \lceil \log_2 k - 1 \rceil, \quad \theta = 2^d + 1, \quad u = \theta - k.$$

Here $0 \leq u < k$. Join G with a clique of u new universal vertices, obtaining G' . Then $\omega(G') = \omega(G) + u$, and therefore

$$\omega(G) \geq k \iff \omega(G') \geq 2^d + 1 \iff d_{\parallel}(R_{G'}) > d.$$

This is a polynomial reduction from CLIQUE to failure of stabilization by an input depth. Its complement is coNP-hard, as required. $\square$

Remark 39.3 (Two distinct complexity boundaries). The structural class of a sparse ray hull uses only pair tests and is polynomial-time decidable, while optimal depth detects an arbitrary clique. There is no contradiction: language class and exact resource optimum are different invariants. Nor does the theorem give coNP-hardness for every fixed depth. For a fixed g , searching all subsets of size $2^g + 1$ is polynomial in the number of rays. Finally, the binary periods in (71) may require very large expanded automata. This result does not establish the optimal complexity of the explicit-DFA stabilization problem studied by the preceding ProveIt report.

[write, batch 66] That explicit-automaton problem is Part I's Question 11.2. Part II, written in parallel, settles it for NFA input (PSPACE-complete over two letters, Corollary 20.3) and places DFA input in PSPACE (Theorem 19.2); the exact DFA complexity remains open (Question 24.1). The two complexity results do not conflict: Part II decides whether stabilization happens at all for an explicit automaton, whereas Theorem 39.2 thresholds the exact depth for a succinct list of rays, whose periods can force exponentially large automata.

40 Explicit examples and arbitrary finite depths

40.1 A four-word-per-length non-context-free fixed point

Theorem 40.1 (One crossover suffices). *Let*

$$L = 0^* \cup \{0^t 10^{2t+1} : t \geq 0\} \cup \{0^{2t+1} 10^t : t \geq 0\}. \quad (72)$$

Then L is binary and linear context-free, and

$$T(L) = \text{Rec}(L) = L \cup \{0^t 10^t 10^t : t \geq 0\}. \quad (73)$$

Its exact stabilization depth is one. Its full closure is not context-free, but has four words at lengths $2 \pmod{3}$ and one word at every other length. Its generating function is

$$\sum_{n \geq 0} |\text{Rec}(L) \cap \{0, 1\}^n| z^n = \frac{1 + z + 4z^2}{1 - z^3}. \quad (74)$$

Proof. An explicit linear grammar is

$$\begin{aligned} S &\rightarrow Z \mid A \mid B, \\ Z &\rightarrow 0Z \mid \varepsilon, \\ A &\rightarrow 0A00 \mid 10, \\ B &\rightarrow 00B0 \mid 01. \end{aligned}$$

The two marker rays have bases $(0, 1)$, $(1, 0)$ and steps $(1, 2)$, $(2, 1)$. At lengths $n = 3t + 2$ they offer positions t and $2t + 1$; no other length offers a one. Thus the hull consists of the all-zero word, the two single-marker words, and the word placing ones at both positions. The latter is $0^t 10^t 10^t$. Theorem 34.3 gives (73) and depth one; at $t = 0$, the word 11 already witnesses strictness.

The two velocities are $1/3$ and $2/3$ and are length compatible, so Theorem 37.1 proves non-context-freeness. More directly, intersection with $0^* 10^* 10^*$ gives precisely $\{0^t 10^t 10^t : t \geq 0\}$, a gap ray with three positive slopes, which is not context-free by Lemma 36.1. The counts and the geometric-series sum give (74). $\square$

This example separates two phenomena especially clearly. The process has no long transient: it is finished after one generation. Yet it has already crossed the context-free boundary. Conversely its growth is bounded by four words per length, so even extremely small rational growth does not prevent the loss of context-freeness. The preceding ProveIt example with an unbounded transient addresses a different phenomenon; it is not needed in this proof. [write, batch 66] That example is Part I's S_q (Theorems 9.2 and 9.4). The series (74) and (76) are further instances of the commutative series that Part I's Question 11.6 asks to characterize; they have the form of Theorem 8.3 with $p = 3$ and $p = k + 1$.

40.2 A uniform family with prescribed depth

For $k \geq 1$ set

$$R_k = \bigcup_{j=1}^k ((j-1, k-j) + \mathbb{N}(j, k+1-j)), \quad L_k^{\text{fam}} = L_{R_k}. \quad (75)$$

For each j the marker branch is generated by $A_j \rightarrow 0^j A_j 0^{k+1-j} \mid 0^{j-1} 10^{k-j}$; adjoin 0^* .

Theorem 40.2 (All finite depths and exact generation counts). *For $k \geq 1$, the active lengths are exactly $n = (k+1)t + k$, $t \geq 0$. At such a length there are k available sites, at positions $j(t+1) - 1$ for $j = 1, \dots, k$. Consequently*

$$K(R_k) = k, \quad d_{\parallel}(R_k) = \lceil \log_2 k \rceil, \quad d_{\text{frozen}}(R_k) = k - 1.$$

Put $B_{k,g} = \sum_{r=0}^{\min(k, 2^g)} \binom{k}{r}$. Then

$$\sum_{n \geq 0} |L_k^{\text{fam}[g]} \cap \{0, 1\}^n| z^n = \frac{1}{1-z} + (B_{k,g} - 1) \frac{z^k}{1-z^{k+1}}. \quad (76)$$

The full closure replaces $B_{k,g}$ by 2^k . For $k = 1$ the seed is already a linear nonregular fixed point. For every $k \geq 2$, all generations from one onward are non-context-free. Every positive integer d is the exact stabilization depth of such a binary linear seed, for instance with $k = 2^{d-1} + 1$.

Proof. The j -th ray has coordinate sum $k - 1 + (k+1)t$, so its word length is $(k+1)t + k$ and its left coordinate is $j - 1 + jt = j(t+1) - 1$. These positions are distinct. The depth and counts follow from Theorem 34.3 and Corollary 34.4. Summing the count excess over the active arithmetic progression gives (76). The velocities are $j/(k+1)$, and all rays have the same length progression. For $k = 1$ there is one interior velocity; for $k \geq 2$ there are two distinct co-occurring interior velocities. Theorem 37.1 completes the proof. $\square$

For example, $k = 5$ has seed counts six at each active length, then generation counts 16, 31, and 32. The process stabilizes at depth three, although non-context-freeness appears already at depth one. The closure count does not disclose at which generation the last support combinations became available.

40.3 A regular seed with the same quantitative difficulty

Take the cycle graph C_5 and apply Lemma 39.1. The output rays all have fixed left positions, so the seed and hull are regular. Their maximum support is $\omega(C_5) = 2$, hence depth one. Replacing C_5 by any finite graph encodes its clique number without leaving the regular language class. The supplied example file `endpoint_cycle5.json` records the five explicit rays and the computed result. This is a compact demonstration of the succinctness qualification in Theorem 39.2. [write, batch 66] The example file is shipped as `data/03-closure-classification-endpoint_cycle5.json`, with the result in `data/03-closure-classification-endpoint_cycle5.result.json`.

41 Verification, implementation, and proof status

41.1 What the executable component does

The package contains an exact standard-library Python implementation for finite unions of rays and points. A ray is specified by nonnegative integer pairs `base` and `step`; a zero step denotes one finite point. The analyzer computes the structural language class, compatibility graph, maximum support, both stabilization depths, and a maximum clique. For a non-context-free case it returns a pair of distinct interior velocities whose length congruences are compatible, including their common congruence. The returned indices refer to the list of infinite rays, with finite points omitted.

The implementation uses arbitrary-precision integers, exact rational numbers, gcd, generalized CRT, and an exact branch-and-bound maximum clique search. The class test is polynomial in the ray description; the clique routine has exponential worst-case running time, consistent with the hardness theorem. It is a reference analyzer rather than a claim of optimal practical clique software.

```
python code/crossover_masks.py examples/two_markers.json
python code/crossover_masks.py examples/endpoint_cycle5.json
python code/verify.py
```

[write, batch 66] In this report the files named above are shipped as

- `code/03-closure-classification-crossover_masks.py` (the analyzer),
- `code/03-closure-classification-verify.py` (the verifier),
- `data/03-closure-classification-<name>.json` (the example inputs `examples/<name>.json`), and
- `data/03-closure-classification-<name>.result.json` (their recorded analyses).

Do not run the verifier in place: it imports the analyzer under its delivered module name `crossover_masks`, and it always writes `../data/verification.json` relative to its own directory, which in this report is Part I's receipt. Section 45 and the report's README give a copy-based recipe; the analyzer itself imports nothing of the package and can be run in place on a shipped example, its result going to standard output or to an explicit `--output` path outside the report.

41.2 Exact finite audit

The verification script was executed successfully with Python 3.13.5. It recorded 12,867 checks, grouped below. All checks passed. Randomized test generation uses the fixed seeds 20260930 and 61030; every check itself uses exact integer arithmetic and exact finite sets.

Audit group	Checks
Generalized CRT, all moduli through 12 and all residue pairs	6,084
Basic edge cases	5
Uniform-family supports, literal generations, counts, depths, classes	288
Depth-one example, exact set comparisons and counts	93
Random ray unions: clique formula versus complete-period scan	600
Random ray unions: pair criterion versus all length phases	600
Random ray unions: literal crossover generations	1,800
All labeled graphs on 1–5 vertices: graph, clique, regularity checks	3,297
Larger random graph reductions	100
Total	12,867

The complete-period scan explicitly goes past starting thresholds and all affine crossings, then covers a full common length period. Thus it checks more than a heuristically chosen short prefix for the finite ray instances tested. The 1,099 labeled graphs on one through five vertices are all checked, not sampled. Each contributes three tests: graph realization, preservation of the clique number, and regularity of the endpoint-ray instance.

41.3 What has not been implemented or certified

The code does *not* implement a general CFG-to-Presburger compiler, a Presburger quantifier eliminator, arbitrary semilinear normalization, or the DFA synthesis in Theorem 33.1. Those are proved algorithms using classical effective constructions, not features of the ray analyzer. The executable tests do not decide context-freeness of arbitrary grammars; they exercise the proved sparse-ray criterion.

No Lean or Rocq formalization was written or built for this manuscript. The 12,867 passing finite checks are not proofs of universal statements. They are independent exact audits of implementations and finite specializations; the universal conclusions depend on the mathematical proofs above. In particular, testing graph realizations through five vertices does not replace the all-graphs CRT construction.

41.4 A practical formalization route

A staged formalization could begin with finite words and the product hull, then establish the one-count generation theorem and binomial slice counts. The next layer is finite arithmetic: generalized CRT, affine crossing avoidance, compatibility graphs, and the maximum-support theorem. These parts avoid general context-free grammar theory.

The language-class proof requires a separate finite-gap-ray library and an explicit pumping lemma interface. Its crucial statement is the finite-union obstruction of Lemma 36.1; formalizing only a single obstructing sublanguage would not suffice. The global-regularity algorithm then needs effective Parikh semilinearity, Presburger quantifier elimination, unary periodic language synthesis, and finite-state transductions. The repository’s Presburger development is a natural possible bridge, but no such integration is claimed here.

[write, batch 66] That development is `Logic/PresburgerArithmetic`, described in Section 10: Cooper quantifier elimination in Lean, with the decision procedure `PresburgerArithmetic.Formula`.

`presburgerArithmetic_decidable`. It is not connected to Part III; placement beside it confers no formal status, and no statement of Part III is formalized. No ProveIt Lean or Rocq file defines context-free grammars, Parikh images, semilinear sets or crossover.

42 Further research questions

The following are proposed research directions. They are not all asserted to be globally new open problems; each states a precise next target left unresolved by this manuscript.

Question 42.1 (The nonsparse context-freeness problem). Given a Presburger relation $R \subseteq \mathbb{N}^2$ with unbounded diagonal sections, is context-freeness of Rec_R decidable? The sparse criterion uses a bounded number of markers and finite gap rays, which fail here. The example $R = \mathbb{N}^2$ already shows that unbounded support does not force non-context-freeness. A useful first target is a finite union of rank-two linear sets with a fixed small number of periods.

[write, batch 66] This continues Part I's Question 11.4 beyond the sparse case answered in Section 37; Part I's seed S_q (Theorem 9.2) is a non-context-free nonsparse instance.

Question 42.2 (Complexity from an actual grammar). What is the optimal complexity of deciding regularity of $\text{Rec}(L)$ from a CFG for L ? Theorem 33.1 proves decidability but allows large intermediate representations. Determine sharp bounds for linear grammars, unambiguous grammars, and grammars whose position supports admit short quantifier-free formulas. Monadic-decomposition bounds cannot be transferred without accounting for this conversion.

Question 42.3 (Explicit automata versus succinct ray descriptions). What is the optimal finite-stabilization complexity for explicit DFA and NFA seeds? The preceding report supplies decidability machinery, while Theorem 39.2 gives a lower bound for binary-encoded ray descriptions. Closing the gap requires either a reduction with small explicit automata or an improved automata algorithm, not merely expanding the enormous periods of our construction.

[Added 30 September 2026, batch 66: Part II answers the NFA case: finite stabilization of NFA seeds is PSPACE-complete, already over a binary alphabet (Corollary 20.3). For DFA seeds it gives a PSPACE upper bound (Theorem 19.2) and a safe-core criterion (Theorem 21.2), improving Part I's EXSPACE bound; the exact DFA complexity is open, and is Part II's Question 24.1. What remains of this question is that DFA case and the gap between explicit automata and the succinct ray inputs of Theorem 39.2.]

Question 42.4 (Minimum regular representations and certificates). When a full closure is regular, how large must its minimal DFA be in terms of support row types, column types, and the original grammar? Can a minimal or near-minimal symbolic automaton be synthesized without materializing every period class? In the negative case, can the progression certificate of Theorem 33.2 be shortened or canonically chosen? Recent decomposition-size lower bounds make output representation a central issue [15].

Question 42.5 (Arithmetic restrictions on the compatibility graph). Which restrictions on the ray moduli make exact depth tractable? Promising cases include boundedly many prime factors, a divisibility chain of moduli, a fixed modulus, unary-encoded coordinates, and bounded treewidth of the compatibility graph. The graph realization shows that no unrestricted graph-theoretic simplification is available, but special arithmetic classes may force perfect or otherwise tractable graphs.

Question 42.6 (Approximation and parameterized depth). Can useful approximation or fixed-parameter guarantees be obtained for $K(R)$ or $d_{\parallel}(R)$ under natural arithmetic restrictions? The logarithm compresses

the optimum but retains exact threshold hardness. Distinguish approximating $K(R)$ multiplicatively from approximating depth additively, and analyze the role of the largest number of simultaneously compatible periods.

Question 42.7 (More than one marker per donor). For seeds with at most $b \geq 2$ nonzero letters per word, the full hull still depends only on one-coordinate supports, but generation depth no longer does. What finite combinatorial invariant replaces the one-count formula? Even two-marker donors can impose geometric constraints on which groups of sites are inherited by one aligned fragment. An exact classification for a fixed number of arithmetic donor templates would connect the sparse theory here to the general aligned-fragment rank of the preceding report.

[write, batch 66] The preceding report's rank is Part I's ρ_L (Definition 3.1); for one-marker seeds it equals the number of ones (Theorem 34.3 and the remark after it).

Question 42.8 (Synthesis of small obstructions). For sparse masks, a pair of compatible interior velocities certifies non-context-freeness. Can such a pair be transformed automatically into a smallest or simplest explicit non-context-free exact-two-marker sublanguage, with a human-readable pumping certificate? For general Presburger masks, is there a similarly bounded family of geometric obstructions, or can every fixed-size obstruction test fail?

Question 42.9 (Proof-carrying arithmetic language analysis). Can the ray analyzer emit certificates checkable by a small verified kernel: CRT witnesses, maximum-clique upper and lower bounds, finite exception checks, and the exact depth? Maximum-clique lower bounds are short; matching upper bounds may not be. A useful architecture would separate a fast untrusted solver from exact certificate checking, and would expose nonregular and non-context-free certificates independently of the difficult optimum calculation.

43 Conclusion

The coordinate hull retains much more decidable structure than an arbitrary language description. For context-free seeds, its genuine position supports are Presburger relations, and this makes full-closure comparison and regularity effective. The regularity mechanism is the classical finite-row principle, used here with a precise constructive language interface.

For one-marker seeds, the interaction between syntax and generation is especially sharp. Uniformly bounded support is exactly finite stabilization. In that regime, endpoint geometry and at most one co-occurring interior velocity are precisely the regular and linear context-free cases; two distinct co-occurring interior velocities cause an immediate context-free failure. At the same time, the exact transient length is controlled by a clique number and is computationally hard even inside the regular endpoint case.

These results separate growth, language class, and stabilization depth rather than treating them as proxies for one another. They also provide concrete next steps: larger donor patterns, nonsparse arithmetic masks, explicit-automata complexity, and formally checked arithmetic certificates. [Added 30 September 2026, batch 66: Of these, the explicit-automata complexity has since been settled for NFA input in Part II; see the note after Question 42.3.]

44 Proof audit and boundary cases

[write, batch 66] This section and the next were the manuscript's Appendices A and B.

Empty words and empty support. The hull preserves the seed's empty-word bit. In the one-marker model, 0^* supplies ε and all-zero words at every length. If R is empty, $K(R) = 0$ and depth is zero, not a logarithm of zero.

Finite points and short lengths. A finite point may increase the exact maximum support at its length; formula (70) checks those lengths separately. Finite changes to a language do not affect membership in the regular, context-free, or linear context-free classes, so they do not affect the velocity trichotomy. The class proof nonetheless keeps finite exceptions explicitly in its normal form.

Coincident rays and crossings. Identical affine functions count as one position when simultaneously active, even with different congruence domains. Distinct affine functions may coincide at one exceptional length; they are eventually distinct along a common progression. The graph definition handles both without assuming all positions are distinct at all lengths.

No inference from a bad subset alone. A language can contain a non-context-free sublanguage and still be regular. The proofs use intersection with a regular marker-count slice, then the finite-union gap-ray pumping lemma. This is why additional rays cannot hide the obstruction in the sparse setting.

No inference from mere velocity inequality. Different interior velocities must occur at common lengths. The CRT condition is necessary; Example 37.3 is an explicit counterexample when it is omitted.

No accidental complexity transfer. The NP/coNP statements use binary ray input and a variable threshold. They do not imply the same bounds for expanded grammars, explicit DFAs, or a fixed generation. The general CFG regularity theorem asserts only decidability and effective synthesis.

A note on complete-period testing. A finite prefix alone cannot validate an arithmetic-ray maximum: a compatible collection may first coexist at a very large CRT solution. The verification script derives a common period and a threshold past starts and crossings for each small sampled instance, then scans a complete period. The symbolic clique proof is what handles arbitrarily large inputs without that scan.

45 Package and reproducibility

Path	Contents
article.tex, article.pdf	This standalone source and its compiled article; bibliography is internal.
code/crossover_masks.py	Exact ray analysis, graph realization, and uniform marker family.
code/verify.py	Reproducible exact finite audit.
data/verification.json	Machine-readable result and per-group counts.
data/verification.txt	Captured console output of the run.
examples/	Small ray inputs and their recorded analysis.
notes/provenance.md	Repository pin, literature sources, and bounded novelty statement.
notes/proof-audit.md	Main dependencies and proof obligations.
README.md, Makefile	Build and execution instructions.

The code requires Python 3.10 or later and no third-party packages. The article uses standard pdfL^AT_EX packages; build it with `latexmk -pdf article.tex`. Running the verifier overwrites the two verification receipts only if console output is explicitly redirected to the text receipt; the script itself writes the JSON receipt. Work in a copy to preserve the delivered record. No repository files were edited in preparing this package.

[write, batch 66] *Shipped layout*. The table above describes the delivered package. In this report the files are shipped, byte-identical, as follows; the manuscript and its PDF are printed or replaced by this report, the delivery README by the report’s README, and the checksum list SHA256SUMS was verified and retired.

Shipped path	Delivered as
code/03-closure-classification-crossover_masks.py	code/crossover_masks.py
code/03-closure-classification-verify.py	code/verify.py
code/03-closure-classification-Makefile	Makefile
data/03-closure-classification-verification.json	data/verification.json
data/03-closure-classification-verification.txt	data/verification.txt
data/03-closure-classification-artifact-validation.json	data/artifact-validation.json
data/03-closure-classification-<name>.json,	examples/<name>.json,
...-<name>.result.json	examples/<name>.result.json, for the five names endpoint_cycle5, finite_point_exception, five_markers, incompatible_velocities, two_markers
03-closure-classification-provenance.md	notes/provenance.md
03-closure-classification-proof-audit.md	notes/proof-audit.md

The shipped Makefile still uses the delivered layout (its `check` target runs `code/verify.py`, which in this report is Part I’s verifier) and must not be used from the report directory. The `pdf_sha256` and `tex_sha256` values in the artifact record identify the delivered PDF and source, not this report’s files. To rerun the audit, copy the two programs into a scratch directory under their delivered names and run the verifier there:

```
mkdir -p run/code run/data run/examples
cp code/03-closure-classification-verify.py run/code/verify.py
cp code/03-closure-classification-crossover_masks.py run/code/crossover_masks.py
cd run && python code/verify.py > data/verification.txt
```

When this Part was written the rerun printed PASS with 12,867 checks; its JSON receipt differed from the shipped one only in the recorded Python version (3.14.4 instead of 3.13.5) and, on Windows, in CRLF line endings, and the five example analyses, written with `--output`, reproduced the shipped `.result.json` files byte for byte apart from CRLF line endings.

A The classical universality input

[Added 30 September 2026, batch 66: Appendices A–D belong to Part I and are printed after Parts II and III so that no letter or number of Part I moved.]

For completeness, here is the standard route to the co-r.e.-completeness used in Section 4. The foundational undecidable problem is Post correspondence [7]. Take two morphisms $h, g : I^* \rightarrow A^*$ and ask whether some $x \in I^+$ satisfies $h(x) = g(x)$. The set of positive instances is recursively enumerable, and the usual effective computation-history reduction gives r.e.-completeness.

Use disjoint alphabets I, A and a separator $\$$. Among encodings $x\$y^R$, where $x \in I^+$ and $y \in A^*$, the language

$$\{x\$y^R : y \neq h(x)\}$$

is context-free. A pushdown automaton pushes the successive output blocks of h while reading x , then compares the reversed candidate output against the stack. It accepts a symbol mismatch, a short candidate, or a long candidate; the regular encoding conditions can be imposed separately. The analogous language for g is also context-free. Their union, together with all malformed encodings, is universal exactly when the correspondence instance has no solution. This proves co-r.e.-hardness of grammar universality. Its complement is r.e. because grammar membership is decidable and nonmembers can be searched for by length.

To obtain a binary alphabet, map each letter of the finite encoding alphabet to a distinct binary block of one common length ℓ . For a language H over that alphabet, set

$$H' = c(H) \cup (\{0, 1\}^* \setminus c(A_0^*)),$$

where A_0 denotes the entire encoding alphabet. The image $c(H)$ is context-free and $c(A_0^*)$ is regular. The injective fixed-length code gives $H' = \{0, 1\}^*$ exactly when $H = A_0^*$. This is a reduction for universality; as Section 11 emphasizes, it is *not* automatically a reduction preserving crossover.

B Why the positional counts are eventually affine

The special case of Presburger counting needed here also has a direct one-dimensional explanation. After quantifier elimination, a predicate in variables (n, i) is a Boolean combination of linear inequalities and fixed modular congruences. We also impose $0 \leq i < n$.

Refine the residue class of n modulo all congruence moduli, and refine the residue class of i similarly. On these classes, congruences have constant truth values. Each nontrivial inequality in i changes truth value at a rational-affine threshold in n . The order of finitely many such thresholds is fixed once n is sufficiently large: two affine functions either agree identically or their order changes at most once.

Thus the allowed positions eventually form a fixed finite union of intervals between affine thresholds, possibly with selected endpoint positions, in each fixed residue class of i . Counting that class inside such an interval is a difference of floor or ceiling functions of rational-affine expressions. Refine the period of n to absorb the finitely many denominators. Each such count then is affine on a progression. Finite union and disjoint refinement preserve that property.

This argument explains both effectivity and the degree-one conclusion of Lemma 8.2; the general counting theorem of Woods supplies a systematic framework. Since counts are nonnegative for all sufficiently large parameters and integer-valued, their progression increments are nonnegative integers. That final observation is what makes the geometric factors in Theorem 8.3 polynomials rather than Laurent expressions.

C A compact proof-dependency map

Result	Essential inputs
Exact contraction and generation	Definition of aligned crossover; restriction and pairing of consecutive intervals. No grammar theorem.
One-step CFG undecidability	One-cut guard repair; classical fixed-binary CFG universality.
Eventual CFG undecidability	Exact generation; disjoint boundary intervals for forbidden blocks; effective CFL union, concatenation, and star.
Regular-input stabilization	Exact generation; explicitly constructed distance automaton; classical limitedness decidability.
Rational weighted hull series	Effective Parikh semilinearity; one-parameter Presburger counting; coordinate-product enumeration.
Non-CFL linear-seed example	Explicit grammar; elementary position inequalities; CFL pumping lemma and intersection with a regular language.
Minimal finite-stage recurrence	Exact binomial count; degree of a polynomial; reduced rational denominators.

D [write] Provenance of this report

One manuscript. This report is built from a single manuscript, manuscript 02 of batch 44 of ProveIt’s incoming-report intake: *Aligned Fragments and Constrained Crossover*, dated 29 September 2026, delivered as the archive `ProveIt_Constrained_Crossover.zip`, which arrived in commit `ae28ea2db` and was placed in commit `203016015`. Its pin is ProveIt commit `9b24a3a8d545af9624f6ac455f5b548be62818b6` (Section 1). The manuscript contributed everything outside the text marked [write]: every result, proof, example, remark, question, limitation and the Lean plan, all printed. The package’s audit notes and verifier are shipped unchanged beside this article (Section 10.2 lists the paths). Its 23-page US-letter PDF and its README are not shipped; they survive in the arrival commit. [Added 30 September 2026, batch 66: this describes Part I; Parts II and III come from two further manuscripts (last paragraph of this appendix).]

What the write added. Everything marked [write]: the title-page note; in Section 1, where Hughes poses the question, the state of the sibling source, and Sections 1.4 and 1.5; the note after Remark 2.3; the notes on the shipped layout and on formal status in Section 10; the note on Hughes’s Sections 13–14 after the research questions; and this appendix. The locations in Hughes’s preprint were checked against its arXiv v1 PDF when the report was written.

Choices. There was no merge, so no result had to be chosen over another. Every label received the prefix `ccc:`, and every reference was updated with it. Labels were added only on new text and on Question 11.1 and Section 11.1; no theorem, section or equation number of the manuscript changed. The page size stays US letter, as delivered. No symbol was renamed; the `shuffle` package was added for the symbol $\sqcup$ of the sibling report.

[Added 30 September 2026, batch 66: Two further manuscripts.] This appendix describes the report as written in batch 44, which is Part I. On 30 September 2026 two manuscripts of batch 66, both written against Part I at ProveIt commit `b8b0fa218` (where Part I’s text was the one printed here), were added as Parts II and III: manuscript 01, *Finite Crossover Stabilization Is PSPACE-Complete* (archive

ProveIt_Crossover_Stabilization.zip), and manuscript 03, *Decidable Regularity and Exact Stabilization of Crossover Closures* (archive ProveIt_Crossover_Classification.zip). Both arrived in commit 9ad899cbe and were placed in commit 4fee1cd07. The report is therefore built from three manuscripts. Sections 13.1 and 29.1 give each one’s provenance and what it contributed. The choices made were these. The two were kept as separate Parts, not merged, because they share no main theorem; the only overlap is a reproof of Part I’s coordinate hull, printed once (Theorem 3.3) with the two reproofs kept as marked second and third routes (Proposition 15.2, Lemma 31.3). Part III’s Lemma 31.4, which restates Part I’s Lemma 8.1 without citing it, now credits it, and its proof is kept as a second route. Every label of the two manuscripts received the prefix `ccc:ps:` or `ccc:cl:`; their appendices became sections, because the appendices belong to Part I; ten symbols were renamed where they clash (Sections 13.3 and 29.3); the bibliographies were merged (each manuscript’s entry for Part I replaced by internal references, its Hughes entry identified with Part I’s, and Part III’s Esparza et al. entry with [3]). Four labels were added to existing text of Part I (`ccc:q:regular`, `ccc:q:grammar`, `ccc:q:series`, `ccc:q:restricted` on its questions) and dated notes were added after Corollary 7.4, after four of its questions, in its Sections 10–11, in its conclusion, on the title page, and here; no existing label, theorem, section, equation or question number of Part I changed.

References

- [1] Charles E. Hughes. *Undecidability of Adjacent Equality for Insertion, Shuffle, and Crossover Language Operations*. arXiv:2608.27755v1, 27 August 2026. Definition on printed p. 2; the constrained one-step question on printed p. 3. [write] Section 2 (definition), Section 4 (the question, pp. 3–4), Section 14 (open directions, p. 12). <https://arxiv.org/abs/2608.27755v1>.
- [2] Vladimir Reshetnikov and ProveIt contributors. *ProveIt: Gaps in Bounded-Shuffle Hierarchies*, Parts I–II. Inspected 29 September 2026 at the revision and path specified in Section 1. AI-assisted research drafts. [write] The sibling report *insertion-degree-spectra* of this collection (Section 1.4). <https://github.com/VladimirReshetnikov/ProveIt>.
- [3] Javier Esparza, Pierre Ganty, Stefan Kiefer, and Michael Luttenberger. Parikh’s theorem: A simple and direct automaton construction. *Information Processing Letters* **111**(12) (2011), 614–619. <https://arxiv.org/abs/1006.3825>. doi:10.1016/j.ipl.2011.03.019.
- [4] Kevin Woods. Presburger arithmetic, rational generating functions, and quasi-polynomials. *Journal of Symbolic Logic* **80** (2015), 433–449. <https://arxiv.org/abs/1211.0020>. doi:10.1017/jsl.2015.4.
- [5] Daniel Kirsten. Distance desert automata and the star height problem. *RAIRO—Theoretical Informatics and Applications* **39**(3) (2005), 455–509. https://www.numdam.org/item/ITA_2005__39_3_455_0/. doi:10.1051/ita:2005027.
- [6] Markus Krötzsch, Tomáš Masopust, and Michaël Thomazo. Complexity of universality and related problems for partially ordered NFAs. *Information and Computation* **255**, Part 1 (2017), 177–192. Preprint arXiv:1609.03460v2. <https://arxiv.org/abs/1609.03460>.
- [7] Emil L. Post. A variant of a recursively unsolvable problem. *Bulletin of the American Mathematical Society* **52** (1946), 264–268.
- [8] Anthony Widjaja To. *Unary finite automata vs. arithmetic progressions*. *Information Processing Letters* 109(17) (2009), 1010–1014. Corrected proof in Section 3 of arXiv:0812.1291v1. <https://arxiv.org/abs/0812.1291>. DOI: 10.1016/j.ipl.2009.06.005.
- [9] Jui-Yi Kao, Narad Rampersad, and Jeffrey Shallit. *On NFAs Where All States are Final, Initial, or Both*. *Theoretical Computer Science* 410(47–49) (2009), 5010–5021. Lemma 6 in arXiv:0808.2417v2, revised 3 July 2009. <https://arxiv.org/abs/0808.2417>. DOI: 10.1016/j.tcs.2009.07.049.
- [10] Walter J. Savitch. *Relationships between nondeterministic and deterministic tape complexities*. *Journal of Computer and System Sciences* 4(2) (1970), 177–192. DOI: 10.1016/S0022-0000(70)80006-X.

- [11] Rohit J. Parikh. On context-free languages. *Journal of the ACM* **13**(4) (1966), 570–581. doi:10.1145/321356.321364.
- [12] Seymour Ginsburg and Edwin H. Spanier. Semigroups, Presburger formulas, and languages. *Pacific Journal of Mathematics* **16**(2) (1966), 285–296. doi:10.2140/pjm.1966.16.285.
- [13] Pablo Barceló, Chih-Duo Hong, Xuan-Bach Le, Anthony W. Lin, and Reino Niskanen. Monadic decomposability of regular relations. In *ICALP 2019*, LIPIcs **132**, article 103. doi:10.4230/LIPIcs.ICALP.2019.103.
- [14] Matthew Hague, Anthony W. Lin, Philipp Rümmer, and Zhilin Wu. Monadic decomposition in integer linear arithmetic. In *IJCAR 2020, Part I*, LNCS **12166**, 122–140. doi:10.1007/978-3-030-51074-9_8. Extended version: <https://arxiv.org/abs/2004.12371>.
- [15] Moses Ganardi and Marin Ricos. Constructing small monadic decompositions in Presburger arithmetic. In *LICS 2026*, LIPIcs **380**, 47:1–47:22. Published 9 July 2026. doi:10.4230/LIPIcs.LICS.2026.47.
- [16] Danny Raz. Length considerations in context-free languages. *Theoretical Computer Science* **183**(1) (1997), 21–32. doi:10.1016/S0304-3975(96)00308-8.
- [17] Richard M. Karp. Reducibility among combinatorial problems. In R. E. Miller and J. W. Thatcher (eds.), *Complexity of Computer Computations*, Plenum, 1972, 85–103. doi:10.1007/978-1-4684-2001-2_9.
- [18] J. Barkley Rosser and Lowell Schoenfeld. Approximate formulas for some functions of prime numbers. *Illinois Journal of Mathematics* **6**(1) (1962), 64–94. doi:10.1215/ijm/1255631807.